

基于 LogisticGAM 的 NIPT 的时点选择与胎儿的异常判定

摘 要

无创产前检测通过分析孕妇血浆中胎儿游离 DNA 评估染色体异常，其准确性受孕周、孕妇 BMI 及个体差异影响。本研究基于大量样本，提出合理分组与时点优化策略，以提高检测准确率、缩短治疗窗口并降低漏诊误诊风险。

针对问题一，我们首先对于数据中的缺失值，异常值及部分变量进行**数据预处理**，接着进行**描述性统计和可视化处理**，数据呈现出非线性、近似正态，弱相关性和独立性的特点，为此我们引入固定和个体随机效应构建**广义加性混合模型（GAMM）**求解得到边际 R²、条件 R² 和 RMSE 分别为 **0.177**、**0.806** 和 **0.012**，说明个体差异解释大量变异且模型准确率较高，通过 **VIF 共线性检验**并与 OLS,LME 模型比较得到无多重共线性且模型性能优越，接着我们对于整体，成组效应，单个系数分别使用了**最大似然比检验**，**Wald 检验**，**t 检验**和 **z 检验**分析显著性得到上述 p 值均小于 0.001，具有极强显著性。

针对问题二，我们创新性引入**多维聚类-决策树区间映射模型**，通过无监督多维聚类利用所有可用特征，并以决策树将簇映射为互不重叠的 BMI 区间，借助 **Bootstrap 稳定性检验**与**敏感性分析**保障分组的可重复性与鲁棒性。接着利用 Y 染色体浓度构造二分类达标特征将 Logistic 与 GAM 结合构建 **LogisticGAM 模型**，利用**两种最佳 NIPT 时点准则**求解得到 BMI 分组和 NIPT 最佳时点（周）为：**[20.70,32.23]-10.94**、**[32.23,34.15]-11.69**、**[34.15,34.70]-12.37**、**[34.70,35.95]-13.23**、**[35.95,46.88]-23.05**。接着分别对于 BMI 分组和 NIPT 最佳时点求解施加多种扰动，得到平均变化不超过 10%，模型鲁棒性强。最后通过多模型对比得到 LogisticGAM 模型 AUC=0.66，与其他模型相比**速度快，精度较高**的特点。

针对问题三，我们扩充变量使用多维聚类-决策树区间映射模型，并从核心变量，临床扩展变量和测序扩展变量三个维度增加变量构建**多维广义加性模型（Multi-LogisticGAM）**，解得 BMI 分组和 NIPT 最佳时点（周）为：**[20.70,31.91]-12.21**、**[31.91,33.83]-20.83**、**[33.83,35.55]-20.73**、**[35.55,36.57]-23.92**、**[36.57,46.88]-26.34**。接着从更多维度施加扰动，NIPT 最佳时点**表现稳健**，BMI 分组对数据变化**略为明显**。最后多模型对比显示模型为 **AUC=0.88**，较问题二和其他模型优势明显。

针对问题四，首先利用散点图分析不同变量间的关系，接着从染色体信号，测序质量和个体差异三个维度构建特征，建立**单染色体 Logit 模型**，同多模型比较下表现最优模型 AUC 和准确率分别为 0.689 和 73.61%，最后我们针对不同的实际情况给出 **GC 和 Z 值赋权判别法**，**OOF 阈值法**，**决策树判别法**，以便灵活使用。

关键词：多维聚类-决策树区间映射模型；LogisticGAM 模型；单染色体 Logit 模型

一、问题重述

1.1 问题背景

无创产前检测 (NIPT) 是通过采集孕妇血液、检测胎儿游离 DNA 片段, 来判定早期胎儿是否存在 21 号 (唐氏综合征)、18 号 (爱德华氏综合征) 或 13 号 (帕陶氏综合征) 染色体异常, 以明确胎儿健康状况。其准确性的核心取决于胎儿性染色体浓度 (男胎 Y 染色体浓度需达到或高于 4%, 女胎 X 染色体浓度需无异常) 和检测时点的风险等级 (12 周以内 (早期) 发现异常的干预风险较低, 13-27 周 (中期) 风险较高, 28 周以后 (晚期) 风险极高)。因此, 做到 “尽早检测” 与 “结果准确” 的平衡是成功应用 NIPT 的关键。

同时实践表明男胎 Y 染色体浓度与孕妇孕周数、身体质量指数 (BMI) 等紧密相关。但由于忽略了孕妇年龄、孕情等个体差异, 一般的根据 BMI 简单分组并设定统一检测时点的方法准确性不高, 还可能因时点不当导致测序失败或缩短治疗窗口期, 增加潜在风险。因此为了尽可能减少检测风险并提高准确度, 有必要建立合适的模型分析胎儿染色体浓度与相关指标的关系、优化男胎孕妇 BMI 分组及最佳检测时点, 并探索女胎染色体异常的判定方法, 提升检测可靠性。

1.2 问题要求

附件所给出的是大多为高 BMI 孕妇的 NIPT 数据, 包含孕妇年龄、BMI、身高、体重、孕周、检测时间、抽血次数、原始测序总读段数、比对比例、不同特殊染色体的 Z 值、浓度及 GC 含量、染色体非整倍体判定结果等信息。为探究 NIPT 的最佳时点选择与胎儿异常判定方法, 现需结合实际情况与上述数据建立数学模型分析以下问题:

问题一: 建立合适的关系模型, 分析男胎 Y 染色体浓度与孕妇孕周数和 BMI 等指标的相关特性, 并检验该模型的显著性。

问题二: 鉴于男胎孕妇的 BMI 是影响胎儿 Y 染色体浓度达标时间 (浓度 $\geq 4\%$ 的最早时间) 的主要因素, 要求对男胎孕妇的 BMI 进行合理分组并给出每组的 BMI 区间和最佳 NIPT 时点以最小化孕妇潜在风险, 分析检测误差对结果的影响。

问题三: 综合考虑身高、体重、年龄等多因素、检测误差及胎儿 Y 染色体浓度达标比例 (浓度 $\geq 4\%$ 的比例), 依据男胎孕妇的 BMI 重新进行合理分组并给出每组的最佳 NIPT 时点以最小化潜在风险, 分析检测误差对结果的影响。

问题四: 由于孕妇和女胎均不携带 Y 染色体, 要求以女胎孕妇的 13 号、18 号、21 号染色体非整倍体判定结果为依据, 综合考虑 X 染色体及上述染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等因素给出女胎异常的合理判定方法。

二、问题分析

2.1 问题一的分析

针对问题一, 这是一个统计分析类题目, 我们对于数据中的缺失值, 异常值及部分变量进行数据预处理, 接着进行描述性统计并利用小提琴图, PP 图, Pearson 相关性分析可视化处理, 观察数据呈现特点数据呈现出非线性、近似正态, 弱相关性和独立性的特点, 为此我们引入固定效应和个体随机效应构建广义加性混合模型 (GAMM) 求解, 并通过多模型对比和 VIF 共线性检验检验模型性能和数据特征, 最后对于整体, 成组效

应，单个系数分别使用了最大似然比检验，Wald 检验，t 检验和 z 检验分析显著性。

2.2 问题二的分析

针对问题二，这是一个回归类题目，我们通过无监督多维聚类利用所有可用特征，并以决策树将簇映射为互不重叠的 BMI 区间创新性引入多维聚类-决策树区间映射模型利用 Y 染色体浓度构造二分类达标特征将 Logistic 与 GAM 结合构建 LogisticGAM 模型，利用两种最佳 NIPT 时点准则求解，接着分别对于 BMI 分组和 NIPT 最佳时点求解施加多种扰动分析模型求解结果的敏感性。

2.3 问题三的分析

针对问题三，这是在问题二的基础上进一步改进扩充变量使用多维聚类-决策树区间映射模型，并从核心变量，临床扩展变量和测序扩展变量三个维度对于增加变量构建多维广义加性模型（Multi-LogisticGAM），接着从更多维度施加扰动，最后通过多模型对比分析模型优劣。

2.4 问题四的分析

针对问题四，首先利用散点图分析不同变量间的关系，接着从染色体信号，测序质量和个体差异三个维度构建特征，分别对于 21 号，18 号和 13 号建立单染色体 Logit 模型。最后根据回归结果结合实际情况提出相应模型。

文章总体思路图如下：

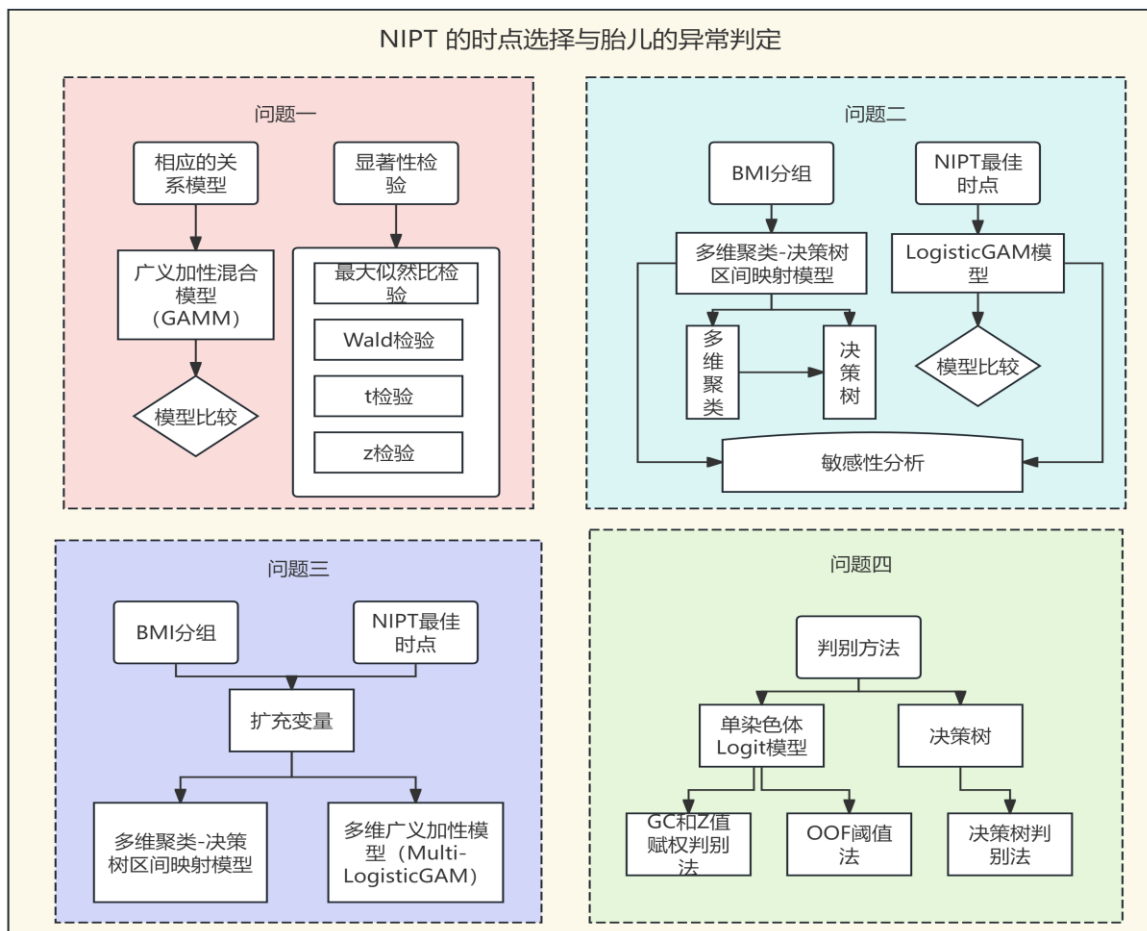


图 1 文章总体流程图

三、模型假设

- 1.假设除 BMI、孕周、测序质量指标（如 GC 含量、读段比对率）外，其他因素（如孕妇吸烟史、基础疾病）对 Y 染色体浓度达标时间的影响可忽略。
- 2.假设 Y 染色体浓度 $\geq 4\%$ 为“达标”的判定阈值适用于所有孕周（10-25 周），且达标状态可直接反映 NIPT 检测准确性。
- 3.假设不考虑染色体异常的遗传背景（如家族史）和环境诱因（如辐射、化学物质暴露）等其他因素。

四、符号说明

符号	含义	单位
$week_c$	做中心化处理后的孕周	周
bmi_c	做中心化处理后的 BMI	Kg/m ²
age_c	做中心化处理后的年龄	岁
ifv_{flag}	IFV 妊娠方式（0-1 变量）	/
$Y_{frac,i,j}$	第 i 个孕妇第 j 次检测的 Y 染色体浓度	/
$achieved$	Y 染色体浓度是否达标（0-1 变量）	/
p_0	Y 染色体浓度达标概率阈值	/
$Z_{13}/Z_{18}/Z_{21}/Z_X$	常染色体 Z 值	/

五、数据预处理

5.1 时间格式统一

- 附件数据中次月经时间与检测日期分别出现“2023/2/1”和“20230531”两种格式，且男胎数据中的末次月经时间还额外存在“2022-12-19”的格式差异，为消除格式不一致对后续分析的干扰，我们将所有时间记录统一转换为标准 `datetime` 格式，为准确计算实际孕周、匹配检测时间与胎儿发育阶段提供可靠的时间维度基础；
- 同时，为更好量化分析胎儿 Y 染色体浓度与孕周的关联特性和最佳检测时点的确定，我们将原“周数+天数”形式记录的检测孕周统一转换为以“周数”为单位的连续量化指标，确保孕周数据的一致性与可比性，为后续建模分析提供标准化的数据支撑。

5.2 变量处理

5.2.1 缺失值处理

经检测发现“男胎检测数据”中末次月经时间存在 12 个缺失值，针对该缺失情况，我们采用“检测日期减去检测孕周天数”的方式进行填充，这考虑到检测孕周与检测日期和末次月经时间较为接近，且更好地保留了孕妇信息。

经过检测数据我们仅发现孕妇代码为 B044 的样本 BMI 值存在缺失，我们对此利用对应的身高体重将其填充。

5.2.2 变量利用

由于末次月经时间与检测日期的记录属于日期类型的数据无法作为变量直接用于后续的讨论分析，在此处我们引入“检测日期与末次月经时间差”将上述转化为定量数据，用以表征检测时的胎儿发育阶段。

但是，考虑到实际情况，我们对“检测日期与末次月经时间差”和“检测孕周天数”进行误差分析对比，结果显示男胎、女胎检测数据的两变量的平均相对误差分别为 3.50% 和 4.26%，误差水平均较低且差异微小，这意味着二者对于后续分析贡献接近，为简化数据维度，我们在下文中不再考虑“检测日期”与“末次月经时间”，这也印证了我们上文中缺失值处理的合理性。

5.2.3 异常值处理

在异常数据处理中，考虑到身高年龄等作为孕妇基础生理指标在孕期短时间内的取值相对稳定，且有较为明确的生理学合理数值波动范围；而染色体浓度、Z 值、GC 含量等其他数值统计型变量由于检测操作误差等因素的影响可能会出现异常值，但其总体波动应符合正常的统计学规律。因此我们依据不同变量的特性采取差异化处理策略：

● 年龄和身高异常值处理

在进行孕妇个体基础信息的一致性检查时发现年龄与身高数据存在异常波动：其中，男胎和女胎数据中共计 4 位孕妇在孕期不同检测时间点的年龄记录相差 2 岁，另有 1 位孕妇的年龄记录相差 8 岁；身高数据方面，有 2 位孕妇孕期不同检测时间点的记录相差 2.0cm，1 位相差 3.0cm，1 位相差 5.0cm。考虑到年龄与身高作为孕妇稳定的生理指标，在怀孕期间不应出现此类显著差异，推测该异常为记录误差所致。因此我们对上述异常数据采用对应孕妇个体数据中的众数替代。

● GC 含量处理

题目中提到正常 GC 含量范围为 40%~60%，但是经检测男胎检测数据异常 GC 含量有 451 条，最大偏离 1.38 个百分点；女胎检测数据异常 GC 含量有 220 条，最大偏离 3.23 个百分点。实际上这几乎占据了全部样本 1/3，且偏离程度较低，我们在此处暂时不对 GC 含量的异常值进行处理。

● 其余定量变量处理

由图 2 和图 3 可以看出重复读段的比例、GC 含量和 X 染色体 Z 值等变量均存在超出 IQR 阈值范围的异常值，对于此类异常值，我们均采用硬阈值裁剪（对于不在阈值范围内的数值，若小于阈值下界则取下界值，若大于阈值上界则取上界值）的方式进行处理，保证变量合理性。

六、数据侧写

为更好地分析和理解数据，我们利用附件中的身高体重、检测孕周、GC 含量等检测数据指标进行数据侧写来观察不同胎儿指标的数据分布和变化情况。

6.1 胎儿检测指标描述性统计分析

我们首先利用 Python 分别统计计算得到男胎和女胎的检测变量描述性统计指标表如下表 1 和表 2 所示：

表 1 男胎检测数据描述性统计变量表

变量名	均值	标准差	中位数	偏度	峰度	IQR 阈值
年龄	28.93	3.66	29.00	0.61	1.16	[21.00,37.00]

身高	161.06	5.23	161.00	-0.11	-0.09	[147.50,175.50]
体重	83.89	9.92	82.91	0.81	1.93	[58.89,107.20]
检测孕周/周	117.92	28.53	112.00	0.61	-0.79	[22.50,210.50]
孕妇 BMI	32.29	2.97	31.81	0.86	2.65	[24.63,39.50]
GC 含量	0.40	0.00	0.40	0.51	3.29	[0.39,0.41]
X 染色体 Z 值	0.33	1.29	0.24	0.61	2.33	[-2.85,3.47]
Y 染色体 Z 值	0.16	1.31	0.02	0.87	2.41	[-2.96,3.15]
Y 染色体浓度	0.08	0.03	0.08	0.71	1.01	[-0.02,0.17]
X 染色体浓度	0.06	0.04	0.06	0.43	0.82	[-0.05,0.16]

表 2 女胎检测数据描述性统计变量表

变量名	均值	标准差	中位数	偏度	峰度	IQR 阈值
年龄	30.00	4.32	30.00	0.55	0.55	[19.50,39.50]
身高	161.28	5.22	161.50	-0.19	0.54	[147.50,175.50]
体重	83.82	9.56	82.00	0.84	1.42	[57.50,109.50]
检测孕周/周	128.42	30.59	124.00	0.09	-1.27	[21.00,245.00]
孕妇 BMI	32.16	2.95	31.48	1.49	3.35	[25.24,38.23]
GC 含量	0.40	0.00	0.40	0.86	28.35	[0.39,0.41]
X 染色体 Z 值	0.48	1.30	0.53	0.20	8.34	[-2.72,3.62]
X 染色体浓度	-0.01	0.02	-0.01	0.53	4.26	[-0.05,0.04]

观察表 1 和表 2 我们可以发现，男胎和女胎两组检测数据在年龄、身高、体重等基础生理指标的统计指标差异不大，而在检测孕周方面女胎组均值显著高于男胎组，两组的染色体浓度差异也较大；GC 含量上女胎组的偏度和峰度更高，分布形态差异明显。

为了进一步观察数据的分布情况，我们对男胎和女胎检测数据变量进行 Z-score 标准化来消除不同指标检测指标的量纲差异，具体公式如下：

$$Z = \frac{X - \mu}{\sigma}, \quad (1)$$

其中 X 为胎儿检测样本中各指标变量值， μ 和 σ 为不同指标变量各自均值和标准差。接着我们利用 Python 绘制得到男胎和女胎检查数据的小提琴图 2 和图 3：

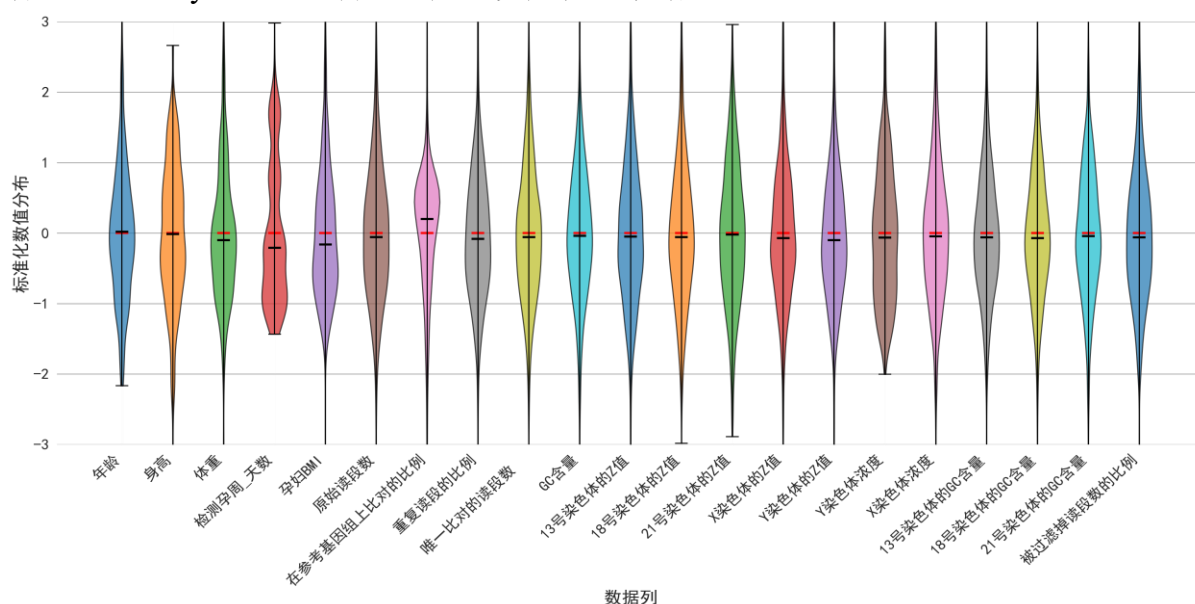


图 2 男胎检测数据各指标分布小提琴图

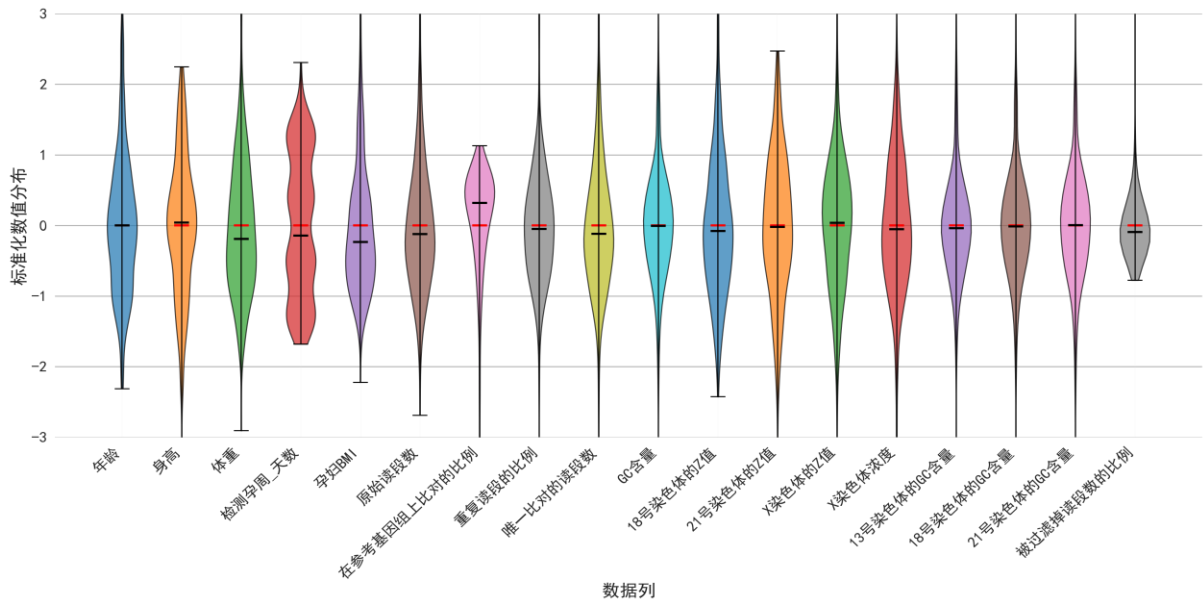


图 3 女胎检测数据各指标分布小提琴图

观察上述图表可以发现，男胎和女胎孕妇在临床指标相关分布上呈现出不同的分布范围和偏态趋势，如男胎组的检测孕周相比女胎组的分布范围更小，整体数据取值分布更分散，反映胎儿性别对母体相关检测指标的潜在影响；对于 13、18、21 号常染色体相关分子指标（如各染色体 Z 值、GC 含量等），虽整体分布均趋于对称且集中，但男胎与女胎组在这些指标的峰度、四分位数间距上仍存在细微差别。这些差别提示我们性别因素也可能通过母体生理状态间接作用于相关检测指标的分布模式。

6.2 胎儿检测指标正态性分布检验

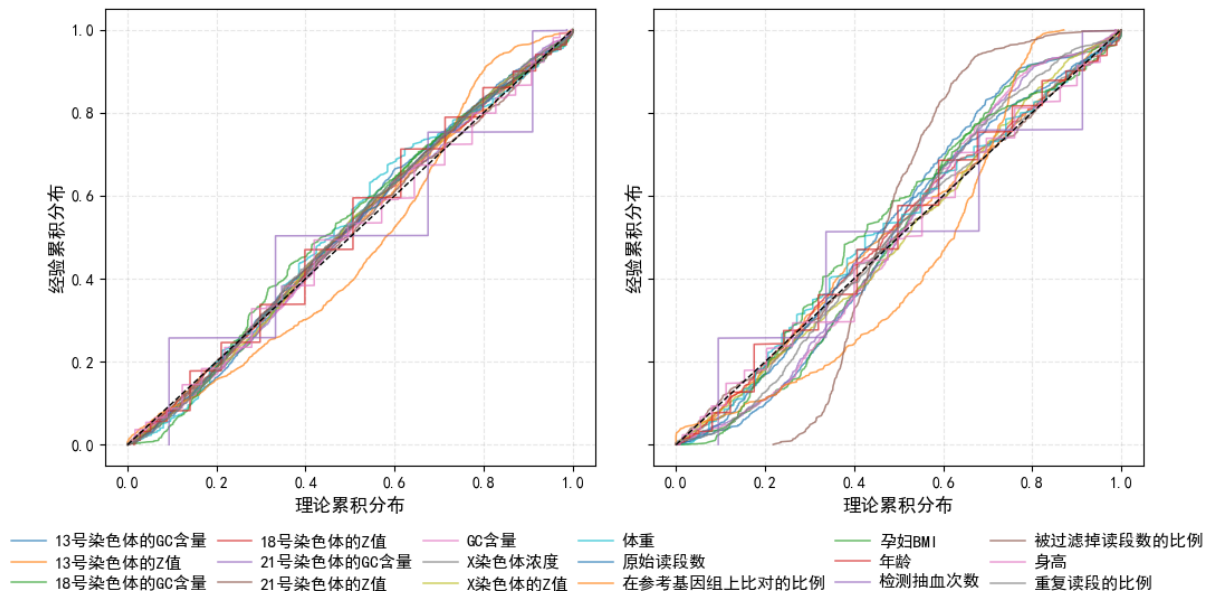


图 4 胎儿各检测指标正态性检验 P-P 图（左为男胎右为女胎）

由图 4 我们可以发现男胎和女胎基本上吻合正态分布的特征，其中男胎组中各常染色体含量、Z 值和 GC 含量等核心分子检测指标的累积分布曲线与理论正态分布对角线贴合度较好，表明其指标呈现较强的正态分布；而女胎组中体重、年龄、读段过滤比例

等临床及衍生指标的累积分布曲线与理论对角线偏离程度整体较为显著，表明其指标呈现较弱的正态分布。

6.3 胎儿检测指标 Pearson 相关性检验

根据图 4 的正态性检验我们可以发现男胎和女胎的各项 NIPT 指标的累积分布基本上与理论正态分布具有较好的贴合性。由于 Pearson 相关性分析的关键前提是变量需复查正态分布，而 Spearman 相关性分析更适用于非正态分布或有序分类变量的关联分析。因此相较于不依赖分布类型的 Spearman 相关分析，采用 Pearson 相关性分析对于题目中连续、近似正态有着更好的效果，更能准确量化指标间的线性关联程度。

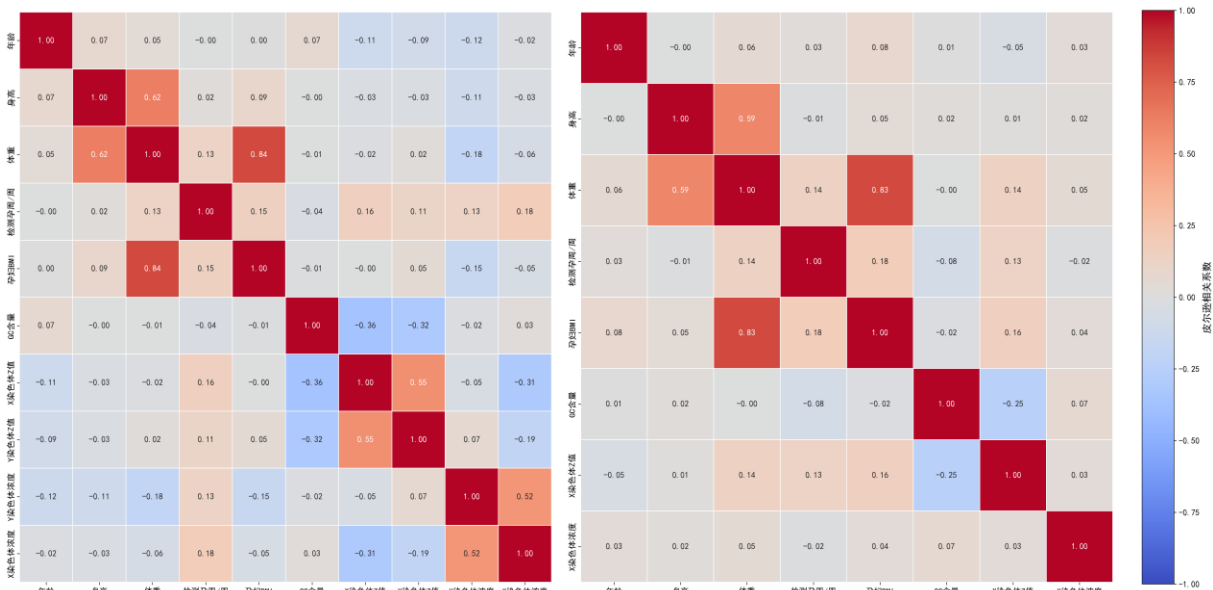


图 5 胎儿各检测指标 Pearson 相关性检验图（左为男胎右为女胎）

由图 5 我们可以发现男胎和女胎检测数据中多数变量间的线性相关性整体较弱；仅在两组中身高、体重、孕妇 BMI 等本身存在生理线性关联的指标间呈现出较弱的线性关联特征；同时男胎数据中的 X 染色体与 Y 染色体的浓度及 Z 值间还表现出一定程度的线性相关性。这些提示我们在 NIPT 检测体系中各指标变量间的内在关联更可能以非线性关系为主导。

七、 问题一模型建立与求解

问题分析：问题一是一个分析建模类问题，实际上可以转化成基于不同胎儿 NIPT 数据特性的统计建模问题，这要求我们应该选择一个兼顾母体特异性和变量潜在的非线性关系来量化分析男胎 Y 染色体浓度与孕周、BMI 等指标的关联规律的模型。同时题目中提到，某些孕妇有多次采血多次检测或一次采血多次检测的情况，这种意味着数据之间的非独立性，经计算 14.88%男胎孕妇均存在重复测量的情况，因此将其纳入模型变量中进行考虑是更加妥当的处理方式而非将其直接删去[1]。

7.1 模型建立

国内外对于处理重复多次观测的数据分析或对 NIPT 已有许多讨论：如 Chiu 等（2011）[2]利用普通最小二乘法（OLS）结合临床试验挖掘出 NIPT 在胎儿非整倍体检测中的应用价值；Bolin 与 Diggle（2020）[3]针对非高斯分布的重复测量数据，提出了一种基于多元正态方差-均值混合模型的线性混合效应模型（LME）框架打破了传统高

斯 LME 在处理偏态或厚尾数据时候的局限性。但是，上述传统的 OLS 模型对于处理上述复杂变量性能受限，而 LME 能够处理个体独立性但是对于线性性质要求较高，对于本文中的非线性，独立性的文本处理能力相对较弱。下文中我们选择利用 GAMM 通过引入固定效应和随机效应适配本文问题。

7.1.1 数据准备

● 缺失值和异常值处理

为保证样本数据完整性，我们删除“孕周”和“Y 染色体”数据列都缺失的样本数据；同时用 2%-98% 分位数限制极端值，避免存在的异常值导致的模型偏差。

● 重复测次质量筛选

根据题目设定，唯一比对读段数越高、过滤比例越低的母体测序数据质量越好。考虑到同一孕妇存在多次采血多次检测或一次采血多次检测的情况，我们按照“唯一比对读段数最大→过滤比例最小→总读段数最大”的标准排序，选择出最优测次。

● 变量中心化

为了消除不同连续变量量纲影响，使模型系数可直接用来比较效应大小，我们对年龄、检测孕周、孕妇 BMI 做中心化处理，具体公式如下：

$$week_c = week - \bar{week}, \quad (2)$$

$$bmi_c = bmi - \bar{bmi}, \quad (3)$$

$$age_c = age - \bar{age}, \quad (4)$$

其中， $week$ 、 bmi 和 age 分别对应未处理的检测孕周、孕妇 BMI 和年龄数据； $\bar{week}$ 、 $\bar{bmi}$ 和 $\bar{age}$ 分别对应各自数据的总体均值。

● 分类变量编码

为了更好地量化辅助生产方式对 Y 染色体浓度的潜在影响，适配回归模型对输入数值变量的要求，我们将 IVF 妊娠转换为 0-1 变量，具体定义如下：

$$ivf_{flag} = \begin{cases} 1 & \text{IVF 辅助生殖} \\ 0 & \text{自然妊娠} \end{cases} \quad (5)$$

7.1.2 广义加性混合模型（GAMM）建立

通过分析附件所给胎儿 NIPT 检测数据可以发现所给数据存在如下几个特点：1. 非线性：通过图 5 的 Pearson 相关性分析，我们可以得到实际上变量之间线性相关性很低，变量呈现非线性的特点；2. 独立性：由于孕妇存在重复测量的情况，变量之间呈现出独立性，这也意味着可能个体之间所体现出的随机效应更强；3. Y 染色体浓度是比例型变量：天然受限于 [0,1] 区间，符合概率的分布范围。本文引入 GAMM[4] 同时把平滑函数（处理非线性）和随机效应（处理重复测量）结合在一起，因此既能灵活拟合非线性关系，又能正确估计标准误与个体差异，具体模型如下所示：

$$Y_{frac,i,j} = \text{固定效应} + \text{随机效应}, \quad (6)$$

$$\text{固定效应} = \beta_0 + f(week_{c,i,j}) + \beta_2 bmi_{c,i,j} + \beta_3 age_{c,i,j} + \beta_4 ivf_{flag,i,j} + \beta_5 (week_{c,i,j} \times bmi_{c,i,j}), \quad (7)$$

$$\text{随机效应} = u_{0i} + u_{1i} week_{c,i,j} + \epsilon_{ij}, \quad (8)$$

其中， $Y_{frac,i,j}$ 表示第 i 个孕妇第 j 次检测的 Y 染色体浓度； β_0 为截距项表示所有自变量为 0 时的 Y 染色体浓度理论均值； β_2 、 β_3 、 β_4 和 β_5 分别是 BMI、年龄、IVF 妊娠方式和孕周×BMI 交互效应的固定效应系数（即控制其他变量不变时该变量每增加一个单位 Y 染色体浓度的平均变化量）； u_{0i} 和 u_{1i} 分别表示第 i 个孕妇的随机截距和斜率； ϵ_{ij} 表示第 i 个孕妇第 j 次检测的随机误差，服从 $N(0, \sigma^2)$ ； $f(week_{c,i,j})$ 为该模型引入的 B 样条

函数（B-spline）用来灵活拟合孕周与 Y 染色体浓度的非线性关系，具体公式表示为：

$$f(\text{week}_{c,i,j}) = \text{bs}(\text{week}_{c,i,j}, df = 4) \quad (9)$$

7.2 模型求解

在模型建立和求解的过程中我们选择的是 GAMM 模型，同时将 OLS、LME 于 GAMM 相对比，突出 GAMM 在本问上的优越性能表现。我们的模型设置如下：1.OLS 不引入随机效应，即每一条数据之间相互独立，不考虑孕妇序号的影响；2.LME 加入随机截距与随机斜率；3.GAMM 随机效应部分与 LME 一致，将 LME 中孕周项替换为 B 样条项，自由度设置为 4。对于 LME 我们先尝试随机截距，再尝试随机斜率，使用极大似然（ML）的 AIC 选优，GAMM 的随机效应格式与 LME 保持相同。

接着我们以 R^2 和 RMSE 为评价指标对各模型进行拟合优度检验得到表 3，同时得到相关回归散点图和残差 QQ 图如图 6 和图 9 所示；然后我们对三种模型进行显著性检验和多重共线性检验得到有关表 4、图 6、图 7、图 8 和图 10 所示：

表 3 各模型 R^2 和 RMSE 拟合优度检验表

模型名称	边际 R^2 （固定效应变异）	条件 R^2 （固定+随机效应总变异）	RMSE
OLS	0.095	/	0.032
LME	0.136	0.762	0.014
GAMM	0.177	0.806	0.012

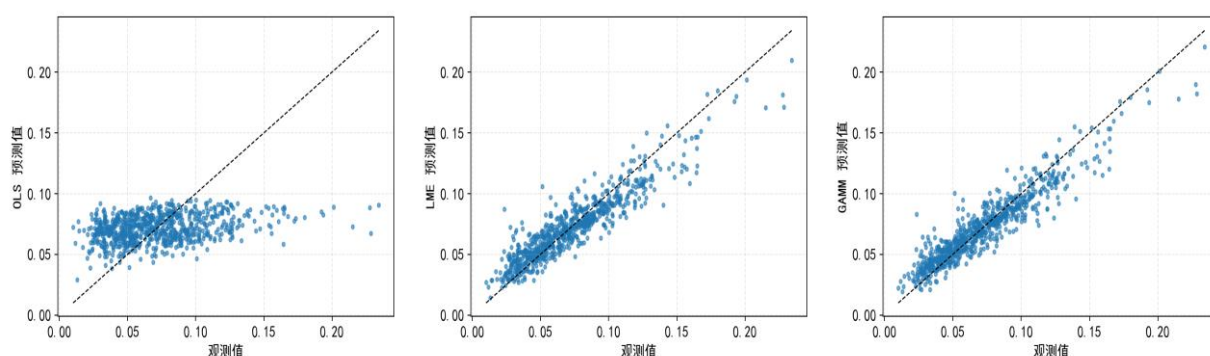


图 6 OLS、LME 和 GAMM 回归模型散点图

观察表 3 和图 6 我们可以发现，我们所使用的 GAMM 模型的边际和条件 R^2 均比另外两个模型高，能够解释更多比例的 Y 染色体浓度变异和固定效应；同时我们也可以观察到如果将所有个体视为独立样本，那么他们的边际 R^2 仅为 0.095，但是我们在引入个体因素的随机效应后，可以发现条件 R^2 可以达到 0.806，说明该模型的拟合程度大部分由个体来决定，这也意味着实际上个体差异性在有关于 Y 染色体浓度的讨论上是不可忽视的。同时 GAMM 的 RMSE 为 0.012，结合散点图发现其预测精度也优于 LME 和 OLS。

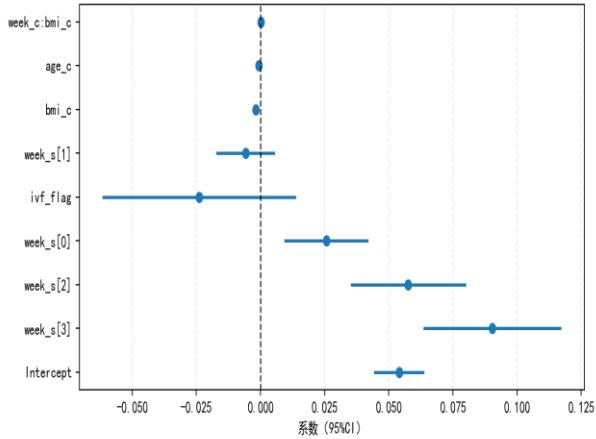


图 7 GAMM 固定效应系数森林图(95%CI)

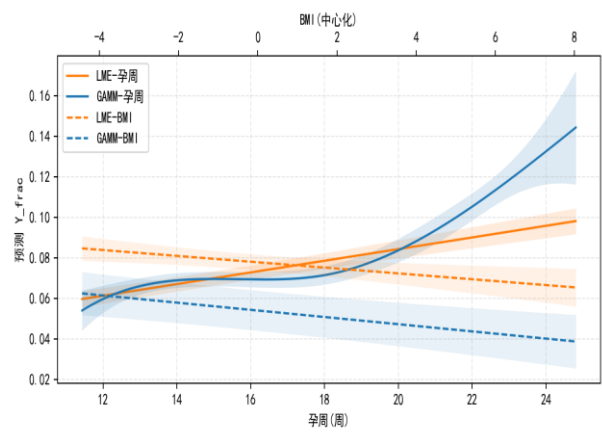


图 8 LME 和 GAMM 孕周和 BMI 效应关系图

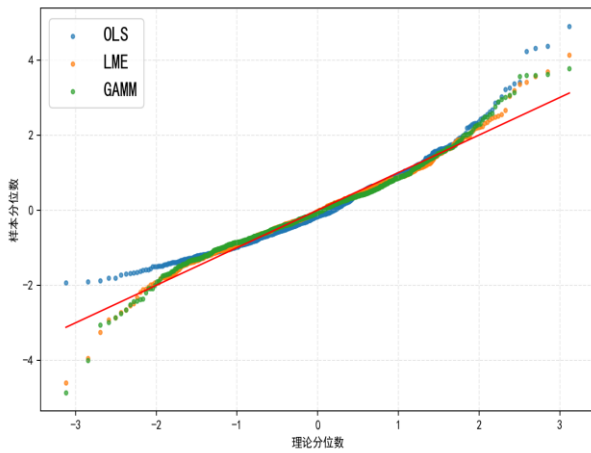


图 9 三种模型拟合残差 QQ 图

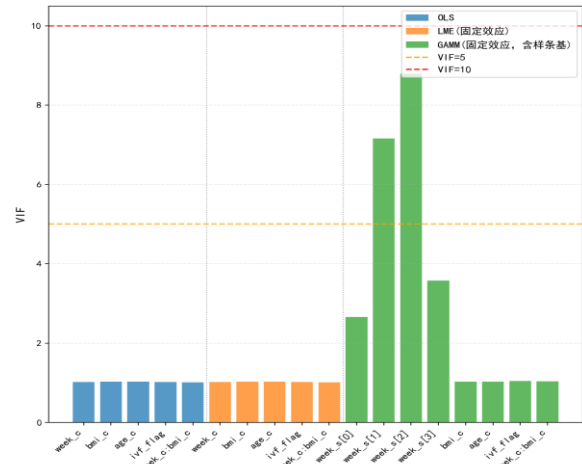


图 10 三种模型多重共线性检验 (VIF) 柱状图

由表 4、图 6、图 7、图 8 和图 10 可知，孕周、BMI 和两者的交互项都是影响 Y 染色体浓度的主要因素 ($p < 0.05$ 显著)，具体来说，孕周与 Y 浓度成正相关关系，非线性效应较弱但显著；BMI 与 Y 浓度成负相关；两者交互项成正交互协同作用，即高 BMI 值的孕妇的孕周正相关效应更强；同时通过 VIF 共线性检验发现，实际上仅有二者产生的样条基函数的共线性较高（而实际样条基函数的特点导致共线性本身会提高，所以对于显著性并无影响），而其余变量的共线性均较低，因此三种模型均没有多重共线性，VIF 共线性检验可以接受，并允许进行下文的显著性检验，三种模型均没有异常值和多重共线性，模型可靠。

7.3 模型显著性检验

针对本问比较三种模型的整体显著性、固定效应显著性和 GAMM 模型的孕周样条项非线性效应分别设立假设：

1. 模型整体显著性假设（针对 OLS、LME、GAMM）

- 原假设 H_0 ：模型中所有自变量对 Y 染色体浓度的联合效应为 0
- 备择假设 H_1 ：模型中至少有一个自变量对 Y 染色体浓度的联合效应不为 0

2. 成组 Wald 检验假设（OLS 和 LME 全部固定效应不含截距）

- 原假设 H_0 ：模型中除截距外的全部固定效应自变量的系数均为 0
- 备择假设 H_1 ：模型中除截距外的全部固定效应自变量的系数不全为 0

3.单个固定效应显著性假设（以 $week_c$ 为例，其他变量同理）

- 原假设 H_0 ： $week_c$ 对Y染色体浓度的效应系数为0
- 备择假设 H_1 ： $week_c$ 对Y染色体浓度的效应系数不为0

4.GAMM模型孕周样条项显著性假设

- 原假设 H_0 ：GAMM模型中孕周样条项的系数均为0
- 备择假设 H_1 ：GAMM模型中孕周样条项的系数不全为0

接着选择检验方法并设定显著性水平：

1.检验方法选择

- OLS、LME、GAMM模型的整体显著性检验：采用似然比检验（Likelihood Ratio Test），通过计算 $LR \times \chi^2$ 统计量衡量模型拟合优度与“无效应模型”的差异。
- OLS和LME的成组Wald检验采用 $Wald \times \chi^2$ 检验；GAMM模型孕周样条项的非线性效应检验采用平滑项成组Wald检验，通过 $Wald \times \chi^2$ 统计量检验样条项系数的联合显著性。
- OLS模型的单个固定效应显著性采用t检验；LME和GAMM模型采用z检验。

在此处我们来对于检验方法的选择做出解释（1）对于整体显著性检验，我们选择 $LR \times \chi^2$ 是因为它能统一地比较含效应模型相对空模型的似然改进，同时对于混合效应模型我们采用ML（非REML）以保证固定和随机结构间的可比性，而对于正态分布下OLS的LR与整体F检验近似等价。（2）对于成组效应检验我们使用 $Wald \times \chi^2$ ，便于对一组固定效应做联合假设检验，同时对GAMM，孕周的平滑项由多条样条基系数组成，单个系数不具可解释性，故以“平滑项成组Wald”检验其整体非线性是否显著。（3）单个系数方面，OLS在经典假设下采用t检验（小样本有精确t分布），但LME/GAMM的固定效应因自由度界定复杂，我们此处使用大样本近似的z检验对其进行替代。

2.显著性水平设定

统一设定 $\alpha = 0.05$ ，即当P值 < 0.05 时，拒绝原假设；当P值 ≥ 0.05 时，接受原假设。计算得到整体的显著性检验和具体参数的显著性检验如表4、表5和表6所示：

表4 各模型整体显著性检验（ $\alpha = 0.05$ ）

模型名称	$LR \chi^2$ 值	自由度（df）	p 值
OLS	81.075	5	5.551e-16
LME	93.070	5	<1e-300
GAMM	170.516	8	<1e-300

表5 各模型Wald检验结果表

模型名称	Wald 检验类型	χ^2 值	自由度（df）	p 值
LME	全部固定效应（不含截距） 成组检验	121.493	5	<1e-300
OLS	全部固定效应（不含截距） 成组检验	84.851	5	1.11e-16
GAMM（样条+混合效应）	平滑项成组检验	191.062	4	<1e-300

表6 单个固定效应检验结果表（coef为系数，SE为标准误差，t/z为检验统计量）

变量	OLS（coef/SE/t/p）	LME（coef/SE/z/p）	GAMM（coef/SE/z/p）
----	------------------	------------------	-------------------

Intercept	0.072/0.001/59.122/1.64 2e-286	0.0778/0.0019/40.150/p= 0	0.0540/0.0050/10.734/7. 041e-27
week_c	0.002/0.0003/7.813/1.85 7e-14	0.0029/0.0003/10.235/1. 388e-24	以样条项呈现
bmi_c	-0.0017/0.0004/-4.650/3. 911e-06	-0.0015/0.0005/-2.877/0. 0040	-0.0019/0.0005/-3.648/0. 000264
age_c	-0.0009/0.0003/-2.759/0. 0059	-0.0006/0.0005/-1.275/0. 2022	-0.0006/0.0005/-1.348/0. 1777
ivf_flag	-0.0160/0.0119/-1.345/0. 1790	-0.0251/0.0196/-1.281/0. 2003	-0.0239/0.0192/-1.244/0. 2135
week_c:b mi_c	0.0001/0.0001/0.953/0.3 411	0.0002/0.0001/2.226/0.0 260	0.0001/0.0001/0.938/0.3 480
样条项 1[0]	\	\	0.0256/0.0084/3.069/0.0 021
样条项 1[1]	\	\	-0.0059/0.0058/-1.004/0. 3152
样条项 1[2]	\	\	0.0576/0.0115/5.015/5.2 98e-07
样条项 1[3]	\	\	0.0904/0.0137/6.581/4.6 63e-11

由表 4、表 5 和表 6 可知，三种模型 p 值均小于 0.05，其中 LME 和 GAMM 的 p 值实际上为 0，均拒绝原假设。其中 GAMM 的 LR χ^2 值为 170.516，显著高于 OLS 和 LME，表明其对数据的拟合优度更优；同时成组 Wald 检验结果显示 GAMM 模型中孕周的样条项联合效应显著，表明孕周与 Y 染色体浓度存在显著非线性关系；对于三个模型单个固定效应检验存在显著变量效应：OLS 模型中孕周（week_c）对 Y 染色体浓度呈显著正向效应，BMI（bmi_c）和年龄（age_c）呈显著负向效应；LME 模型中孕周（week_c）的显著正向效应（系数=0.0029）强于 OLS 模型，BMI（bmi_c）仍呈显著负向效应，同时，孕周与 BMI 的交互项（week_c:bmi_c）呈显著正向效应，表明 BMI 较高时孕周对 Y 染色体浓度的正向效应可能更强；GAMM 模型中 BMI（bmi_c）的显著负向效应强于 OLS 和 LME 模型，孕周末以线性项呈现显著效应，但通过样条项体现出显著非线性特征，其中样条项 1[0]、样条项 1[2]和样条项 1[3]均呈显著正向效应，说明孕周在不同区间对 Y 染色体浓度的正向影响存在差异，且后期效应更强。

八、问题二模型建立与求解

问题分析：问题二这是一个回归类问题，题目需要根据 Y 染色体浓度的最早达标时间，选取利用所有可用特征对男胎孕妇 BMI 进行分组并给出最佳 NIPT 检查时点。这要求我们建立合适的能够兼顾多特征融合能力和分组时点可解释性的数学模型。

8.1 模型建立

8.1.1 数据准备

● 数据筛选

根据题目所给条件可知，通常孕妇的孕期在 10 周~25 周之间可以较为准确的检测到

胎儿的染色体浓度情况，因此我们筛选出检测孕周在此窗口限制内的样本；同时删除孕妇代码、检测周数、孕妇 BMI 或者 Y 染色体浓度缺失的样本数据。

● 建立二分类变量

为了后续更好适配建模需求，我们将连续的 Y 染色体浓度转化为二分类变量，具体公式如下：

$$achieved = \begin{cases} 1 & \text{if } Y\text{染色体浓度} \geq 0.04 \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

8.1.2 BMI 优化分组：多维聚类-决策树区间映射模型

BMI 分组方式的选择实际上有很多种[5]，本小组对于大量的分组方法进行了实验和探究但是目前大多数分组方式实际上存在三大问题。（1）分组方式仅依靠 BMI 的特点而不考虑其他变量，分组过于简单同时不能体现出不同 BMI 人群存在的实质区别（包括但不限于单纯采用分位数分组，对 BMI 使用一维 Kmeans 聚类）主观因素。（2）BMI 分组依赖主观因素的判断，构造一些相应的风险函数其弊端在于需要选择合适风险函数且构造合理的参数同时需要自己对不同的风险状况的影响进行人为赋权。（加速时间失效模型（AFT），风险函数+动态规划类）（3）构造连续区间进行回归：此种方案虽然在一定程度上消除部分主观因素的影响，且可以根据问题进行相应拟合，但是对于回归模型的选择仍然是一个重要挑战，同时多变量的情况下回归效果仍待考虑。为了处理上述弊端，本文创新性的提出了多维聚类-决策树区间映射模型，其核心思想是通过无监督的多维聚类将特征相似的个体汇聚到相同的簇，这充分利用了题目中出 BMI 之外的其他变量，但带来一个挑战，在多维聚类-决策树区间映射模型的情况下，BMI 分组区间会出现相互重叠的现象而我们要构造的是互不相交的 BMI 区间，对于此我们将多维聚类所得到的簇作为“伪标签”代入决策树模型中，仅依赖 BMI 分裂创造出分裂点进一步将分裂出的区间映射到不同的簇中生成新的 BMI 区间并通过 Bootstrap 稳定性检验和敏感性分析加强所得区间的稳定性。在整个流程中我们可以看到，我们在利用了所有可用变量特征的同时不引入任何的主观因素（包括参数和函数的设置），这增强了 BMI 分组结果对于数据的依赖，组内个体有着更相近的特征，更加符合实际情况，同时当扩充到多变量时借助聚类和 Bootstrap 仍能保持较高的稳定性，多维聚类-决策树区间映射模型的具体操作如下所示。因此在本问的 BMI 分组优化问题上我们最终选择聚类-决策树区间映射模型[6]，具体操作如下：

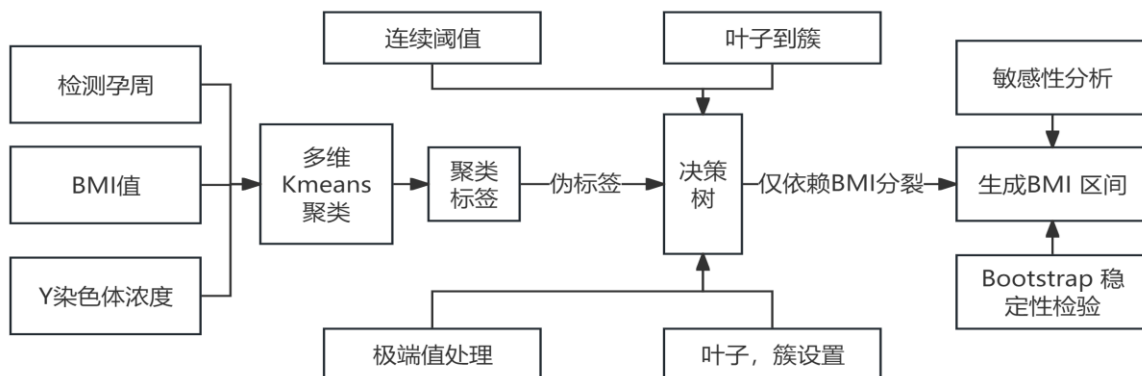


图 11 BMI 分组处理流程图

Step1: 聚类特征选取

根据问题二的题目要求，为了捕捉到“BMI-孕周-Y 浓度”的协同特征效应，我们选择 BMI 值，检测孕周，Y 染色体浓度作为多维聚类的输入特征，构建得到三维聚类特征[BMI, week, $Y_{frac,i,j}$]。

Step2: 肘部法则确定最佳聚类数 K

该法则的主要原理是通过计算不同 K 值下的惯性（Inertia），即簇内所有 NIPT 检测样本到簇中心的平方和。随着 K 的增大，惯性也随之增大，但当 K 超过“肘部点”后，惯性下降幅度骤减，此时的 K 值即为最佳聚类数，具体公式如下：

$$Inertia(K) = \sum_{g=1}^K \sum_{x \in C_g} \|x - \mu_g\|^2, \quad (11)$$

$$d = \frac{|(y_2 - y_1)K - (x_2 - x_1)Inertia + x_2y_1 - y_2x_1|}{\sqrt{(y_2 - y_1)^2 + (x_2 - x_1)^2}}, \quad (12)$$

其中，我们将 K 的范围设置为 $K \in [2, 10]$ ； C_g 代表第 g 个簇； μ_g 是第 g 个簇的中心； x 是簇 C_g 内的样本点； $\|x - \mu_g\|^2$ 是样本 x 到簇中心 μ_g 的欧几里得距离平方； (x_1, y_1) 为 K 最小对应的惯性， (x_2, y_2) 为 K 最大对应的惯性， d 为垂距， d 最大时的 K 即为最佳聚类数值。

Step3: 最小样本量约束

设定每组样本量 ≥ 50 ，确保其 BMI 分组后组内相同特征的数量更多，分组代表性更强，增强统计可靠性；若样本量过少，则将样本数最少的组，与 BMI 中位数最接近的相邻组合并，重复直至满足样本量约束。

Step4: 聚类-决策树 BMI 区间映射

由于我们是结合多个变量聚类得到簇，簇的定义不仅仅依赖于 BMI 这一个变量，因此各簇在 BMI 变量水平上可能会有重叠部分，无法通过直接投影到 BMI 水平轴上得到分组连续区间。所以此处我们采用“浅决策树”的方法，即只用 BMI 列作为训练特征，将多维聚类得到的簇作为“伪标签”，生成 BMI 区间分裂点，把原来的多维簇按照 BMI 区分开。

接着从浅决策树的分裂点提取 BMI 的阈值，按照大小排序后得到互不相交的连续分组区间，最后对区间结果做敏感性分析和 Bootstrap 稳定性检验得到最终 BMI 最优分组。

8.1.3 广义加性模型（LogisticGAM）建立

考虑到检测数据的特点和问题一已验证的孕周（week）与 BMI 的显著交互效应且数据存在的非线性，我们在此处继续沿用 GAM 模型，同时结合逻辑回归的二分类特点，建立了 LogisticGAM 模型，构建“week、BMI $\rightarrow$ 达标概率 $p(t)$ ”的预测模型，可解释性强，且能直观展示单个变量对达标概率的影响趋势，即

$$p(t) = \Pr(achieved = 1 | week, BMI) \quad (13)$$

采用 logit 链接函数，将达标概率与自变量的非线性关系联系为：

$$\text{logit}(p(t)) = s1(\text{week}) + s2(BMI) + te(\text{week}, BMI), \quad (14)$$

其中， $\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$ 将概率 p 映射到实数域；样条函数 $s()$ 用来拟合孕周、BMI 与 $\text{logit}(p)$ 的非线性关系；交互样条 $te()$ 用来捕捉孕周与 BMI 之间的协同影响。

8.1.4 最佳 NIPT 时点计算

根据题目条件，当最佳检测时点超过 28 周时的治疗风险极高。但是在回归过程中，

高 BMI 分组回归所得的最佳检测时点有着滞后特征（在较高概率阈值的情况下，计算所得的最佳检测点的时间远超 28 周），这无疑加强了治疗风险，因此为避免此种情况出现，我们基于正常概率判断和对于天数施加惩罚构造了准确性优先和风险最小化两种最佳时点判定准则：

● **准则 1：最早达标时点 $T_{p_0}^*$**

即满足“达标概率 $\geq P_0$ ”的最早孕周时间（ P_0 取 0.9），具体计算步骤如下：

Step1: 对每组固定 BMI 中位数（代表该组特征）生成 10-25 周的孕周网格后，用 LogisticGAM 模型预测得到网格上的达标概率 $p(t)$ ；

Step2: 找到 $p(t)$ 首次 $\geq p_0$ 的孕周时间，若恰好落在两个网格点之间，用线性插值的方法计算得到 $T_{p_0}^*$ ：

$$T_{p_0}^* = t_{i-1} + \frac{0.9 - p(t_{i-1})}{p(t_i) - p(t_{i-1})}(t_i - t_{i-1}), \quad (15)$$

其中， p_0 为达标概率阈值， t_{i-1} 和 t_i 为相邻孕周， $p(t_{i-1}) < p_0 < p(t_i)$ 。

● **准则 2：风险最优时点 T_{risk}^***

即同时平衡未达标风险和时间风险（检测过晚的临床风险），通过最小化损失函数实现，具体公式定义如下：

$$L(t) = (1 - p(t)) + w \cdot \text{penalty}(t), \quad (16)$$

其中，权重 w 代表惩罚函数的权重为 0.08； $\text{penalty}(t)$ 表示时间风险惩罚项，对于惩罚权重的选择和风险惩罚项形式的确定我们以模型拟合的准确度为标准，分别对惩罚权重和风险惩罚项记性网格搜索调优得到，三种惩罚项形式如下：

$$\text{penalty}(t) = \begin{cases} \max(0, t - 12) & \text{线性惩罚} \\ \max(0, t - 12)^k & \text{多项式惩罚} \\ \exp(k_{\text{exp}}(t - 12)) - 1 & \text{指数惩罚} \end{cases} \quad (17)$$

8.2 模型求解

在聚类求解最优 BMI 分组阶段我们初始化次数取 20，Bootstrap 设置为 200 次，确保结果的稳健性和可复现性。最终我们利用 Python 求解得到 5 组 BMI 区间和三种达标概率阈值下（0.85，0.87 和 0.90）的最佳 NIPT 时点表现如表 7 所示：

表 7 问题二 BMI 区间和最佳 NIPT 时点结果表

BMI 区间	BMI 代表值	最佳 NIPT 时点 (周) $P_0=0.85$	最佳 NIPT 时点 (周) $P_0=0.87$	最佳 NIPT 时点 (周) $P_0=0.90$
[20.70,32.23]	30.40	10.94	11.47	12.38
[32.23,34.15]	33.15	11.69	12.19	13.45
[34.15,34.70]	34.34	12.37	13.00	22.45
[34.70,35.95]	35.32	13.23	22.26	23.08
[35.95,46.88]	37.23	23.05	23.57	24.55

观察表 7 我们可以看出，随着 BMI 值不断变大，对应的最佳 NIPT 检测时间逐渐升高，如 BMI 代表值为 30.40 组，在每个达标概率下都是所有组中最早达到最佳时点的；同时在同一 BMI 组别内，达标概率阈值越大，最佳检测时点越晚。同时我们还绘制了 3D 聚类结果散点图、BMI 分组散点图、3D 拟合结果图和预测混淆矩阵图如图 12、图 13 所示：

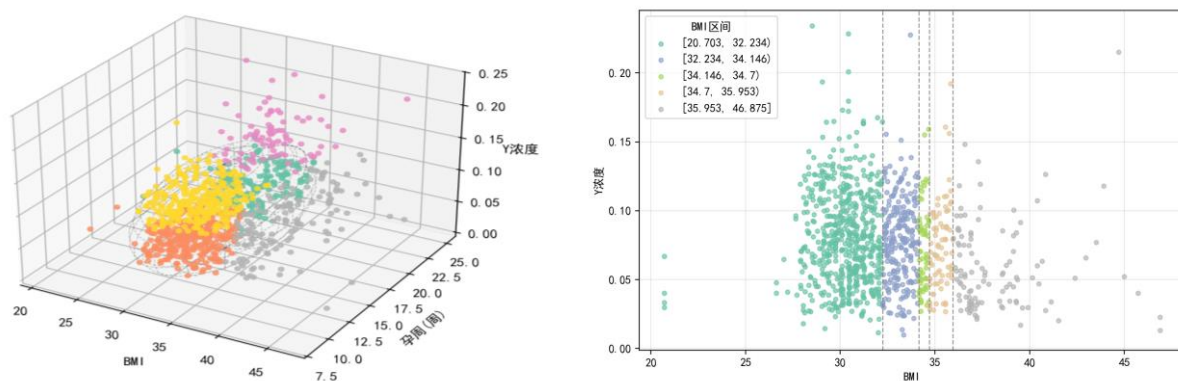


图 12 3D 聚类结果散点图（左）和 BMI 分组散点图（右）

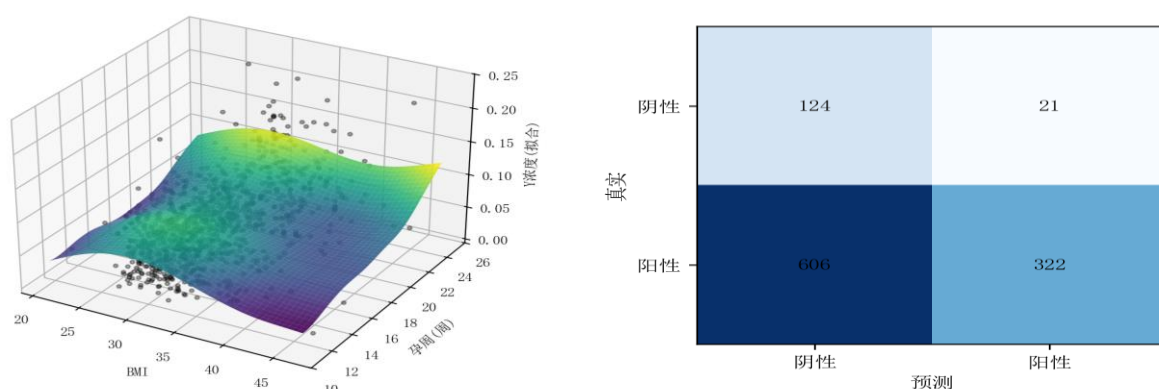


图 13 3D 拟合结果图（左）和预测混淆矩阵图（右）

观察图 12 和图 13 我们发现，聚类的不同颜色簇分布满足“BMI 越高，Y 浓度达标越晚”的结论，且每一簇对应区间的分割线与结果表 7 中分组区间基本一致；拟合结果的曲面平滑度高与数据贴合性强，拟合效果较好；同时模型对达标样本（阳性）的精确率较高，为 NIPT 最佳时点推荐提供置信基础。

8.3 模型评价与敏感性分析

8.3.1 模型评价

在模型建立和求解过程中我们选择的是 LogisticGAM 模型，在此处我们分别给出在本问上的性能表现。我们的模型设置如下：1.LogisticGAM 引入交互样条和样条函数；2.随机森林（RF）设置决策树为 300，随机种子数为 42；3.逻辑回归（Logit）设置 max_iter=2000；4.广义线性模型（GLM）采用 Python 默认设置；5.广义估计方程（GEE）设置同患者样本可交换；6.梯度提升树（GB）设置随机种子数为 42。最终实验结果证明 LogisticGAM 模型在本问中展现出良好的性能。

我们以 MAE、RMSE、AUC、Accuracy、LogLoss 和运行时间为评估指标，对不同的效果进行检验并绘制相关曲线图（对比模型以 RF 为例）和性能雷达图如表 8、图 14 和图 15 所示：

表 8 各模型评估指标对比结果表

模型	MAE	RMSE	AUC	Accuracy	LogLoss	运行时间（秒）
（分类阈值为 0.7）						
GAM	0.22	0.33	0.66	0.86	0.37	0.01

Logit	0.23	0.34	0.61	0.86	0.38	0.04
GLM	0.23	0.34	0.60	0.86	0.38	0.01
GB	0.17	0.27	0.93	0.92	0.25	0.21
RF	0.08	0.15	1.00	0.97	0.10	0.86
GEE	0.23	0.34	0.54	0.87	0.40	1.40

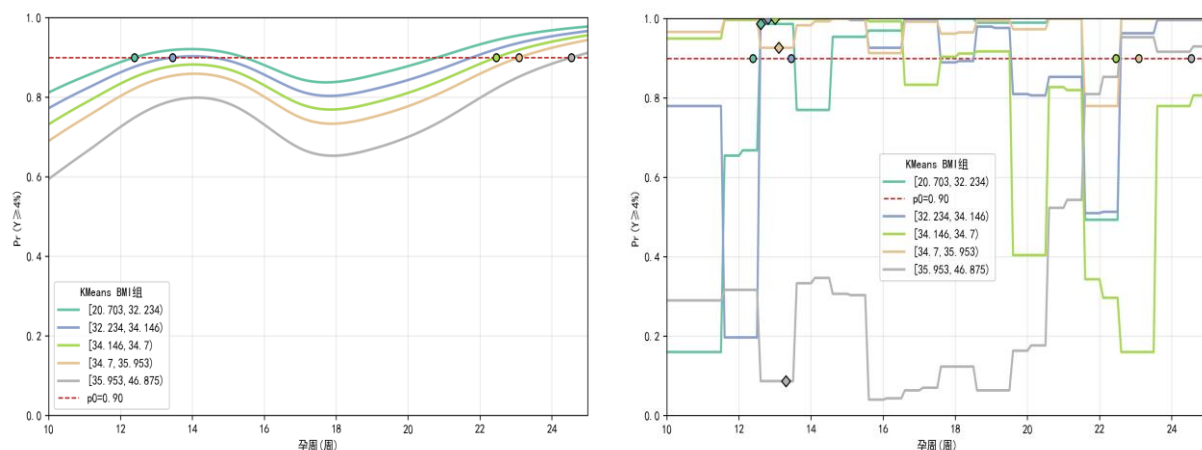


图 14 不同 BMI 区间 NIPT 检测 Y 浓度达标概率随孕周变化曲线（左 GAM 右 RF）

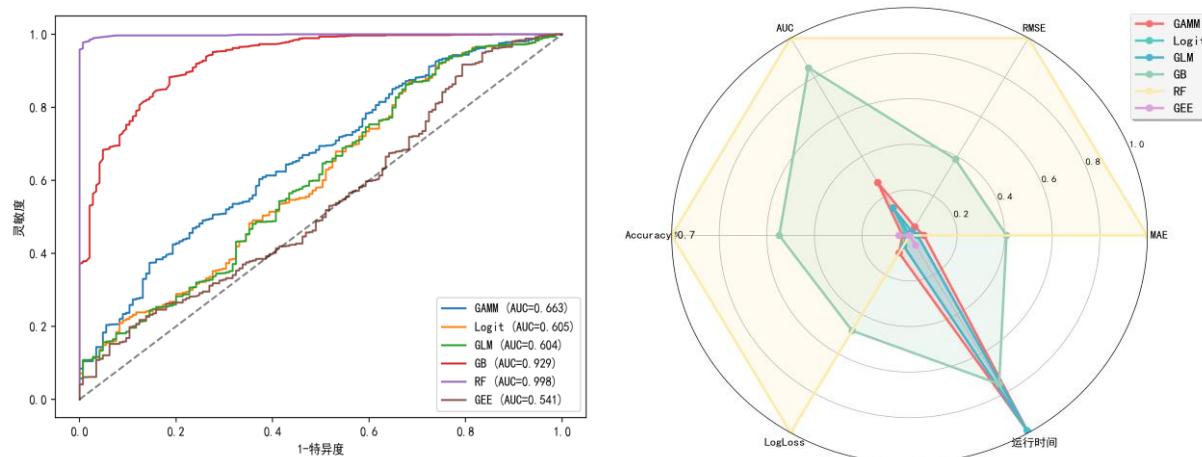


图 15 不同模型预测 Y 浓度达标性能 ROC 曲线图和不同模型性能雷达图（右）

从表 8 和图 15 我们发现,单从 RMSE、AUC 及精确率等评估指标来看,LogisticGAM 模型表现与 RF、GB 等其他机器学习模型之外的模型相比较速度, AUC, 准确率等指标都较为优越,但是在于 RF、GB 等其他机器学习模型表现欠佳,但经过交叉验证后实际上 RF、GB 存在过拟合现象其交叉验证后性能远不及 GAMM（由于篇幅限制交叉验证后的图像我们将在附录中给出）。同时图 14 的结果显示, LogisticGAM 模型在不同 BMI 分组下, 呈现出的 Y 浓度达标概率随孕周变化的曲线走势具有高度一致性, 这种一致性为临床实际应用提供了更高的参考价值; 相反对比 RF 模型, 其在各 BMI 分组中的变化曲线呈现出显著差异, 这正是由其机器学习的非线性拟合导致的数值大幅度变化, 没有规律和参考价值, 这无疑会增加临床实践中判断最佳干预时点的难度。因此, 我们认为我们的 LogisticGAM 模型是解决本问题的最优模型。

8.3.2 敏感性分析

为分析检测误差对 BMI 分组和最佳 NIPT 时点的影响，我们对 BMI、孕周和 Y 浓度增加不同程度的检测误差扰动得到扰动混淆矩阵图 16 和图 17：

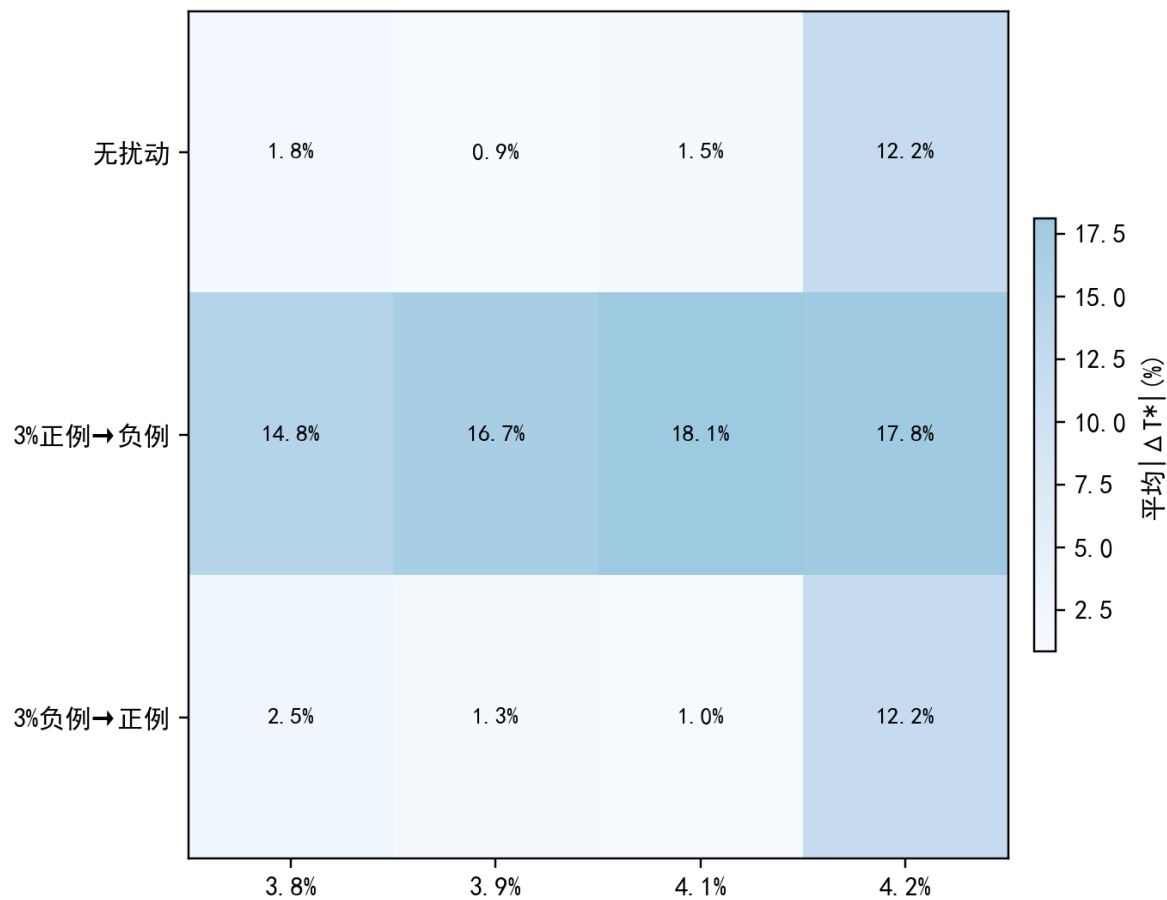


图 16 最佳 NIPT 检测时点扰动影响混淆矩阵图

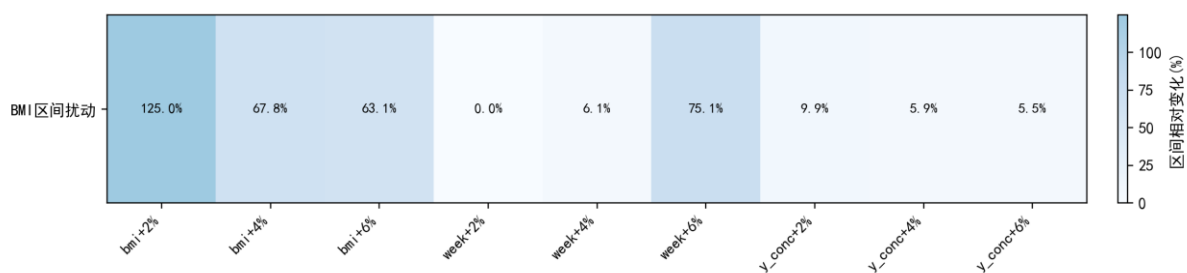


图 17 BMI 最优分组扰动影响混淆矩阵图

观察图 16 和图 17 可以发现，对于 NIPT 最佳检测多数时间点来说在无外界扰动时稳定性强，仅存在少数特殊区间（4.2%）自身变异性大；在“3%正例→负例”扰动时所有区间变化率显著上升（14.8%~17.8%），会强烈影响模型对各区间样本的预测稳定性；而在“3%负例→正例”扰动时变化率回落至低水平（ $\leq 2.5\%$ ，仅最右侧仍为 12.2%），与无外界扰动情况变化趋势一致。对于 BMI 最优分组区间来说，BMI 自身作为核心变量，其自身区间的扰动会强烈影 BMI 分位数区间内的样本；不同孕周的变化扰动对 BMI 分组区间影响敏感度差异较大，但在 Y 浓度区间扰动下较稳定。但实际上 BMI 和孕周

实际上属于相对稳定的数值，基本不会出现测量误差，而在可能出现测量误差的 Y 染色体浓度的扰动下，我们的多维聚类-决策树区间映射模型表现出极强的稳定性，鲁棒性强。

九、问题三模型建立与求解

问题分析：问题三相当于在问题二的基础上我们要考虑身高、体重、年龄等多个因素进去，但是由于我们的 BMI 分组模型多维聚类-决策树区间映射模型的设置，其不需要修改任何的参数或公式，只是增加变量的输入数量即可，为我们问题三的解决大大提供了便利性，我们需要构建新的变量来加强 LogisticGAM 模型，最后通过施加多种扰动分析敏感性。

9.1 多维广义加性模型（Multi-LogisticGAM）建立

在问题二基础上，本问要求我们综合身高、体重、年龄等多个因素，同时考虑检测误差和胎儿 Y 染色体浓度达标比例。由此我们综合多种因素得到如下三类变量：

- 1.核心变量：孕妇代码（ $patient_{id}$ ）、孕周（week）、BMI（bmi）、Y 染色体浓度（ y_{conc} ）；
- 2.临床扩展变量：年龄（age）、身高（ $height_m$ ）、体重（ $weight_{kg}$ ）、怀孕次数（gravidity）、生产次数（parity）；
- 3.测序扩展变量：总读段数（total_reads）、比对率（map_ratio）、重复率（dup_ratio）、GC 含量（gc_content）、染色体 Z 值（ $Z_{13}/Z_{18}/Z_{21}/Z_X$ ）

将上述三类变量统一合并为多维特征集结合 LogisticGAM 模型构建 Multi-LogisticGAM 模型，进而聚类预测确定 BMI 最优分组和每组的最佳 NIPT 时点。

9.2 模型求解

和问题二一致，在聚类求解最优 BMI 分组阶段我们初始化次数取 20，Bootstrap 设置为 200 次，最终我们利用 Python 求解得到 5 组 BMI 区间和三种达标概率阈值下（0.93，0.95 和 0.97）的最佳 NIPT 时点表现如表 9 所示：

表 9 问题三 BMI 区间和最佳 NIPT 时点结果表

BMI 区间	BMI 代表值	最佳 NIPT 时点(周) $P_0=0.93$	最佳 NIPT 时点 (周) $P_0=0.95$	最佳 NIPT 时点(周) $P_0=0.97$
[20.70,31.91]	30.27	11.33	12.21	20.09
[31.91,33.83]	32.85	13.38	20.83	22.29
[33.83,35.55]	34.52	12.75	20.73	22.30
[35.55,36.57]	35.88	23.06	23.92	25.09
[36.57,46.88]	38.22	25.58	26.34	27.46

观察表 9 我们可以看出，同样符合 BMI 越高，达标概率阈值越大，最佳检测时点越晚。同时我们也绘制了 3D 聚类结果散点图、BMI 分组散点图、3D 拟合结果图和预测混淆矩阵图如图 18 和图 19 所示：

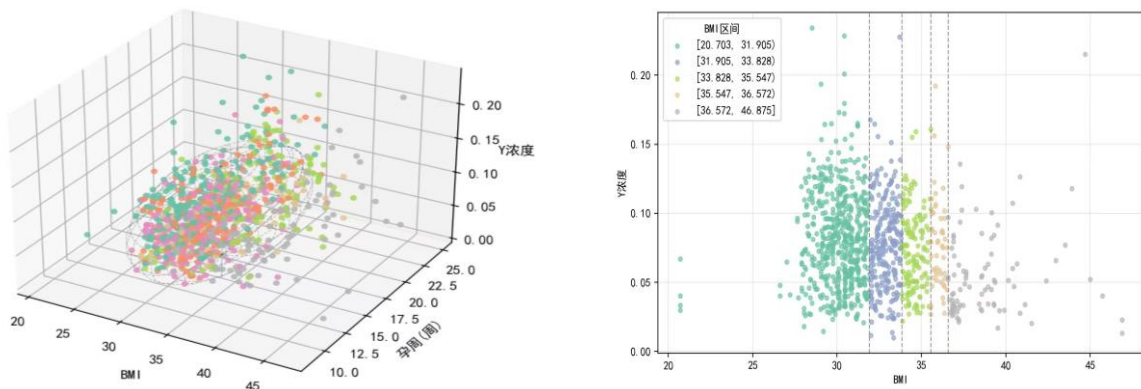


图 18 3D 聚类结果散点图（左）和 BMI 分组散点图（右）

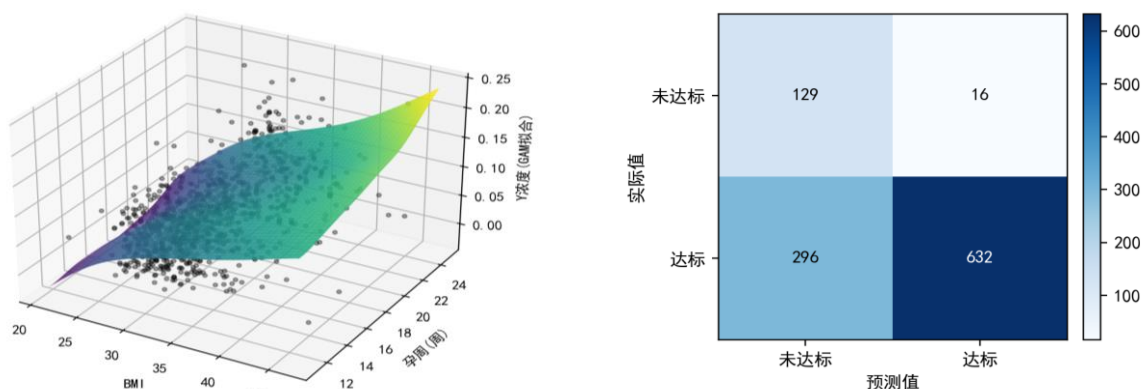


图 19 3D 拟合结果图（左）和预测混淆矩阵图（右）

观察图 18 和图 19 我们发现，聚类的不同颜色簇分布也满足“BMI 越高，Y 浓度达标越晚”的结论，且每一簇对应区间的分割线与结果表 9 中分组区间基本一致；拟合结果的曲面平滑度高与数据贴合性较好；在预测达标的精准度上同样表现较好，约为 97.5%，可以为 NIPT 最佳时点推荐提供置信基础。

9.3 模型评价与敏感性分析

9.3.1 模型评价

在模型建立与求解过程中，我们选取 Multi-LogisticGAM 作为核心模型，并与多种模型进行对比验证。各模型的关键设置如下：1.Multi-LogisticGAM 模型引入 week 与 bmi 的交互样条，同时对 week、bmi 及其他可选变量设置单变量样条，特征采用核心变量（week、bmi）+ 非全空可选变量（如 age、z13/z18 等）；2.随机森林（RF）设置决策树数量为 300，随机种子数为 42，特征仅使用 week 与 bmi；3.逻辑回归（Logit）设置最大迭代次数 max_iter=2000，特征构建为 week、bmi 的二阶多项式及交互项；4.广义线性模型（GLM）采用二项分布+Logistic 链接函数，特征同逻辑回归，其余参数默认；5.广义估计方程（GEE）设置同孕妇样本为可交换相关性结构（Exchangeable），特征为二阶多项式及交互项；6.梯度提升树（GB）设置随机种子数为 42，特征仅使用 week 与 bmi，其余参数默认。

我们以 MAE、RMSE、AUC、Accuracy、LogLoss 和运行时间为评估指标，对不同的效果进行检验并绘制相关曲线图（对比模型以 RF 为例）和性能雷达图如图 20 和图 21 所示：

表 10 各模型评估指标对比结果表

模型	MAE	RMSE	AUC	Accuracy	LogLoss	运行时间（秒）
(分类阈值为 0.7)						
GAM_Multi	0.17	0.28	0.88	0.88	0.27	0.01
Logit	0.23	0.34	0.60	0.86	0.38	0.01
GLM	0.23	0.34	0.60	0.86	0.38	0.00
GB	0.17	0.27	0.93	0.92	0.25	0.07
RF	0.08	0.15	1.00	0.97	0.10	0.29
GEE	0.23	0.34	0.54	0.87	0.40	0.29

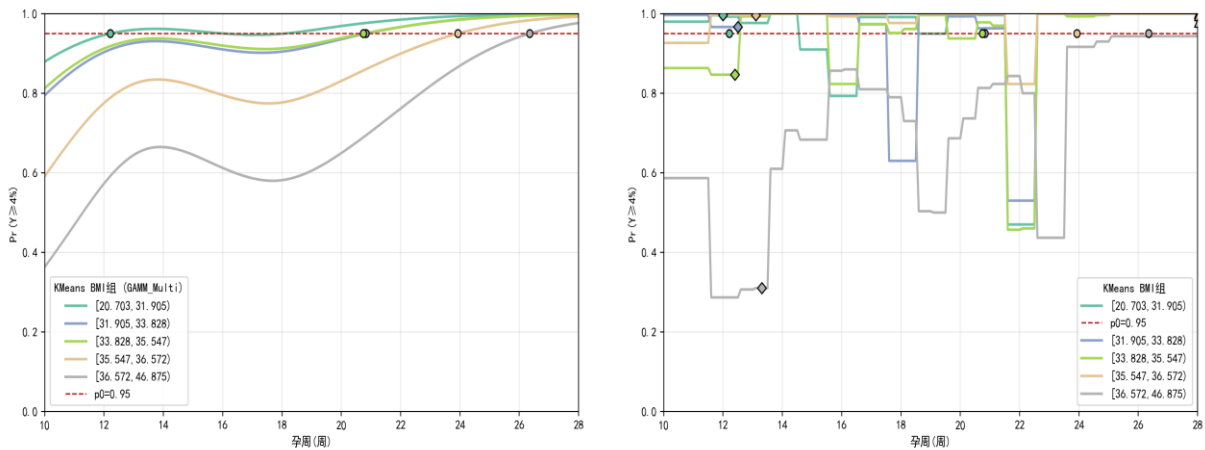


图 20 不同 BMI 区间 NIPT 检测 Y 浓度达标概率随孕周变化曲线（左 GAM_Multi 右 RF）

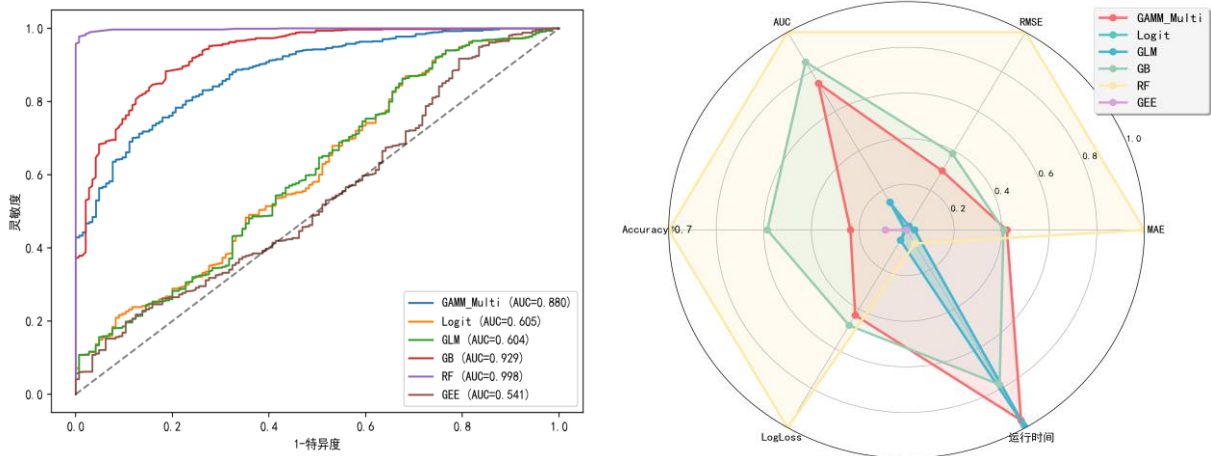


图 21 不同模型预测 Y 浓度达标性能 ROC 曲线图和不同模型性能雷达图（右）

观察表 10、图 20 和图 21 可以发现，与问题二得到的模型评价结果类似，我们的 Multi-LogisticGAM 模型在 MAE、RMSE 等评估指标上的表现不如 RF、GB 等机器学习模型，但其不同 BMI 区间 NIPT 检测 Y 浓度达标概率随孕周变化曲线仍然有趋势一致性，更有临床参考价值。

9.3.2 敏感性分析

同样地，我们对 BMI、孕周和 Y 浓度增加不同程度的检测误差扰动得到扰动混淆矩阵图 22 和图 23：

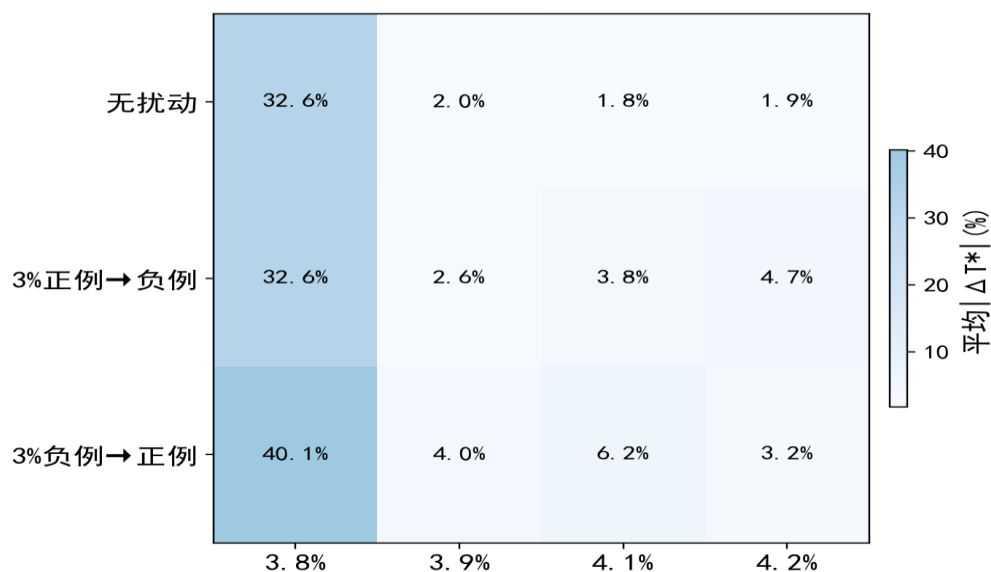


图 22 最佳 NIPT 检测时点扰动影响混淆矩阵图

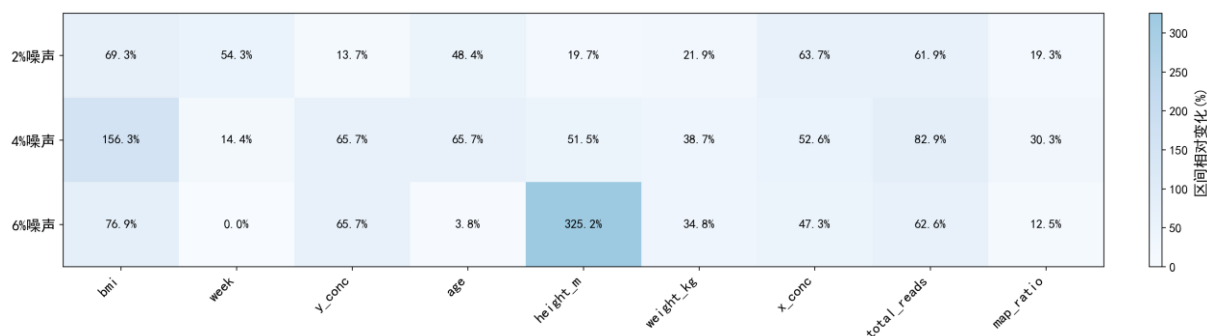


图 23 BMI 最优分组扰动影响混淆矩阵图

从图 22 和图 23 我们发现，对 NIPT 最佳检测时点来说多数区间稳定性好，仅 3.8% 区间自身变异性大；3%正例→负例扰动时各区间变化率小幅上升，但 3.8%列仍维持 32.6%变化比例，整体影响有限；3%负例→正例扰动时 3.8%列变化率进一步升至 40.1%，其余区间也从低水平升至 4.0%~6.2%，表明负例转正例的扰动对区间稳定性影响更显著。对 BMI 最优分组来说，噪声强度和不同特征对区间产生的影响不同，噪声强度越大，区间变化越明显，其中 BMI 和身高变量分别在 4%和 6%噪声扰动时对区间变化影响尤为显著。但 BMI 实际上不会出现较多的检测误差，所以可以接受，再者我们的多维聚类-决策树区间映射模型实际上是仅依赖数据进行的无监督学习而不施加任何主观因素，在增添更多数据的情况下表现可能会更稳定。

十、问题四模型建立与求解

问题分析：问题四是综合多因素的女胎染色体异常判定的二分类任务，这要求我们应该先分析出各个变量间的关系，由于题目涉及到了三条不同的染色体，不同于问题三，在本问中应建立针对单染色体的 logit 模型，最后综合三条染色体判定结果判定胎儿异常情况。

10.1 数据准备与分析

10.1.1 数据准备

- 次数型变量格式统一

为简化分析维度，我们将 ≥ 3 、3次及以上等文本型数据转为数值型，且 ≥ 3 的次数统一为3。

- 缺失值与异常值筛选

对于检测孕周、孕妇 BMI、13/18/21 号染色体 Z 值、读段数等关键变量缺失的样本，为保障建模有效性我们选择直接删除；

检测孕周、BMI 值和前三问一样将其限制在合理窗口范围内；唯一比对的读段数限制 $\geq 1e6$ ，保证检测深度；重复读段的比例 $\geq 50\%$ ，以此保证测序数据的有效性。

10.1.2 数据分析

- 染色体 Z 值与各协变量关系

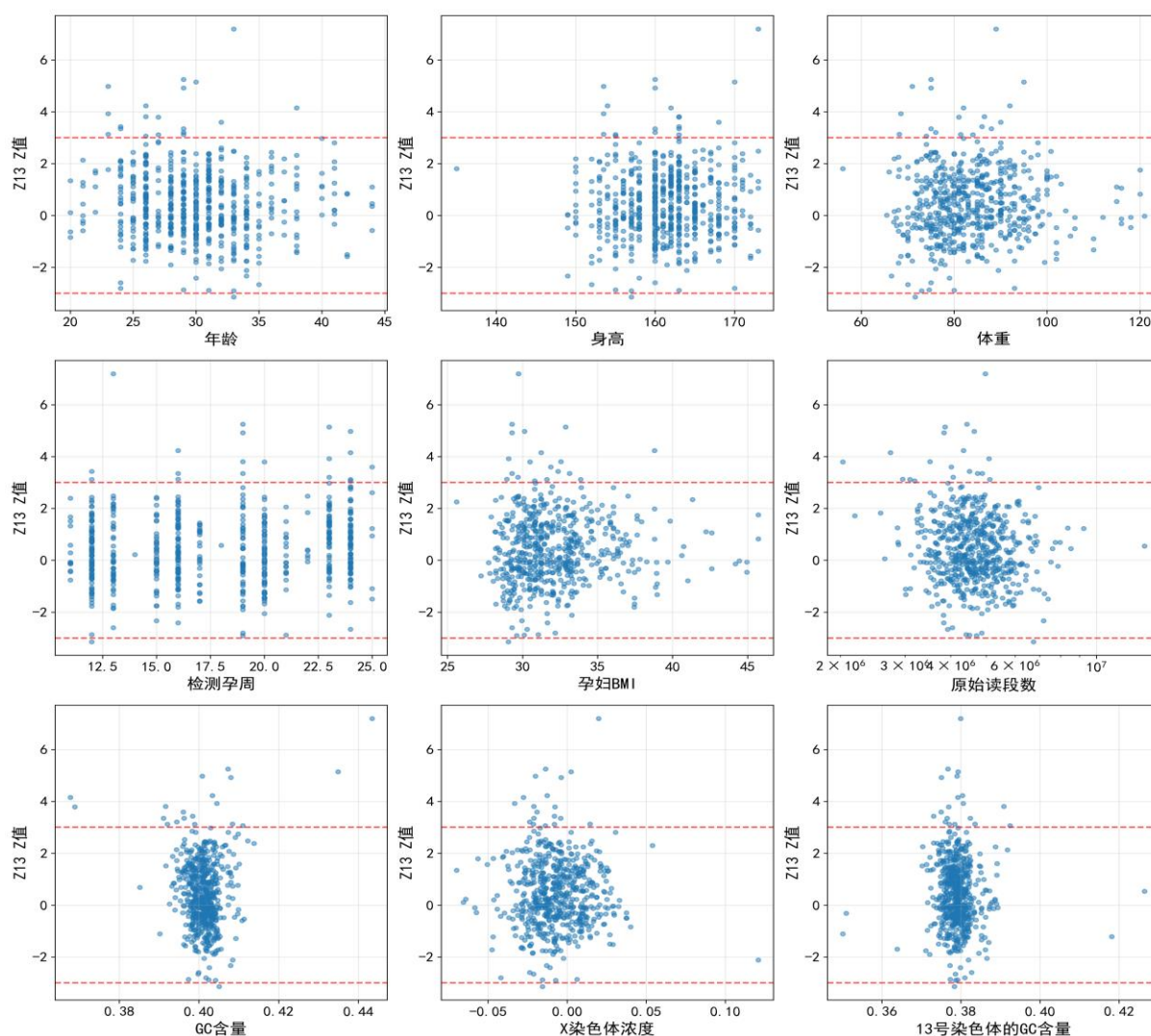


图 24 染色体 Z 值与各变量关联散点图（以 13 号染色体为例）

观察各变量与 Z13 值的散点分布图 24 显示，年龄、身高、体重、检测孕周、孕妇 BMI、原始读段数及 X 染色体浓度与 Z13 值的关联较弱，其散点普遍分散且无明显的线性趋势；而基因组 GC 含量（包括全基因组 GC 含量与 13 号染色体 GC 含量）与 Z13 值的散点高度聚集，其中 13 号染色体 GC 含量的聚集性最为显著，表明其与 Z13 值的

关联更紧密且稳定性更强。

10.2 模型建立

根据题目要求，本问要求我们综合多个特征给出关于女胎是否异常的判定方法。而 Logit 回归是解决“二分类判定问题”的经典统计模型，能够直接输出作为判定依据的常染色体异常的概率，更加符合临床风险评估预测的实际场景，同时模型系数的可直观解释性较强。考虑到解决本问的关键在于判定三种常染色体是否存在非整倍体，又因为三条染色体的异常机理相互独立，因此本问我们采用“单染色体 Logit 建模+整体整合”的框架。

10.2.1 特征工程

● 目标变量构建

为了更好地标的胎儿的异常情况，结合原始所给数据的异常标记情况，定义目标变量如下：

$$Y_i = \begin{cases} 1 & i \text{ 号染色体存在非整倍体} \\ 0 & \text{正常} \end{cases}, \quad (18)$$

其中， $i \in \{13, 18, 21\}$ 表示三条常染色体。

● 特征提取

根据题目要求综合考虑 X 染色体及三种常染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等因素，我们从所给检测数据中提取得到三类核心特征：

1. 染色体信号特征：三种常染色体的 Z 值（ Z_{13} ， Z_{18} 和 Z_{21} ）；
2. 测序质量特征：三种常染色体的 GC 含量（ GC_{13} ， GC_{18} 和 GC_{21} ）、总读段数（Reads）、比对率（Map）；
3. 个体差异特征：孕妇 BMI 和检测孕周（Week）。

10.2.2 BMI 优化分组

尽管题目未明确要求对 BMI 进行分组，但考虑到 BMI 作为影响 NIPT 检测准确性的关键协变量，为实现对不同 BMI 区间孕妇的个性化、精准化女胎异常判定，避免“一刀切”判定规则的局限性，在本问求解过程中依然延续前几问采用多维聚类-决策树区间映射方法形成 BMI 优化分组，为后续针对不同 BMI 群体定制差异化的异常判定规则提供了群体划分依据。

10.2.3 单染色体 Logit 模型构建

针对每条染色体单独建立的 Logit 模型公式如下（以 21 号为例，13 和 18 号同理）：

$$\begin{aligned} \text{logit}(P_{21}) = \ln\left(\frac{P_{21}}{1 - P_{21}}\right) = & w_{0,21} + w_{1,21}Z_{21} + w_{2,21}GC_{21} \\ & + w_{3,21}BMI + w_{4,21}Week + w_{5,21}Reads + w_{6,21}Map \end{aligned} \quad (19)$$

其中， P_{21} 为 21 号染色体异常的概率； $w_{0,21}$ 为截距项，表示所有自变量为 0 时的基础对数优势比； $w_{1,21} \sim w_{6,21}$ 为回归系数，用来量化对应自变量对 P_{21} 的影响方向和强度（如 $w_{1,21}$ 为正时说明 Z_{21} 越大，染色体异常概率越高）。

10.2.4 女胎整体异常判定规则

基于三条染色体的 Logit 模型，结合题目所说只要有任一条染色体出现异常情况，女胎整体也异常，由此得到如下整体异常判定规则：

$$\text{女胎异常} = \begin{cases} 1 & \text{若 } P_{13} \geq P_{\text{阈值}} \text{ 或 } P_{18} \geq P_{\text{阈值}} \text{ 或 } P_{21} \geq P_{\text{阈值}} \\ 0 & \text{否则} \end{cases} \quad (20)$$

10.3 模型求解与评价

在模型建立与求解过程中,我们选取 Logit 模型作为核心模型,并与随机森林(RF)、XGBoost (XGB)、LogisticGAM (GAM)、广义线性模型 (GLM)、广义估计方程 (GEE)、梯度提升树 (GB) 等模型进行对比验证以确保女胎异常判定结果的可靠性。各模型的关键设置如下: 1.Logit 模型设置最大迭代次数 $\text{max_iter}=2000$, solver 选用 lbfgs (适配多特征优化), class_weight 设置为 balanced (缓解非整倍体样本类别不平衡), 正则化参数 $C=0.5$ (避免过拟合), 随机种子数设置为 42; 2.随机森林 (RF) 模型设置决策树数量 $n_estimators=300$, 最小叶节点样本数 $\text{min_samples_leaf}=3$, 最大特征数 $\text{max_features}='sqrt'$, 随机种子数为 42, 类别权重采用 balanced_subsample, 动态调整每棵树的类别平衡, 并行计算 $n_jobs=-1$, 限制最大树深 $\text{max_depth}=8$; 3.XGBoost (XGB) 设置随机种子数为 42, 学习率 $\text{learning_rate}=0.05$, 决策树数量 $n_estimators=400$, 最大树深 $\text{max_depth}=3$, 样本采样率 $\text{subsample}=0.9$, 特征采样率 $\text{colsample_bytree}=0.8$, 正则化参数为 1.0; 4.LogisticGAM (GAM) 设置样条自由度 $\text{df}=5$, 惩罚项 $\alpha=0.1$; 5.广义线性模型 (GLMM_RE) 设置分布类型 $\text{family}=\text{Binomial}()$ (二分类任务), 链接函数 $\text{link}=\text{Logit}()$, 其余参数默认; 6.广义估计方程 (GEE) 相关性结构采用同一孕妇的多次检测样本可交换相关, 分布类型 $\text{family}=\text{binomial}$, 链接函数 $\text{link}=\text{logit}$, 以孕妇为单位进行分组拟合; 7.梯度提升树 (GB) 设置随机种子数为 42, 学习率 $\text{learning_rate}=0.1$, 决策树数量 $n_estimators=200$, 最小样本分裂数 $\text{min_samples_split}=10$, 损失函数采用 deviance (对数似然损失), 子采样比例 $\text{subsample}=0.8$ 。

我们绘制不同模型 ROC 相关曲线图 25 比较不同模型性能效果:

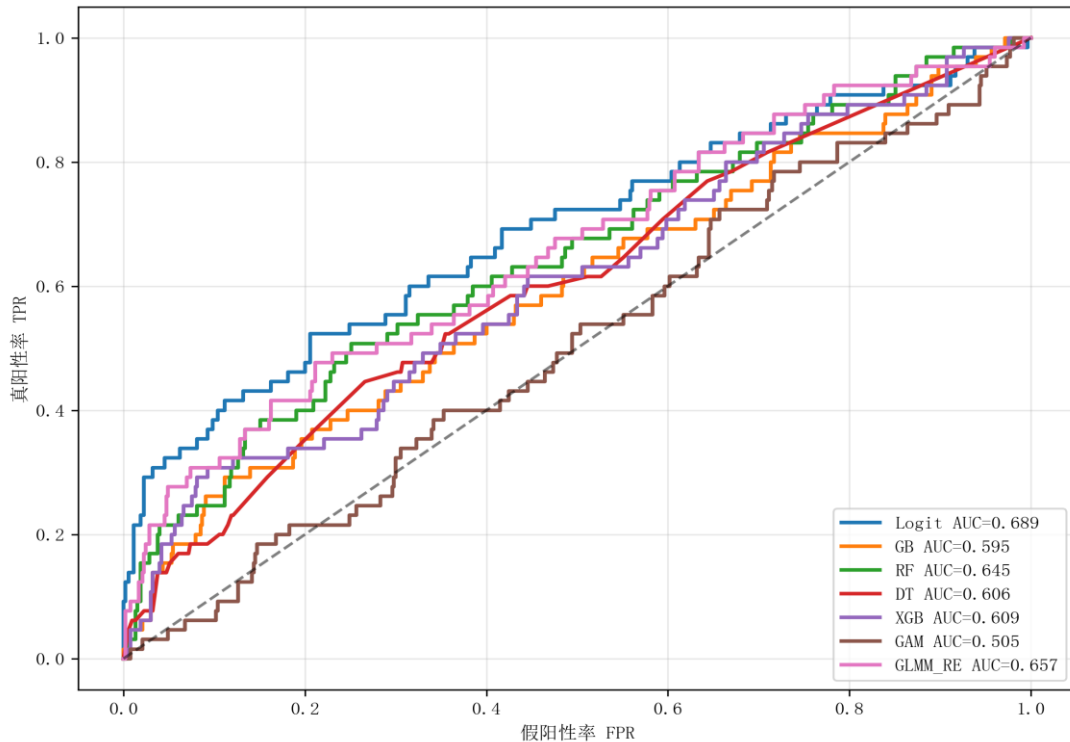


图 25 不同模型 ROC 曲线对比图

观察图 25 可以发现，我们所使用的 Logit 模型（AUC=0.689）的 AUC 在所有模型中最高，且 ROC 曲线整体位于最上方，能够很好的区分出正常与异常女胎。接着我们进一步利用 Youden's Index 和特异性指标（Specificity）评估我们模型在三条不同染色体上的表现，具体公式如下：

$$Youden's\ Index = TPR - FPR, \quad (21)$$

$$Specificity = \frac{TN}{FP + TN}, \quad (22)$$

$$TPR = \frac{TP}{FN + TP}, \quad (23)$$

$$FPR = \frac{FP}{TN + FP} = 1 - Specificity, \quad (24)$$

其中， TN 为真阴性，即正常样本被正确判定为正常； FP 为假阳性，即正常样本被错误判定为异常。

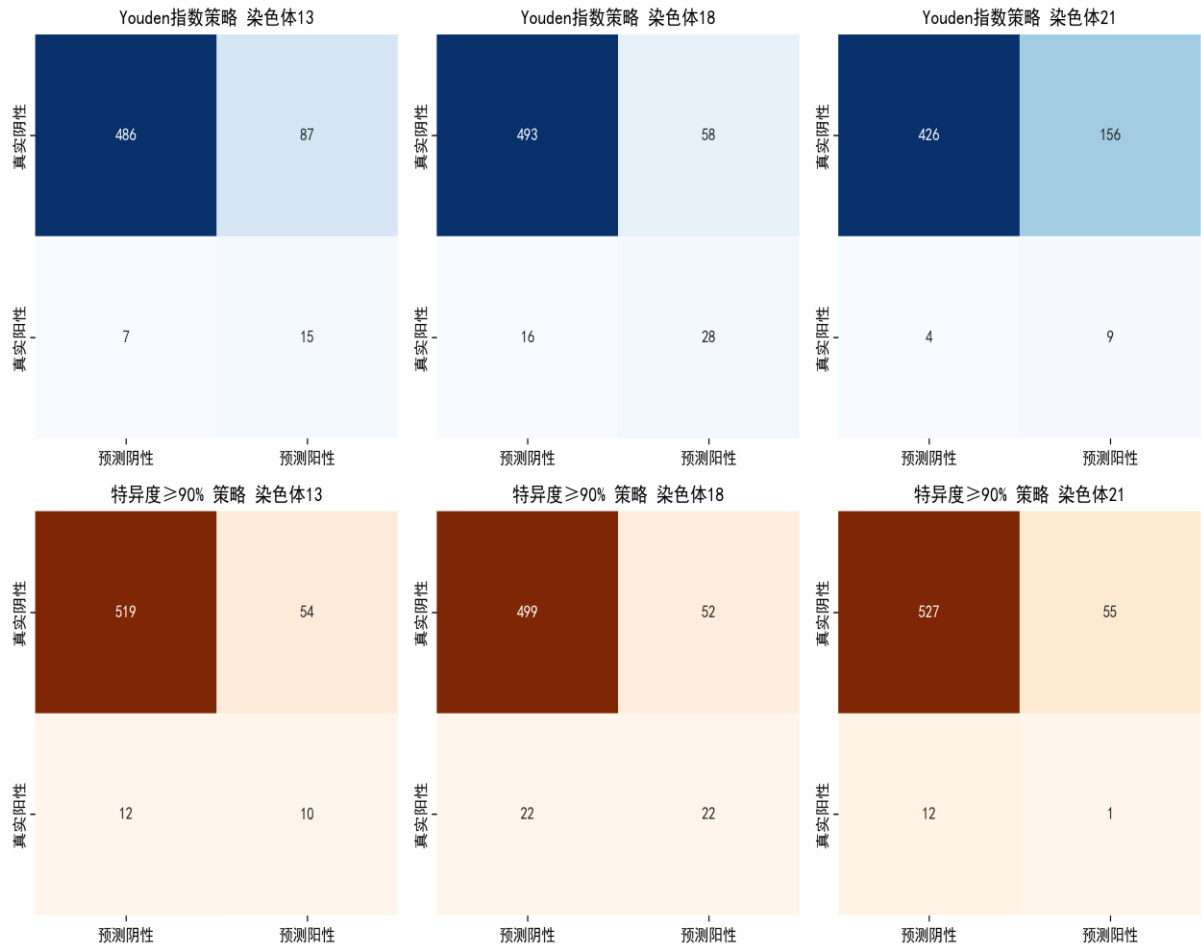


图 26 三条染色体的 Youden's Index 和特异性指标预测评估混淆矩阵图

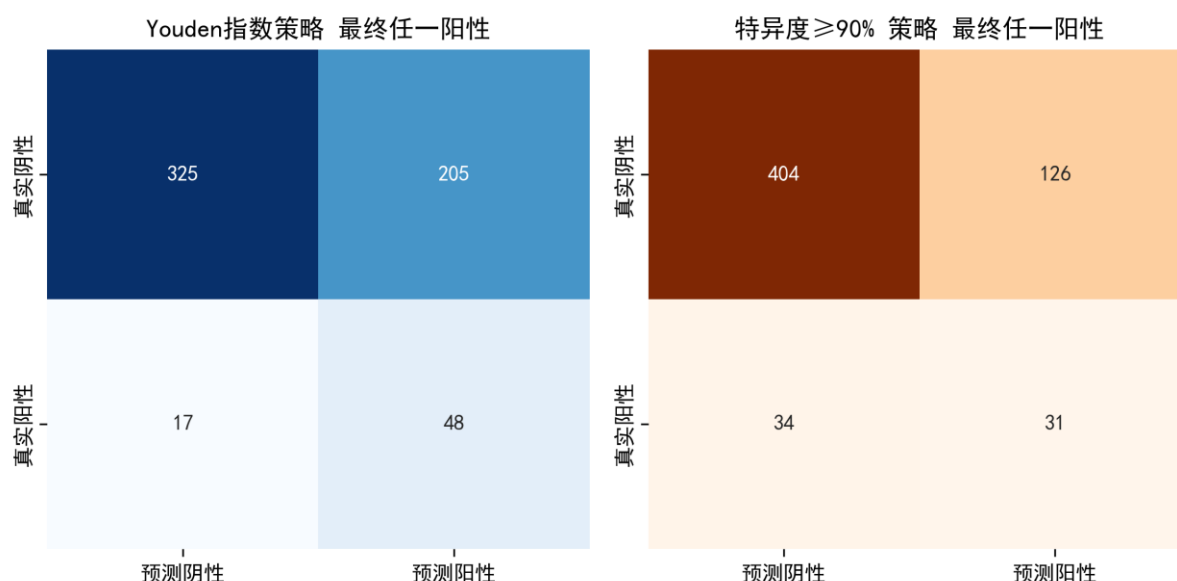


图 27 综合不同染色体、Youden's Index 和特异性指标预测评估混淆矩阵图

观察图 26 和图 27 可以发现，不论是在单染色体判定还是多染色体联合判定结果中特异度 $\geq 90\%$ 的准确性和灵敏度均优于 Youden's Index，同时多染色体联合判定能进一步提升预测准确性，更加符合 NIPT 临床检测需求。

10.4 三种判别女胎异常方案

基于上述结果我们在此处给出问题四的三种判别方案（三种模型的操作方式，可解释性，操作难度均不同），分别是通过 GC 值和 Z 值代入公式进行加权的 GC 和 Z 值赋判别法，带有黑箱性质的 OOF 阈值法，按条件进行判断的决策树模型，三种模型的设置主要是考虑到在实际决策中可能会面临不同的场景，需要进行快速显示判断还是进行黑箱拟合得到更精确的解：

1.GC 和 Z 值赋权判别法：

对于每条染色体 $j \in \{13, 18, 21\}$ ，按 BMI 组 g 使用对应系数 a,b,c,d,e,f，判定阈值 Z（给定目标概率 p_0 ）：

$$Z_j = \frac{\text{logit}(p_0) - a \cdot c \cdot GC_{total} - d \cdot GC_j}{b + e \cdot GC_{total} + f \cdot GC_j}, \quad (25)$$

其中， $\text{logit}(p) = \ln[p/(1-p)]$ 。

其中根据 GAMM 的拟合结果我们给出不同 BMI 分组下的个性化系数矩阵，更能够符合不同 BMI 分组人群的特点。

表 11 不同 BMI 分组下的个性化系数矩阵表

染色体	BMI 组	a	b	c	d	e	f
13	[25.59, 30.56]	-2.8	0.032	-1.1	-1.1	0.0087	0.023
13	[30.56, 31.63]	12	-33	4.2	-45	17	67
13	[31.63, 32.74)	-74	40	5.2	1.8e+2	-82	-18
13	[32.74, 34.47]	-3	-0.52	-1.2	-1.2	-0.2	-0.2
13	[34.47, 45.73]	-1.2e+2	85	-24	2.2e+2	1.3	-2.9e+2
18	[25.59, 30.56]	-5.1	-6	-6.1	11	-21	38
18	[30.56, 31.63]	65	-12	-1.5e+2	-21	16	16

18	[31.63, 32.74]	-86	-18	1.9	2.1e+2	-6.6	53
18	[32.74, 34.47]	-0.6	24	-46	41	-1e+2	44
18	[34.47, 45.73]	-1.3e+2	62	-34	3.7e+2	1.3e+2	-2.9e+2
21	[25.59, 30.56]	8.2	-29	13	-43	-8.2	81
21	[30.56, 31.63]	12	-33	-3.4	-36	11	71
21	[31.63, 32.74]	-2.8	0.52	-1.1	-1.1	0.21	0.21
21	[32.74, 34.47]	-3.3	0.5	-1.3	-1.3	0.2	0.19

我们对于每一条染色体进行判定，如果 $Z_j \geq Z_j$ 则判定为该染色体异常，否则为正常。如果任意一个染色体出现异常我们则判定女胎异常。注：若分母 $b + e \cdot GC_{total} + f \cdot GC_j \approx 0$ ，则回退到闭式规则 $Z_j \geq 3$ 。

2.OOF 阈值法：

本方法基于上述分析中的逻辑回归模型得到，先在按孕妇分组的交叉验证中得到每条染色体的样本外概率 $\hat{p}$ ，再按 BMI 组(可叠加 GC 层)为每组选择阈值 t_g ，最后以 $\hat{p} \geq t_g$ 判阳，具体的流程图如图 28 所示：

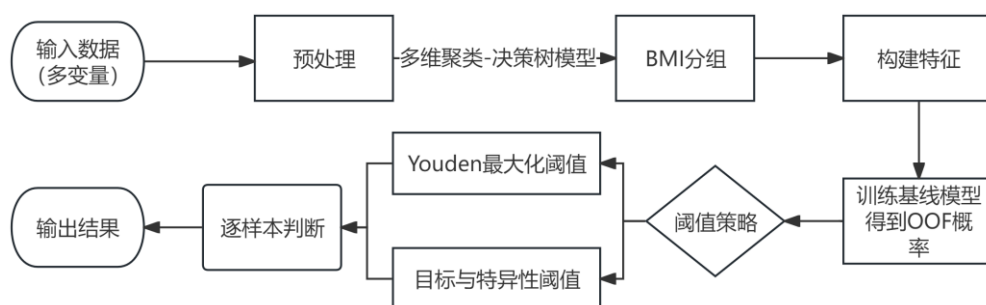


图 28 OOF 阈值法流程图

在下面我们依旧给出不同 BMI 分组下的 OOF 阈值，具体阈值如下表 12 所示：

表 12 不同 BMI 分组下的 OOF 阈值表

染色体	BMI 组	阈值
13	[25.594, 30.555]	0.6815436379827639
13	[30.555, 31.625]	0.10793596102554925
13	[31.625, 32.739]	0.10248686314766335
13	[32.739, 34.467]	0.11654364091269703
13	[34.467, 45.725]	0.9709541864209809
18	[25.594, 30.555]	0.5808832185996341
18	[30.555, 31.625]	0.27128077623457425
18	[31.625, 32.739]	0.6125064278823358
18	[32.739, 34.467]	0.28094130663979505
18	[34.467, 45.725]	0.3506341614615014
21	[25.594, 30.555]	0.16695891997782678
21	[30.555, 31.625]	0.024518443281812914
21	[31.625, 32.739]	0.017164530365332744
21	[32.739, 34.467]	0.00013620040757587523
21	[34.467, 45.725]	0.5

根据上述不同 BMI 分组对应的阈值 t_g ，我们将其代入 OOF 阈值法中进行处理，将所得变量输入进去即可得到对应结果。

3.决策树纯显示判断:

在此处我们给出一个纯显示的判别方式，基于该方法，我们可以不用计算得到，下述判断流程图基于模型比较中的决策树判断方式进而得到，具体流程图如下图 29 所示：

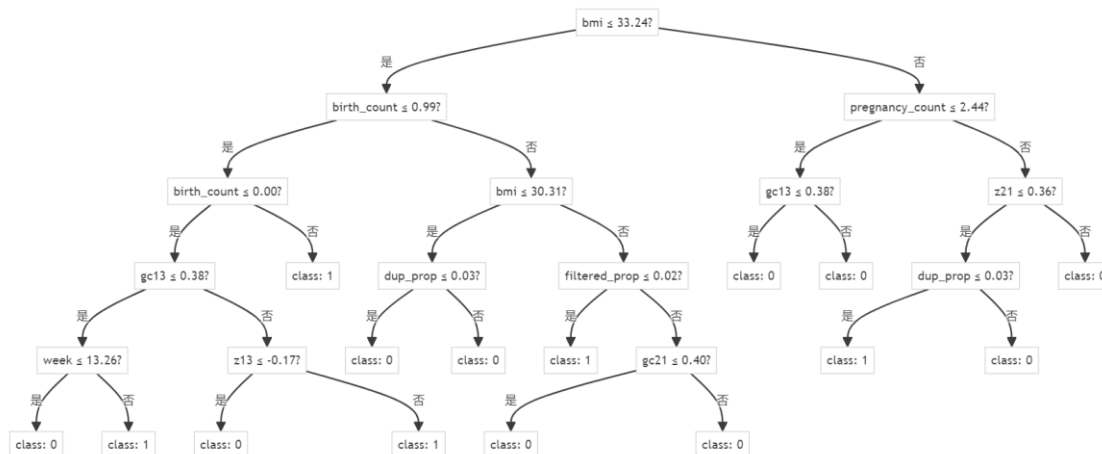


图 29 决策树纯显示判断流程图[7]

十一、模型评价

● 模型优点:

1.BMI 分组方法选用多维聚类-决策树区间映射模型可以利用所有可用变量特征的同时不引入任何的主观因素，仅依赖数据进行判断，抗干扰能力强，组内个体有着更相近的特征，更加符合实际情况，通过 Bootstrap 稳定性检验和敏感性分析加强所得区间的稳定性。

2.在四个问题中均进行了模型比较，通过比较不同模型之间的比较，我们的模型均表现优越，速度快准确性强。

3.针对非线性、近似正态，弱相关性和独立性特点的数据，我们恰当的选择了 GAM 作为我们通篇的基石模型，三问均在此基础上进行改进，该模型对于本文数据和问题能够较好适配

● 模型不足:

1.多维聚类-决策树区间映射模型在多维数据的变动情况下有待提高，因为本题中的样本数据仅有 1000 条左右，对于聚类的分组，每一组内的数量较少在数据波动时比较容易发生变化，若数据量增加，可能会有更好的表现效果。

十二、参考文献

- [1].DeepSeek-V3, DeepSeek-V3, 深度求索 (DeepSeek), 2025-09-05.
- [2].Skrzypek, H., Hui, L. (2017). Noninvasive prenatal testing for fetal aneuploidy and single gene disorders. Best Practice & Research Clinical Obstetrics & Gynaecology, vol 42, pp. 26–38. Elsevier, Amsterdam. <https://doi.org/10.1016/j.bpobgyn.2017.02.007>
- [3].Özgür Asar, David Bolin, Peter J. Diggle, Jonas Wallin, Linear Mixed Effects

- Models for Non-Gaussian Continuous Repeated Measurement Data, Journal of the Royal Statistical Society Series C: Applied Statistics, Volume 69, Issue 5, November 2020, Pages 1015–1065, <https://doi.org/10.1111/rssc.12405>
- [4]. Wood, S.N. (2017). Generalized Additive Models: An Introduction with R, Second Edition (2nd ed.). Chapman and Hall/CRC. <https://doi.org/10.1201/9781315370279>.
- [5]. 豆包, 字节跳动自研大模型, 北京字节跳动科技有限公司, 2025-09-06.
- [6]. Mahon, L., Lukasiewicz, T. (2021). Selective Pseudo-Label Clustering. In: Edelkamp, S., Möller, R., Rueckert, E. (eds) KI 2021: Advances in Artificial Intelligence. KI 2021. Lecture Notes in Computer Science(), vol 12873. Springer, Cham. https://doi.org/10.1007/978-3-030-87626-5_12.
- [7]. ChatGPT, GPT-5, OpenAI, 2025-09-07.

十三、附录

支撑材料文件列表	代码行数	备注
查询重复采血比例.py	105 行	查询重复采血使用
检测-末次月经时间.py	200 行	计算检测时间与末次月经时间差值
Pearson 相关性分析.py	220 行	对数据变量进行 Pearson 相关性分析
缺失值处理.py	138 行	对数据变量进行缺失值处理
问题一.py	1140 行	集成问题一求解操作，模型评价，显著性检验
正态性检验.py	178 行	利用 PP 图进行正态性检验
检测误差.py	495 行	计算检测误差扰动
雷达图.py	109 行	绘制模型评价指标的雷达图
问题二.py	977 行	集成问题二求解，模型评价，扰动
问题三.py	1752 行	集成问题二求解，模型评价，扰动
问题四.py	1936 行	集成问题四的求解，模型评价
描述性统计分析.py	357 行	描述性统计分析相关代码
年龄身高检查.py	350 行	进行升高年龄检查
附件_修正后.xlsx		
AI 工具使用详情.pdf		

因为我们代码行数共计 7957 行在此处由于篇幅限制我们只放置问题 3.py 和问题 4.py 的代码，其余代码均放置在支撑材料中在此处不做展示。

问题三代码
<pre> import os import re from typing import List, Tuple, Optional import numpy as np import pandas as pd import matplotlib matplotlib.use('Agg') import matplotlib.pyplot as plt from matplotlib import font_manager from sklearn.cluster import KMeans from sklearn.preprocessing import StandardScaler from sklearn.linear_model import LogisticRegression from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier, RandomForestRegressor from sklearn.tree import DecisionTreeClassifier from sklearn import metrics from sklearn.model_selection import GroupKFold import time from mpl_toolkits.mplot3d import Axes3D # noqa: F401 </pre>

```

from numpy.linalg import eig
try:
    from imblearn.over_sampling import SMOTE
except Exception:
    SMOTE = None
XGBClassifier = None
LGBMClassifier = None

EX_P = r"C:\Users\27832\Desktop\附件.xlsx"
OU_D = os.path.join(os.path.dirname(__file__), "输出")
os.makedirs(OU_D, exist_ok=True)

WE_MI = 10.0
WE_MA = 28.0
FI_WE_RA = True

try:
    simhei_path = r"C:\Windows\Fonts\simhei.ttf"
    if os.path.exists(simhei_path):
        font_manager.fontManager.addfont(simhei_path)
        plt.rcParams["font.sans-serif"] = ["SimHei"]
        plt.rcParams["font.family"] = "sans-serif"
        plt.rcParams["axes.unicode_minus"] = False
except Exception:
    plt.rcParams["font.sans-serif"] = ["SimHei"]
    plt.rcParams["font.family"] = "sans-serif"
    plt.rcParams["axes.unicode_minus"] = False

def _pa_we(x: object) -> Optional[float]:
    if pd.isna(x):
        return None
    s = str(x).strip()
    try:
        if 'w' in s or 'W' in s:
            parts = s.replace('W', 'w').replace('w', '+').split('+')
            w = float(parts[0])
            if len(parts) > 1 and parts[1]:
                w += float(parts[1]) / 7.0
            return w
        return float(s)
    
```

```
except Exception:
    m = re.findall(r"(\d+)", s)
    if not m:
        return None
    w = int(m[0]); d = int(m[1]) if len(m) > 1 else 0
    return w + d / 7.0
```

```
def _to_np_aw(s: pd.Series) -> pd.Series:
    s_str = s.astype(str)
    has_pct = s_str.str.contains('%', na=False)
    s_clean = s_str.str.replace('%', "", regex=False)
    vals = pd.to_numeric(s_clean, errors='coerce')
    vals.loc[has_pct] = vals.loc[has_pct] / 100.0
    return vals.astype(float)
```

```
def _fi_fi_co(df: pd.DataFrame, keywords: List[str]) -> Optional[str]:
    cols = list(df.columns)
    low_map = {c: str(c).lower() for c in cols}
    for c in cols:
        s = low_map[c]
        for kw in keywords:
            if kw.lower() in s:
                return c
    return None
```

```
def _cl_gp(series: pd.Series, cap_at: int = 3) -> pd.Series:
    def clean_val(x):
        if pd.isna(x):
            return np.nan
        try:
            val = float(x)
            return min(val, cap_at)
        except:
            s = str(x)
            matches = re.findall(r'\d+', s)
            if matches:
                val = float(matches[0])
                return min(val, cap_at)
            return np.nan
```

```
return series.apply(clean_val)
```

```
def lo_ma_sh(pa: str, week_min: float = 10.0, week_max: float = 25.0, filter_week_range:
bool = True) -> pd.DataFrame:
    df = pd.read_excel(pa, sheet_name=0)
    c = {k: None for k in ['patient_id', 'week', 'bmi', 'y_conc']}
    for col in df.columns:
        s = str(col)
        if c['patient_id'] is None and any(k in s for k in ['孕妇代码', '样本序号', '样本编号', 'ID']):
            c['patient_id'] = col
        if c['week'] is None and any(k in s for k in ['孕周', '检测孕周', '周数']):
            c['week'] = col
        if c['bmi'] is None and any(k in s for k in ['BMI', '孕妇 BMI']):
            c['bmi'] = col
        if c['y_conc'] is None and any(k in s for k in ['Y 染色体浓度', 'Y 浓度', 'Y 比例']):
            c['y_conc'] = col
    df = df.rename(columns={
        c['patient_id']: 'patient_id',
        c['week']: 'week_raw',
        c['bmi']: 'bmi',
        c['y_conc']: 'y_conc',
    })
    df['week'] = df['week_raw'].apply(_pa_we)
    df['bmi'] = pd.to_numeric(df['bmi'], errors='coerce')
    df['y_conc'] = pd.to_numeric(df['y_conc'], errors='coerce')
    df = df.dropna(subset=['patient_id', 'week', 'bmi', 'y_conc'])
    if bool(filter_week_range):
        df = df[(df['week'] >= float(week_min)) & (df['week'] <= float(week_max))]
    df['achieved'] = (df['y_conc'] >= 0.04).astype(int)

    col_age = _fi-fi_co(df, ['年龄', 'age', '孕妇年龄'])
    if col_age is not None:
        df['age'] = pd.to_numeric(df[col_age], errors='coerce')

    col_height = _fi-fi_co(df, ['身高', 'height', '孕妇身高'])
    if col_height is not None:
        h_val = pd.to_numeric(df[col_height], errors='coerce')
        if h_val.notna().any() and np.nanpercentile(h_val.dropna(), 95) > 10:
            h_val = h_val / 100.0
```

```

df['height_m'] = h_val

col_weight = _fi-fi_co(df, ['体重', 'weight', '孕妇体重'])
if col_weight is not None:
    df['weight_kg'] = pd.to_numeric(df[col_weight], errors='coerce')

col_x_conc = _fi-fi_co(df, ['x 染色体浓度', 'x 染色体', 'x 浓度', 'x 比例', 'x_conc'])
if col_x_conc is not None:
    x_vals = _to_np_aw(df[col_x_conc])
    if x_vals.notna().any() and x_vals.max() > 1.5 and x_vals.max() <= 100:
        x_vals = x_vals / 100.0
    df['x_conc'] = x_vals

col_total_reads = _fi-fi_co(df, ['原始测序数据', '总读段数', '原始读段数',
'total_reads'])
if col_total_reads is not None:
    df['total_reads'] = pd.to_numeric(df[col_total_reads], errors='coerce')

col_map_ratio = _fi-fi_co(df, ['比对率', '比对的比例', 'mapping', 'map_ratio', '参考基因
组'])
if col_map_ratio is not None:
    df['map_ratio'] = _to_np_aw(df[col_map_ratio])

col_dup_ratio = _fi-fi_co(df, ['重复读段', '重复率', 'duplicate', 'dup_ratio'])
if col_dup_ratio is not None:
    df['dup_ratio'] = _to_np_aw(df[col_dup_ratio])

col_unique_reads = _fi-fi_co(df, ['唯一比对', 'unique', '读段数'])
if col_unique_reads is not None:
    df['unique_reads'] = pd.to_numeric(df[col_unique_reads], errors='coerce')

col_gc_content = _fi-fi_co(df, ['gc 含量', 'gc content', 'gc%', 'gc_content'])
if col_gc_content is not None:
    df['gc_content'] = _to_np_aw(df[col_gc_content])

z_mappings = {
    'z13': ['13 号染色体的 z 值', 'z13', 'z_13', 'z-13'],
    'z18': ['18 号染色体的 z 值', 'z18', 'z_18', 'z-18'],
    'z21': ['21 号染色体的 z 值', 'z21', 'z_21', 'z-21'],
    'zX': ['x 染色体的 z 值', 'zx', 'z_x', 'z-x'],
    'zY': ['y 染色体的 z 值', 'zy', 'z_y', 'z-y']
}

```

```

for z_name, keywords in z_mappings.items():
    col_z = _fi-fi-co(df, keywords)
    if col_z is not None:
        df[z_name] = pd.to_numeric(df[col_z], errors='coerce')

col_y_conc_extra = _fi-fi-co(df, ['y 染色体浓度', 'y 染色体', 'y 浓度'])

gc_chr_mappings = {
    'gc13': ['13 号染色体的 gc 含量', 'gc13', 'chr13_gc'],
    'gc18': ['18 号染色体的 gc 含量', 'gc18', 'chr18_gc'],
    'gc21': ['21 号染色体的 gc 含量', 'gc21', 'chr21_gc']
}

for gc_name, keywords in gc_chr_mappings.items():
    col_gc = _fi-fi-co(df, keywords)
    if col_gc is not None:
        df[gc_name] = _to_np_aw(df[col_gc])

col_filtered_ratio = _fi-fi-co(df, ['过滤掉的读段数', '过滤率', 'filtered', 'filter_ratio'])
if col_filtered_ratio is not None:
    df['filtered_ratio'] = _to_np_aw(df[col_filtered_ratio])

col_gravidity = _fi-fi-co(df, ['孕次', 'gravidity', '怀孕次数'])
if col_gravidity is not None:
    df['gravidity'] = _cl_gp(df[col_gravidity], cap_at=3)

col_parity = _fi-fi-co(df, ['产次', 'parity', '生产次数'])
if col_parity is not None:
    df['parity'] = _cl_gp(df[col_parity], cap_at=3)

detected_vars = []
optional_var_names = [
    'age', 'height_m', 'weight_kg', 'x_conc', 'total_reads', 'map_ratio', 'dup_ratio',
    'unique_reads',
    'gc_content', 'z13', 'z18', 'z21', 'zX', 'zY', 'gc13', 'gc18', 'gc21', 'filtered_ratio',
    'gravidity', 'parity'
]

for var_name in optional_var_names:
    if var_name in df.columns and not df[var_name].isna().all():
        non_na_count = df[var_name].notna().sum()

```

```

        detected_vars.append(f'{var_name}(n={non_na_count})')

    if detected_vars:
        print(f'✓ 检测到可选变量: {', '.join(detected_vars)}')
    else:
        print("X 未检测到可选变量，仅使用核心变量(bmi, week, y_conc)")

    return df.reset_index(drop=True)

def elbow_kmeans_multi(features: np.ndarray, k_min: int = 3, k_max: int = 6) -> Tuple[int,
List[int], List[float]]:
    inertias = []
    ks = list(range(k_min, k_max + 1))
    for k in ks:
        km = KMeans(n_clusters=k, n_init=20, random_state=42)
        km.fit(features)
        inertias.append(float(km.inertia_))
    x1, y1 = ks[0], inertias[0]
    x2, y2 = ks[-1], inertias[-1]
    best_k = ks[0]
    best_d = -np.inf
    for k, y in zip(ks, inertias):
        num = abs((y2 - y1) * k - (x2 - x1) * y + x2*y1 - y2*x1)
        den = np.sqrt((y2 - y1)**2 + (x2 - x1)**2)
        d = num / (den + 1e-12)
        if d > best_d:
            best_d = d
            best_k = k
    return best_k, ks, inertias

def build_kmeans_groups_multi(df: pd.DataFrame, k: int, feature_cols: List[str] = None) ->
Tuple[KMeans, StandardScaler, np.ndarray, List[dict]]:
    if feature_cols is None:
        feature_cols = ['bmi', 'week', 'y_conc']

    feats_list = []
    for col in feature_cols:
        if col in df.columns and not df[col].isna().all():

```

```

        feats_list.append(df[col].astype(float).values)

if not feats_list:
    raise ValueError("没有可用的特征列")

feats = np.column_stack(feats_list)
scaler = StandardScaler().fit(feats)
Z = scaler.transform(feats)
km = KMeans(n_clusters=k, n_init=20, random_state=42).fit(Z)
labels = km.labels_
df_lab = df.copy()
df_lab['__label__'] = labels
group_info: List[dict] = []
for lab, g in df_lab.groupby('__label__'):
    g_bmi = g['bmi'].astype(float)
    g_week = g['week'].astype(float)
    g_y = g['y_conc'].astype(float)
    group_info.append({
        'label': int(lab),
        'n': int(len(g)),
        'bmi_min': float(g_bmi.min()), 'bmi_max': float(g_bmi.max()), 'bmi_med':
float(g_bmi.median()),
        'week_min': float(g_week.min()), 'week_max': float(g_week.max()),
        'y_min': float(g_y.min()), 'y_max': float(g_y.max()),
    })
group_info.sort(key=lambda d: d['bmi_med'])
old_to_new = {gi['label']: i for i, gi in enumerate(group_info)}
labels = np.array([old_to_new[int(l)] for l in labels], dtype=int)
for gi in group_info:
    gi['label'] = int(old_to_new[int(gi['label'])])
return km, scaler, labels, group_info

def _compute_group_info_from_labels(df: pd.DataFrame, labels: np.ndarray) ->
Tuple[np.ndarray, List[dict]]:
    df_lab = df.copy()
    df_lab['__label__'] = labels
    group_info: List[dict] = []
    for lab, g in df_lab.groupby('__label__'):
        g_bmi = g['bmi'].astype(float)
        g_week = g['week'].astype(float)
        g_y = g['y_conc'].astype(float)

```

```

        group_info.append({
            'label': int(lab),
            'n': int(len(g)),
            'bmi_min': float(g_bmi.min()), 'bmi_max': float(g_bmi.max()), 'bmi_med':
float(g_bmi.median()),
            'week_min': float(g_week.min()), 'week_max': float(g_week.max()),
            'y_min': float(g_y.min()), 'y_max': float(g_y.max()),
        })
    group_info.sort(key=lambda d: d['bmi_med'])
    old_to_new = {gi['label']: i for i, gi in enumerate(group_info)}
    labels_new = np.array([old_to_new[int(l)] for l in labels], dtype=int)
    for gi in group_info:
        gi['label'] = int(old_to_new[int(gi['label'])])
    return labels_new, group_info

```

```

def enforce_min_group_size(df: pd.DataFrame, labels: np.ndarray, min_n: int = 20) ->
Tuple[np.ndarray, List[dict]]:
    labels_curr = labels.copy()
    while True:
        labels_curr, gi_list = _compute_group_info_from_labels(df, labels_curr)
        if len(gi_list) <= 1:
            return labels_curr, gi_list
        sizes = np.array([gi['n'] for gi in gi_list], dtype=int)
        if np.all(sizes >= int(min_n)):
            return labels_curr, gi_list
        i_small = int(np.argmin(sizes))
        if i_small == 0:
            j_nei = 1
        elif i_small == len(gi_list) - 1:
            j_nei = len(gi_list) - 2
        else:
            d_left = abs(gi_list[i_small]['bmi_med'] - gi_list[i_small-1]['bmi_med'])
            d_right = abs(gi_list[i_small]['bmi_med'] - gi_list[i_small+1]['bmi_med'])
            j_nei = i_small-1 if d_left <= d_right else i_small+1
        new_id = min(i_small, j_nei)
        old_id = max(i_small, j_nei)
        labels_tmp = labels_curr.copy()
        labels_tmp[(labels_curr == old_id) | (labels_curr == new_id)] = new_id
        labels_curr = labels_tmp

```

```

def build_bmi_intervals_min_count(
    bmi_series: pd.Series,
    k: int,
    min_n: int = 20,
) -> Tuple[List[Tuple[float, float]], List[int], List[float]]:
    b = pd.to_numeric(bmi_series, errors='coerce').dropna().astype(float).values
    n = int(b.size)
    if n == 0:
        return [], [], []
    b.sort()
    k_eff = int(min(k, max(1, n // max(1, int(min_n)))))
    cuts = [0]
    idx = 0
    for g in range(k_eff - 1):
        remain_bins = (k_eff - g)
        remain_n = n - idx
        max_end = n - (remain_bins - 1) * int(min_n)
        nominal = int(round(remain_n / remain_bins))
        next_idx = max(idx + int(min_n), min(max_end, idx + nominal))
        if next_idx <= idx:
            next_idx = min(n - (remain_bins - 1) * int(min_n), idx + int(min_n))
        cuts.append(next_idx)
        idx = next_idx
    cuts.append(n)
    intervals: List[Tuple[float, float]] = []
    counts: List[int] = []
    reps: List[float] = []
    for i in range(len(cuts) - 1):
        a = cuts[i]; bidx = cuts[i+1]
        seg = b[a:bidx]
        if seg.size == 0:
            continue
        lo = float(seg[0]); hi = float(seg[-1])
        intervals.append((round(lo, 3), round(hi, 3)))
        counts.append(int(seg.size))
        reps.append(float(np.median(seg)))
    fixed: List[Tuple[float, float]] = []
    for i, (lo, hi) in enumerate(intervals):
        if i > 0 and lo <= fixed[-1][1]:
            lo = fixed[-1][1] + 1e-3
        if hi < lo:
            hi = lo + 1e-3

```

```
        fixed.append((round(lo, 3), round(hi, 3)))
    return fixed, counts, reps
```

```
def build_bmi_intervals_by_tree(
    df: pd.DataFrame,
    labels: np.ndarray,
    min_n: int = 30,
    max_leaf_nodes: int = 6,
    round_digits: int = 3,
) -> Tuple[List[Tuple[float, float]], List[int], List[float]]:
    bmi_all = pd.to_numeric(df['bmi'], errors='coerce').astype(float).values
    y = np.asarray(labels, dtype=int)
    mask = np.isfinite(bmi_all) & np.isfinite(y)
    X = bmi_all[mask].reshape(-1, 1)
    y = y[mask]
    if X.size == 0:
        raise RuntimeError('BMI 为空，无法训练决策树')
    tree = DecisionTreeClassifier(
        criterion='entropy',
        random_state=42,
        max_leaf_nodes=max_leaf_nodes,
        min_samples_leaf=int(min_n),
    )
    tree.fit(X, y)
    thresholds = tree.tree_.threshold
    feats = tree.tree_.feature
    cuts = [float(t) for t, f in zip(thresholds, feats) if f == 0 and np.isfinite(t)]
    cuts = sorted({round(c, round_digits) for c in cuts})
    global_lo = float(np.nanmin(bmi_all))
    global_hi = float(np.nanmax(bmi_all))
    bounds = [global_lo] + cuts + [global_hi]
    clean_bounds = [bounds[0]]
    for v in bounds[1:]:
        if v <= clean_bounds[-1]:
            v = clean_bounds[-1] + 10**(-round_digits)
        clean_bounds.append(v)
    intervals: List[Tuple[float, float]] = []
    counts: List[int] = []
    reps: List[float] = []
    for i in range(len(clean_bounds) - 1):
        lo = round(float(clean_bounds[i]), round_digits)
```

```

hi = round(float(clean_bounds[i+1]), round_digits)
if i < len(clean_bounds) - 2:
    m = (bmi_all >= lo) & (bmi_all < hi)
else:
    m = (bmi_all >= lo) & (bmi_all <= hi)
n_i = int(np.sum(m))
if n_i == 0:
    continue
intervals.append((lo, hi))
counts.append(n_i)
reps.append(float(np.median(bmi_all[m])))
if any(c < int(min_n) for c in counts) and len(intervals) > 1:
    while True:
        if len(intervals) <= 1:
            break
        idx_small = int(np.argmin(counts))
        if counts[idx_small] >= int(min_n):
            break
        if idx_small == 0:
            j = 1
        elif idx_small == len(intervals) - 1:
            j = len(intervals) - 2
        else:
            d_left = abs(reps[idx_small] - reps[idx_small-1])
            d_right = abs(reps[idx_small] - reps[idx_small+1])
            j = idx_small-1 if d_left <= d_right else idx_small+1
        lo = min(intervals[idx_small][0], intervals[j][0])
        hi = max(intervals[idx_small][1], intervals[j][1])
        n_new = counts[idx_small] + counts[j]
        reps_new = float(np.median(bmi_all[(bmi_all >= lo) & (bmi_all <= hi)]))
        keep = [k for k in range(len(intervals)) if k not in (idx_small, j)]
        intervals = [intervals[k] for k in keep] + [(round(lo, round_digits), round(hi,
round_digits))]
        counts = [counts[k] for k in keep] + [n_new]
        reps = [reps[k] for k in keep] + [reps_new]
        order = np.argsort([iv[0] for iv in intervals])
        intervals = [intervals[k] for k in order]
        counts = [counts[k] for k in order]
        reps = [reps[k] for k in order]
    return intervals, counts, reps

```

```

def evaluate_interval_purity(df: pd.DataFrame, intervals: List[Tuple[float, float]], labels:
np.ndarray) -> dict:
    bmi_all = df['bmi'].astype(float).values
    purities = []
    dominant_clusters = []

    for i, (lo, hi) in enumerate(intervals):
        if i < len(intervals) - 1:
            mask = (bmi_all >= lo) & (bmi_all < hi)
        else:
            mask = (bmi_all >= lo) & (bmi_all <= hi)

        if np.sum(mask) == 0:
            purities.append(0.0)
            dominant_clusters.append(-1)
            continue

        cluster_counts = np.bincount(labels[mask])
        purity = float(np.max(cluster_counts) / np.sum(cluster_counts))
        dominant_cluster = int(np.argmax(cluster_counts))

        purities.append(purity)
        dominant_clusters.append(dominant_cluster)

    overall_purity = float(np.mean(purities))
    return {
        'purities': purities,
        'dominant_clusters': dominant_clusters,
        'overall_purity': overall_purity
    }

def bootstrap_tree_stability(
    df: pd.DataFrame,
    labels: np.ndarray,
    n_bootstrap: int = 1000,
    min_n: int = 30,
    max_leaf_nodes: int = 6,
    random_state: int = 42
) -> dict:
    np.random.seed(random_state)
    n_samples = len(df)

```

```

all_thresholds = []
successful_runs = 0

for i in range(n_bootstrap):
    # Bootstrap 采样
    indices = np.random.choice(n_samples, size=n_samples, replace=True)
    df_boot = df.iloc[indices].reset_index(drop=True)
    labels_boot = labels[indices]

    try:
        bmi_boot = df_boot['bmi'].astype(float).values
        y_boot = labels_boot.astype(int)
        mask = np.isfinite(bmi_boot) & np.isfinite(y_boot)
        X_boot = bmi_boot[mask].reshape(-1, 1)
        y_boot = y_boot[mask]

        if X_boot.size == 0:
            continue

        tree = DecisionTreeClassifier(
            criterion='entropy',
            random_state=42,
            max_leaf_nodes=max_leaf_nodes,
            min_samples_leaf=int(min_n),
        )
        tree.fit(X_boot, y_boot)

        thresholds = tree.tree_.threshold
        feats = tree.tree_.feature
        cuts = [float(t) for t, f in zip(thresholds, feats) if f == 0 and np.isfinite(t)]
        all_thresholds.extend(cuts)
        successful_runs += 1

    except Exception:
        continue

if not all_thresholds:
    return {'thresholds': [], 'threshold_ci': [], 'stability_score': 0.0,
'bootstrap_success_rate': 0.0}

all_thresholds = np.array(all_thresholds)
unique_thresholds, counts = np.unique(all_thresholds, return_counts=True)

```

```

sorted_indices = np.argsort(counts)[::-1]
stable_thresholds = unique_thresholds[sorted_indices[:max_leaf_nodes-1]]

if len(stable_thresholds) > 1:
    stability_score = float(np.mean(counts[sorted_indices[:max_leaf_nodes-1]])) /
len(all_thresholds))
else:
    stability_score = 0.0

threshold_ci = []
for thresh in stable_thresholds:
    thresh_range = 0.05 * abs(thresh) + 0.1
    similar_thresholds = all_thresholds[np.abs(all_thresholds - thresh) <= thresh_range]
    if len(similar_thresholds) > 1:
        ci_low = float(np.percentile(similar_thresholds, 2.5))
        ci_high = float(np.percentile(similar_thresholds, 97.5))
    else:
        ci_low = ci_high = float(thresh)
    threshold_ci.append((ci_low, ci_high))

bootstrap_success_rate = successful_runs / n_bootstrap

return {
    'thresholds': stable_thresholds.tolist(),
    'threshold_ci': threshold_ci,
    'stability_score': stability_score,
    'bootstrap_success_rate': bootstrap_success_rate,
    'all_thresholds': all_thresholds.tolist(),
    'threshold_frequencies': dict(zip(unique_thresholds, counts)),
    'n_successful_runs': successful_runs
}

def visualize_tree_analysis(
    df: pd.DataFrame,
    intervals: List[Tuple[float, float]],
    labels: np.ndarray,
    stability_results: dict,
    purity_results: dict,
    output_dir: str
):

```

```

fig, axes = plt.subplots(2, 2, figsize=(15, 12))

ax1 = axes[0, 0]
if stability_results['all_thresholds']:
    ax1.hist(stability_results['all_thresholds'], bins=30, alpha=0.7, color='skyblue',
edgecolor='black')
    for thresh in stability_results['thresholds']:
        ax1.axvline(thresh, color='red', linestyle='--', alpha=0.8, label=f'稳定阈值:
{thresh:.1f}')
    ax1.set_xlabel('BMI 阈值')
    ax1.set_ylabel('Bootstrap 频次')
    ax1.set_title(f'阈值稳定性分析\n(成功率:
{stability_results["bootstrap_success_rate"]:.1%}, 稳定性:
{stability_results["stability_score"]:.3f})')
    ax1.legend()
else:
    ax1.text(0.5, 0.5, '无稳定阈值数据', ha='center', va='center',
transform=ax1.transAxes)
    ax1.set_title('阈值稳定性分析')

ax2 = axes[0, 1]
if purity_results['purities']:
    x_pos = range(len(intervals))
    bars = ax2.bar(x_pos, purity_results['purities'], alpha=0.7, color='lightgreen',
edgecolor='black')
    ax2.set_xlabel('BMI 区间索引')
    ax2.set_ylabel('簇纯度')
    ax2.set_title(f'区间簇纯度\n(平均纯度: {purity_results["overall_purity"]:.3f})')
    ax2.set_xticks(x_pos)
    ax2.set_xticklabels([f'{i}' for i in range(len(intervals))], rotation=45)

    for i, bar in enumerate(bars):
        height = bar.get_height()
        ax2.text(bar.get_x() + bar.get_width()/2., height + 0.01,
f'{height:.2f}', ha='center', va='bottom', fontsize=9)
else:
    ax2.text(0.5, 0.5, '无纯度数据', ha='center', va='center', transform=ax2.transAxes)
    ax2.set_title('区间簇纯度')

ax3 = axes[1, 0]
bmi_data = pd.to_numeric(df['bmi'], errors='coerce').dropna()
ax3.hist(bmi_data, bins=50, alpha=0.5, color='lightblue', edgecolor='black', label='BMI

```

分布')

```
for i, (lo, hi) in enumerate(intervals):
    ax3.axvline(lo, color='red', linestyle='-', alpha=0.7)
    if i == len(intervals) - 1:
        ax3.axvline(hi, color='red', linestyle='-', alpha=0.7)
    mid = (lo + hi) / 2
    ax3.text(mid, ax3.get_ylim()[1]*0.9, f'区间 {i}', ha='center', va='top',
             rotation=90,      fontsize=8,      bbox=dict(boxstyle="round,pad=0.1",
facecolor="white", alpha=0.8))

ax3.set_xlabel('BMI')
ax3.set_ylabel('样本数')
ax3.set_title('BMI 分布与区间划分')
ax3.legend()

ax4 = axes[1, 1]
unique_labels = np.unique(labels)
colors = plt.cm.Set3(np.linspace(0, 1, len(unique_labels)))

for i, cluster_id in enumerate(unique_labels):
    cluster_bmi = bmi_data[labels == cluster_id]
    if len(cluster_bmi) > 0:
        ax4.hist(cluster_bmi, alpha=0.6, color=colors[i], label=f'簇 {cluster_id}',
bins=30)

for lo, hi in intervals:
    ax4.axvline(lo, color='red', linestyle='--', alpha=0.5)
    if lo == intervals[-1][0]:
        ax4.axvline(hi, color='red', linestyle='--', alpha=0.5)

ax4.set_xlabel('BMI')
ax4.set_ylabel('样本数')
ax4.set_title('各簇在 BMI 轴的分布')
ax4.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.savefig(os.path.join(output_dir, '问题二_决策树 BMI 分析.png'), dpi=200,
bbox_inches='tight')
plt.close()

stability_report = []
```

```

    stability_report.append("=== 决策树 BMI 区间稳定性分析报告 ===\n")
    stability_report.append(f"Bootstrap 运行次数: {stability_results.get('n_successful_runs',
0)}}")
    stability_report.append(f"成功率: {stability_results.get('bootstrap_success_rate',
0):.1%}")
    stability_report.append(f"稳定性评分: {stability_results.get('stability_score', 0):.3f}")
    stability_report.append(f"平均区间纯度: {purity_results.get('overall_purity', 0):.3f}\n")

    stability_report.append("稳定阈值及置信区间:")
    for i, (thresh, ci) in enumerate(zip(stability_results.get('thresholds', []),
                                         stability_results.get('threshold_ci', []))):
        stability_report.append(f"      阈值 {i+1}: {thresh:.3f} [95%CI:
{ci[0]:.3f}-{ci[1]:.3f}]"")

    stability_report.append("\n 区间纯度详情:")
    for i, (interval, purity, dominant) in enumerate(zip(intervals,
purity_results.get('purities', []),
purity_results.get('dominant_clusters', []))):
        lo, hi = interval
        stability_report.append(f"  区间 {i}: [{lo:.3f}, {hi:.3f}] -> 纯度:{purity:.3f}, 主
导簇:{dominant}")

    with open(os.path.join(output_dir, '问题二_决策树稳定性报告.txt'), 'w',
encoding='utf-8') as f:
        f.write("\n".join(stability_report))

def mdlp_discretization(
    bmi_series: pd.Series,
    labels: np.ndarray,
    min_n: int = 30,
    max_intervals: int = 8
) -> Tuple[List[Tuple[float, float]], List[int], List[float]]:
    import math

    bmi_data = pd.to_numeric(bmi_series, errors='coerce').astype(float).values
    y = np.asarray(labels, dtype=int)
    mask = np.isfinite(bmi_data) & np.isfinite(y)
    X = bmi_data[mask]
    y = y[mask]

```

```

if len(X) == 0:
    raise RuntimeError('BMI 数据为空，无法进行 MDLP 离散化')

sorted_indices = np.argsort(X)
X_sorted = X[sorted_indices]
y_sorted = y[sorted_indices]

def entropy(labels):

    if len(labels) == 0:
        return 0
    _, counts = np.unique(labels, return_counts=True)
    probs = counts / len(labels)
    return -np.sum(probs * np.log2(probs + 1e-10))

def mdl_criterion(left_labels, right_labels, total_labels):

    n = len(total_labels)
    n_left = len(left_labels)
    n_right = len(right_labels)

    if n_left == 0 or n_right == 0:
        return float('inf')

    total_entropy = entropy(total_labels)
    weighted_entropy = (n_left/n) * entropy(left_labels) + (n_right/n) *
entropy(right_labels)
    info_gain = total_entropy - weighted_entropy

    k = len(np.unique(total_labels)) # 类别数
    description_length = (math.log2(n) + k * math.log2(n)) / n

    return description_length - info_gain

def find_best_split(start_idx, end_idx, depth=0):

    if end_idx - start_idx < 2 * min_n or depth >= math.log2(max_intervals):
        return None

    segment_X = X_sorted[start_idx:end_idx]
    segment_y = y_sorted[start_idx:end_idx]

```

```

best_split = None
best_mdl = float('inf')

for i in range(start_idx + min_n, end_idx - min_n):
    if X_sorted[i] == X_sorted[i-1]:
        continue

    left_y = y_sorted[start_idx:i]
    right_y = y_sorted[i:end_idx]
    total_y = segment_y

    mdl_score = mdl_criterion(left_y, right_y, total_y)

    if mdl_score < best_mdl:
        best_mdl = mdl_score
        best_split = i

if best_split is not None and best_mdl < 0:
    split_value = (X_sorted[best_split-1] + X_sorted[best_split]) / 2
    return split_value, best_split

return None

def build_splits(start_idx, end_idx, depth=0):

    result = find_best_split(start_idx, end_idx, depth)
    if result is None:
        return []

    split_value, split_idx = result
    splits = [split_value]

    left_splits = build_splits(start_idx, split_idx, depth + 1)
    right_splits = build_splits(split_idx, end_idx, depth + 1)

    return left_splits + splits + right_splits

split_points = build_splits(0, len(X_sorted))
split_points = sorted(list(set(split_points)))

if not split_points:

```

```

print("MDLP 未找到有效分割点，使用等频分割")
n_bins = min(max_intervals, len(X) // min_n)
if n_bins <= 1:
    n_bins = 2
quantiles = np.linspace(0, 100, n_bins + 1)[1:-1]
split_points = [np.percentile(X, q) for q in quantiles]

x_min, x_max = float(np.min(X)), float(np.max(X))
boundaries = [x_min] + split_points + [x_max]
boundaries = sorted(list(set(boundaries)))

intervals = []
counts = []
representatives = []

for i in range(len(boundaries) - 1):
    lo, hi = boundaries[i], boundaries[i + 1]

    if i < len(boundaries) - 2:
        mask = (bmi_data >= lo) & (bmi_data < hi)
    else:
        mask = (bmi_data >= lo) & (bmi_data <= hi)

    count = int(np.sum(mask))
    if count == 0:
        continue

    intervals.append((round(lo, 3), round(hi, 3)))
    counts.append(count)
    representatives.append(float(np.median(bmi_data[mask])))

while len(intervals) > 1 and min(counts) < min_n:
    min_idx = int(np.argmin(counts))

    if min_idx == 0:
        merge_with = 1
    elif min_idx == len(intervals) - 1:
        merge_with = len(intervals) - 2
    else:
        left_count = counts[min_idx - 1]
        right_count = counts[min_idx + 1]
        merge_with = min_idx - 1 if left_count <= right_count else min_idx + 1

```

```

    new_lo = min(intervals[min_idx][0], intervals[merge_with][0])
    new_hi = max(intervals[min_idx][1], intervals[merge_with][1])
    new_count = counts[min_idx] + counts[merge_with]

    combined_mask = (bmi_data >= new_lo) & (bmi_data <= new_hi)
    new_rep = float(np.median(bmi_data[combined_mask]))

    keep_indices = [i for i in range(len(intervals)) if i not in (min_idx, merge_with)]
    intervals = [intervals[i] for i in keep_indices] + [(round(new_lo, 3), round(new_hi,
3))]

    counts = [counts[i] for i in keep_indices] + [new_count]
    representatives = [representatives[i] for i in keep_indices] + [new_rep]

    sort_order = np.argsort([iv[0] for iv in intervals])
    intervals = [intervals[i] for i in sort_order]
    counts = [counts[i] for i in sort_order]
    representatives = [representatives[i] for i in sort_order]

    return intervals, counts, representatives

def find_earliest_crossing(weeks: np.ndarray, probs: np.ndarray, p0: float) -> Optional[float]:
    hit = probs >= p0
    idx = np.where(hit)[0]
    if idx.size == 0:
        return None
    i = int(idx[0])
    if i == 0:
        return float(weeks[0])
    x0, x1 = float(weeks[i-1]), float(weeks[i])
    y0, y1 = float(probs[i-1]), float(probs[i])
    if y1 == y0:
        return float(x1)
    t = (p0 - y0) / (y1 - y0)
    t = float(np.clip(t, 0.0, 1.0))
    return x0 + t * (x1 - x0)

def find_risk_optimal_time(weeks: np.ndarray, probs: np.ndarray, w: float = 0.08, k_exp: float
= 0.25) -> float:
    weeks = weeks.astype(float)
    probs = np.clip(probs.astype(float), 1e-9, 1-1e-9)

```

```

late = np.maximum(0.0, weeks - 12.0)
penalty = np.expm1(k_exp * late)
loss = (1.0 - probs) + float(w) * penalty
j = int(np.argmin(loss))
return float(weeks[j])

```

```

def find_risk_optimal_time_general(
    weeks: np.ndarray,
    probs: np.ndarray,
    penalty_type: str,
    w: float,
    k_exp: float = 0.25,
    pow_k: float = 2.0,
) -> float:
    weeks = weeks.astype(float)
    probs = np.clip(probs.astype(float), 1e-9, 1-1e-9)
    late = np.maximum(0.0, weeks - 12.0)
    if penalty_type == 'linear':
        penalty = late
    elif penalty_type == 'poly':
        penalty = late ** float(pow_k)
    elif penalty_type == 'exp':
        penalty = np.expm1(float(k_exp) * late)
    else:
        raise ValueError(f'未知的 penalty_type: {penalty_type}')
    loss = (1.0 - probs) + float(w) * penalty
    j = int(np.argmin(loss))
    return float(weeks[j])

```

```

def _map_bmi_to_group_index(bmi_value: float, intervals: List[Tuple[float, float]]) -> int:
    if np.isnan(bmi_value):
        return 0
    for i, (lo, hi) in enumerate(intervals):
        if i < len(intervals) - 1:
            if (bmi_value >= lo) and (bmi_value < hi):
                return i
    else:
        if (bmi_value >= lo) and (bmi_value <= hi):
            return i
    if bmi_value < intervals[0][0]:

```

```

        return 0
    return len(intervals) - 1

def _evaluate_decision_metrics_by_Trisk(
    df: pd.DataFrame,
    intervals: List[Tuple[float, float]],
    t_risk_by_group: List[float],
) -> dict:
    from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix
    weeks = df['week'].astype(float).values
    bmis = df['bmi'].astype(float).values
    y_true = df['achieved'].astype(int).values
    y_pred = np.zeros_like(y_true)
    for idx in range(len(y_true)):
        gi = _map_bmi_to_group_index(bmis[idx], intervals)
        y_pred[idx] = 1 if weeks[idx] >= t_risk_by_group[gi] else 0
    acc = float(accuracy_score(y_true, y_pred))
    prec = float(precision_score(y_true, y_pred, zero_division=0))
    rec = float(recall_score(y_true, y_pred, zero_division=0))
    f1 = float(f1_score(y_true, y_pred, zero_division=0))
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    spec = float(tn / (tn + fp) if (tn + fp) > 0 else 0.0)
    return {
        'Accuracy': acc,
        'Precision': prec,
        'Recall': rec,
        'Specificity': spec,
        'F1': f1,
        'TN': int(tn), 'FP': int(fp), 'FN': int(fn), 'TP': int(tp),
    }

def _evaluate_decision_metrics_by_Trisk_labels(
    df: pd.DataFrame,
    labels: np.ndarray,
    t_risk_by_group: List[float],
) -> dict:
    from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix
    weeks = df['week'].astype(float).values

```

```

y_true = df['achieved'].astype(int).values
y_pred = np.zeros_like(y_true)
for idx in range(len(y_true)):
    gi = int(labels[idx])
    y_pred[idx] = 1 if weeks[idx] >= t_risk_by_group[gi] else 0
acc = float(accuracy_score(y_true, y_pred))
prec = float(precision_score(y_true, y_pred, zero_division=0))
rec = float(recall_score(y_true, y_pred, zero_division=0))
f1 = float(f1_score(y_true, y_pred, zero_division=0))
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
spec = float(tn / (tn + fp) if (tn + fp) > 0 else 0.0)
return {
    'Accuracy': acc,
    'Precision': prec,
    'Recall': rec,
    'Specificity': spec,
    'F1': f1,
    'TN': int(tn), 'FP': int(fp), 'FN': int(fn), 'TP': int(tp),
}

```

```

def _penalty_norm_from_weeks(weeks: np.ndarray, penalty_type: str, k_exp: float = 0.25,
pow_k: float = 2.0) -> np.ndarray:
    weeks = weeks.astype(float)
    late = np.maximum(0.0, weeks - 12.0)
    if penalty_type == 'linear':
        pen = late
    elif penalty_type == 'poly':
        pen = late ** float(pow_k)
    elif penalty_type == 'exp':
        pen = np.expm1(float(k_exp) * late)
    else:
        raise ValueError(f'未知的 penalty_type: {penalty_type}')
    m = float(np.max(pen))
    if m <= 0:
        return np.zeros_like(weeks, dtype=float)
    return pen / m

```

```

def _evaluate_penalized_probability_rule(
    df: pd.DataFrame,
    gam,
    p0: float,

```

```

    penalty_type: str,
    w_pen: float,
    k_exp: float = 0.25,
    pow_k: float = 2.0,
) -> dict:
    from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix
    Xs = np.column_stack([
        df['week'].astype(float).values,
        df['bmi'].astype(float).values,
    ])
    y_true = df['achieved'].astype(int).values
    p_hat = np.clip(gam.predict_proba(Xs), 1e-9, 1-1e-9)
    pen_norm = _penalty_norm_from_weeks(Xs[:,0], penalty_type, k_exp=k_exp,
    pow_k=pow_k)
    thr = np.clip(p0 + float(w_pen) * pen_norm, 0.0, 0.999)
    y_pred = (p_hat >= thr).astype(int)
    acc = float(accuracy_score(y_true, y_pred))
    prec = float(precision_score(y_true, y_pred, zero_division=0))
    rec = float(recall_score(y_true, y_pred, zero_division=0))
    f1 = float(f1_score(y_true, y_pred, zero_division=0))
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    spec = float(tn / (tn + fp) if (tn + fp) > 0 else 0.0)
    return {
        'Accuracy': acc,
        'Precision': prec,
        'Recall': rec,
        'Specificity': spec,
        'F1': f1,
        'TN': int(tn), 'FP': int(fp), 'FN': int(fn), 'TP': int(tp),
    }

def _earliest_crossing_with_curve(weeks: np.ndarray, probs: np.ndarray, thr: np.ndarray) ->
Optional[float]:
    hit = (probs >= thr)
    idx = np.where(hit)[0]
    if idx.size == 0:
        return None
    return float(weeks[int(idx[0])])

```

```

def main():
    try:
        from pygam import LogisticGAM, LinearGAM, s, te, l
    except Exception:
        print('需要安装 pygam 才能运行本脚本')
        return

    df = lo_ma_sh(EX_P, week_min=WE_MI, week_max=WE_MA,
filter_week_range=FI_WE_RA)
    if df.empty:
        print('数据为空')
        return

    cluster_vars = ['bmi', 'week', 'y_conc']
    optional_vars = [
        'age', 'height_m', 'weight_kg', 'x_conc', 'total_reads', 'map_ratio', 'dup_ratio',
'unique_reads',
        'gc_content', 'z13', 'z18', 'z21', 'zX', 'zY', 'gc13', 'gc18', 'gc21', 'filtered_ratio',
'gravity', 'parity'
    ]
    for v in optional_vars:
        if v in df.columns and not df[v].isna().all():
            cluster_vars.append(v)

    print(f'使用特征变量: {cluster_vars}')

    feats_list = []
    for col in cluster_vars:
        if col in df.columns and not df[col].isna().all():
            feats_list.append(df[col].astype(float).values)

    feats = np.column_stack(feats_list)
    scaler_elbow = StandardScaler().fit(feats)
    Z_elbow = scaler_elbow.transform(feats)
    best_k, ks, inertias = elbow_kmeans_multi(Z_elbow, k_min=2, k_max=10)
    plt.figure(figsize=(5,4))
    plt.plot(ks, inertias, marker='o')
    plt.axvline(best_k, color='r', linestyle='--', label=f'最佳 K={best_k}')
    plt.xlabel('K')
    plt.ylabel('Inertia')
    plt.legend()

```

```

plt.tight_layout()
plt.savefig(os.path.join(OU_D, '问题二_KMeans_肘部法则.png'), dpi=200)
plt.close()

km, scaler, labels, group_info = build_kmeans_groups_multi(df, k=best_k,
feature_cols=cluster_vars)
labels, group_info = enforce_min_group_size(df, labels, min_n=50)

print("\n=== 决策树 BMI 区间生成 ===")
try:
    intervals, interval_counts, bmi_rep_list = build_bmi_intervals_by_tree(
        df, labels, min_n=50, max_leaf_nodes=max(3, min(6, len(group_info))),
round_digits=3
    )
    print(f'决策树方法成功生成 {len(intervals)} 个 BMI 区间')

    purity_results = evaluate_interval_purity(df, intervals, labels)
    print(f'区间平均纯度: {purity_results['overall_purity']:.3f}')

    print("进行 Bootstrap 稳定性分析...")
    stability_results = bootstrap_tree_stability(
        df, labels, n_bootstrap=500, min_n=50, max_leaf_nodes=max(3, min(6,
len(group_info)))
    )
    print(f'阈值稳定性分数: {stability_results['stability_score']:.3f}')
    print(f'Bootstrap 成功率: {stability_results['bootstrap_success_rate']:.1%}')

    if stability_results['stability_score'] < 0.3:
        print("警告: 阈值稳定性较低, 建议检查 BMI 单变量是否能充分表示多维
簇")

        print("尝试 MDLP 方法作为备选...")

        try:
            mdlp_intervals, mdlp_counts, mdlp_reps = mdlp_discretization(
                df['bmi'], labels, min_n=50, max_intervals=len(group_info)+2
            )
            mdlp_purity = evaluate_interval_purity(df, mdlp_intervals, labels)

            print(f'MDLP 方法生成 {len(mdlp_intervals)} 个区间, 平均纯度:
{mdlp_purity['overall_purity']:.3f}')

            if mdlp_purity['overall_purity'] > purity_results['overall_purity'] + 0.05:

```

```

        print("MDLP 方法效果更好, 采用 MDLP 结果")
        intervals, interval_counts, bmi_rep_list = mdlp_intervals,
mdlp_counts, mdlp_reps
        purity_results = mdlp_purity
        stability_results['method_used'] = 'MDLP'
    else:
        print("保持决策树结果")
        stability_results['method_used'] = 'DecisionTree'

    except Exception as mdlp_e:
        print(f"MDLP 方法也失败: {mdlp_e}")
        stability_results['method_used'] = 'DecisionTree'
    else:
        stability_results['method_used'] = 'DecisionTree'

    print("生成决策树分析报告...")
    visualize_tree_analysis(df, intervals, labels, stability_results, purity_results, OU_D)

except Exception as e:
    print(f"决策树方法失败: {e}")
    print("回退到传统方法...")
    intervals, interval_counts, bmi_rep_list = build_bmi_intervals_min_count(df['bmi'],
k=len(group_info), min_n=20)

    X_cols = ['week', 'bmi']
    extra_for_gam = [
        'age', 'height_m', 'weight_kg', 'x_conc', 'total_reads', 'map_ratio', 'dup_ratio',
'unique_reads',
        'gc_content', 'z13', 'z18', 'z21', 'zX', 'zY', 'gc13', 'gc18', 'gc21', 'filtered_ratio',
'gravity', 'parity'
    ]
    for v in extra_for_gam:
        if v in df.columns and not df[v].isna().all():
            X_cols.append(v)

    X = df[X_cols].apply(pd.to_numeric, errors='coerce').astype(float)
    y = df['achieved'].astype(int).values
    mask = np.isfinite(X.values).all(axis=1)

    print(f"用于 GAMM 建模的特征: {X_cols} (共{len(X_cols)}个)")
    print("=" * 60)
    print("GAMM 模型特征详细信息:")

```

```

for i, feat in enumerate(X_cols):
    non_na_count = X[feat].notna().sum()
    print(f' {i+1}. {feat}: {non_na_count} 个有效值")
print("=" * 60)

gam_terms_complex = s(0, n_splines=10) + s(1, n_splines=8) + te(0,1, n_splines=[10,6])
for i in range(2, X.shape[1]):
    gam_terms_complex = gam_terms_complex + s(i, n_splines=8)
gam_complex = LogisticGAM(gam_terms_complex)
gam_complex.fit(X.values[mask], y[mask])

gam_simple = LogisticGAM(s(0, n_splines=10) + s(1, n_splines=8) + te(0,1,
n_splines=[10,6]))
gam_simple.fit(X.values[mask][:, :2], y[mask])

from sklearn.metrics import roc_auc_score
p_complex = np.clip(gam_complex.predict_proba(X.values[mask]), 1e-9, 1-1e-9)
p_simple = np.clip(gam_simple.predict_proba(X.values[mask][:, :2]), 1e-9, 1-1e-9)

auc_complex = float(roc_auc_score(y[mask], p_complex))
auc_simple = float(roc_auc_score(y[mask], p_simple))

print(f'GAMM 复杂模型 AUC: {auc_complex:.4f}')
print(f'GAMM 简化模型 AUC: {auc_simple:.4f}')

if auc_complex > auc_simple:
    gam_best = gam_complex
    gam_best_name = 'GAMM_Multi'
    gam_best_X = X.values[mask]
    gam_best_features = X_cols
    print(f' 选择复杂多变量 GAMM 模型 (AUC 提升 :
+{auc_complex-auc_simple:.4f})")
    print(f'最佳 GAMM 模型使用特征: {gam_best_features}')
else:
    gam_best = gam_simple
    gam_best_name = 'GAMM_Simple'
    gam_best_X = X.values[mask][:, :2]
    gam_best_features = ['week', 'bmi']
    print(f'选择简化 GAMM 模型 (复杂模型无显著提升)")
    print(f'最佳 GAMM 模型使用特征: {gam_best_features}')

rf_clf = RandomForestClassifier(n_estimators=300, random_state=42)

```

```

rf_clf.fit(X.values[mask][:, :2], y[mask])

try:
    from sklearn.metrics import accuracy_score, roc_auc_score, log_loss
    proba_fit = np.clip(rf_clf.predict_proba(X.values[mask][:, :2])[:,1], 1e-9, 1-1e-9)
    acc_07 = float(accuracy_score(y[mask], (proba_fit >= 0.7).astype(int)))
    acc_09 = float(accuracy_score(y[mask], (proba_fit >= 0.9).astype(int)))
    auc = float(roc_auc_score(y[mask], proba_fit))
    brier = float(np.mean((proba_fit - y[mask])**2))
    ll = float(log_loss(y[mask], proba_fit))
    pd.DataFrame([ {
        '样本量': int(mask.sum()),
        'Accuracy@0.7': acc_07,
        'Accuracy@0.9': acc_09,
        'AUC': auc,
        'Brier': brier,
        'LogLoss': ll,
    }]).to_csv(os.path.join(OU_D, '问题二_KMeans_GAMM_拟合指标.csv'),
index=False, encoding='utf-8-sig')
    print(f'RF 拟合: n={int(mask.sum())}, Acc@0.7={acc_07:.3f}, AUC={auc:.3f},
Brier={brier:.4f}, LogLoss={ll:.4f}')
except Exception:
    pass

p0 = 0.95
w_min = 10.0
w_max = 28.0
week_grid = np.arange(w_min, w_max + 0.001, 0.1)
pr_by_group = []
bmi_mid_list = []
t_star_by_group = []
peak_info_by_group = []
for i in range(len(intervals)):
    lo, hi = intervals[i]
    bmi_mid = bmi_rep_list[i]
    bmi_mid_list.append(bmi_mid)
    grid_pts = np.column_stack([week_grid, np.full_like(week_grid, float(bmi_mid))])
    pr = np.clip(rf_clf.predict_proba(grid_pts)[:,1], 1e-9, 1-1e-9)
    pr_by_group.append(pr)
    t_star_by_group.append(find_earliest_crossing(week_grid, pr, p0=p0))
    jpk = int(np.argmax(pr)); peak_info_by_group.append((float(week_grid[jpk]),
float(pr[jpk])))

```

```

penalty_type = 'exp'
w_pen = 0.08
k_exp = 0.25
pow_k = 2.0
if penalty_type == 'linear':
    best_label = f'T_risk_linear*(w={w_pen})'
elif penalty_type == 'poly':
    best_label = f'T_risk_poly*(w={w_pen},p={pow_k})'
else:
    best_label = f'T_risk_exp*(w={w_pen},k={k_exp})'
t_risk_best = [
    find_risk_optimal_time_general(week_grid,    pr_by_group[i],    penalty_type,
w=w_pen, k_exp=k_exp, pow_k=pow_k)
    for i in range(len(intervals))
]

rows = []
plt.figure(figsize=(8,5))
colors = plt.cm.Set2(np.linspace(0,1,len(intervals)))
for i, (lo, hi) in enumerate(intervals):
    pr = pr_by_group[i]
    bmi_mid = bmi_mid_list[i]
    if gam_best_name == 'GAMM_Multi':
        idx_week = X_cols.index('week')
        idx_bmi = X_cols.index('bmi')
        if i < len(intervals) - 1:
            mask_grp = (df['bmi'].astype(float).values >= lo) &
(df['bmi'].astype(float).values < hi)
        else:
            mask_grp = (df['bmi'].astype(float).values >= lo) &
(df['bmi'].astype(float).values <= hi)
        if np.any(mask_grp):
            X_medians = X.loc[mask_grp, :].median(axis=0).values
        else:
            X_medians = X.median(axis=0).values
        grid_full = np.tile(X_medians, (len(week_grid), 1))
        grid_full[:, idx_week] = week_grid
        grid_full[:, idx_bmi] = float(bmi_mid)
        pr_gam_csv = np.clip(gam_best.predict_proba(grid_full), 1e-9, 1-1e-9)
    else:
        grid_pts_gam = np.column_stack([week_grid,    np.full_like(week_grid,

```

```

float(bmi_mid)))]
    pr_gam_csv = np.clip(gam_best.predict_proba(grid_pts_gam), 1e-9, 1-1e-9)

    t_star = find_earliest_crossing(week_grid, pr_gam_csv, p0=p0)
    t_star_p90 = find_earliest_crossing(week_grid, pr_gam_csv, p0=0.93)
    t_star_p97 = find_earliest_crossing(week_grid, pr_gam_csv, p0=0.97)
    t_risk = find_risk_optimal_time_general(week_grid, pr_gam_csv, penalty_type,
w=w_pen, k_exp=k_exp, pow_k=pow_k)
    jpk2 = int(np.argmax(pr_gam_csv)); t_peak = float(week_grid[jpk2]); p_max =
float(pr_gam_csv[jpk2])
    rows.append({
        'BMI 区间': f[{lo}, {hi}],
        'BMI 代表值': round(bmi_mid,2),
        '样本数': int(interval_counts[i]),
        'T*(p0={p0:.2f})': None if t_star is None else round(t_star,2),
        'T*(p0=0.93)': None if t_star_p90 is None else round(t_star_p90, 2),
        'T*(p0=0.97)': None if t_star_p97 is None else round(t_star_p97, 2),
        'best_label': round(t_risk, 2),
        '峰值概率(%)': round(p_max*100.0,1),
        '峰值孕周': round(t_peak,2)
    })
    plt.plot(week_grid, pr, linewidth=2, color=colors[i], label=f'[{lo},{hi}]')
    if i == 0:
        plt.axhline(p0, color='#b22222', linestyle='--', linewidth=1.2,
label=f'p0={p0:.2f}')
        if t_star is not None:
            plt.scatter([t_star],[p0], color=colors[i], edgecolors='k', zorder=3)
            plt.scatter([t_risk],[np.interp(t_risk, week_grid, pr)], marker='D', color=colors[i],
edgecolors='k', zorder=3)
            plt.scatter([t_peak],[p_max], marker='^', color=colors[i], edgecolors='k', zorder=3)

    plt.xlabel('孕周(周)')
    plt.ylabel('Pr(Y ≥ 4%)')
    plt.xlim(w_min, w_max)
    plt.ylim(0,1)
    plt.grid(True, alpha=0.3)
    plt.legend(title='KMeans BMI 组')
    plt.tight_layout()
    plt.savefig(os.path.join(OU_D, '问题二_KMeans_RF_概率曲线.png'), dpi=200)
    plt.close()

pd.DataFrame(rows).to_csv(os.path.join(OU_D, '问题二_KMeans_GAMM_推荐时

```

```
点.csv'), index=False, encoding='utf-8-sig')
```

```
plt.figure(figsize=(10,6))
colors = plt.cm.Set2(np.linspace(0,1,len(intervals)))
for i, (lo, hi) in enumerate(intervals):
    pr = pr_by_group[i]
    plt.plot(week_grid, pr, linewidth=3, color=colors[i], label=f'[{lo},{hi}]')
    if i == 0:
        plt.axhline(p0, color='#b22222', linestyle='--', linewidth=1.5,
label=f'p0={p0:.2f}')
        # 标注 RF 预测的特征点
        t_star_rf = t_star_by_group[i]
        t_peak_rf, p_max_rf = peak_info_by_group[i]
        t_risk_rf = t_risk_best[i]

        if t_star_rf is not None:
            plt.scatter([t_star_rf],[p0], color=colors[i], edgecolors='k', zorder=3, s=80)
            plt.scatter([t_risk_rf],[np.interp(t_risk_rf, week_grid, pr)], marker='D',
color=colors[i], edgecolors='k', zorder=3, s=80)
            plt.scatter([t_peak_rf],[p_max_rf], marker='^', color=colors[i], edgecolors='k',
zorder=3, s=80)

plt.xlabel('孕周(周)', fontsize=16)
plt.ylabel('RF: Pr(Y ≥ 4%)', fontsize=16)
plt.xlim(w_min, w_max)
plt.ylim(0,1)
plt.grid(True, alpha=0.3)
plt.legend(title='BMI 组', fontsize=14, title_fontsize=15)
plt.tick_params(axis='both', which='major', labelsize=12)
plt.tight_layout()
plt.savefig(os.path.join(OU_D, '问题三_RF_概率曲线.png'), dpi=200)
plt.close()

try:
    fig = plt.figure(figsize=(7,5))
    ax = fig.add_subplot(111, projection='3d')
    xs = df['bmi'].astype(float).values
    ys = df['week'].astype(float).values
    zs = df['y_conc'].astype(float).values
    col = plt.cm.Set2((labels - labels.min()) / (labels.max() - labels.min() + 1e-12))
    ax.scatter(xs, ys, zs, c=col, s=12, alpha=0.8)
    ax.set_xlabel('BMI')
```

```

ax.set_ylabel('孕周(周)')
ax.set_zlabel('Y 浓度')
try:
    P = np.column_stack([xs, ys, zs])
    mu = P.mean(axis=0)
    C = np.cov(P.T)
    vals, vecs = eig(C)
    u = np.linspace(0, 2*np.pi, 32)
    v = np.linspace(0, np.pi, 16)
    uu, vv = np.meshgrid(u, v)
    x = np.cos(uu) * np.sin(vv)
    y = np.sin(uu) * np.sin(vv)
    z = np.cos(vv)
    n_std = 2.0
    D = np.diag(np.sqrt(np.maximum(vals, 1e-12)) * n_std)
    R = vecs
    XYZ = (R @ D @ np.vstack([x.ravel(), y.ravel(), z.ravel()])).reshape(3,
*x.shape)
    Xe, Ye, Ze = XYZ[0] + mu[0], XYZ[1] + mu[1], XYZ[2] + mu[2]
    ax.plot_wireframe(Xe, Ye, Ze, color='k', linewidth=0.6, rstride=2, cstride=2,
alpha=0.25, linestyle='--')
except Exception:
    pass
plt.tight_layout()
plt.savefig(os.path.join(OU_D, '问题二_3D_BMI 分组散点.png'), dpi=220)
plt.close()

X_reg = np.column_stack([df['week'].astype(float).values,
df['bmi'].astype(float).values])
y_reg = df['y_conc'].astype(float).values
rf_reg = RandomForestRegressor(n_estimators=400, random_state=42)
rf_reg.fit(X_reg, y_reg)
w_lin = np.linspace(float(df['week'].min()), float(df['week'].max()), 50)
b_lin = np.linspace(float(df['bmi'].min()), float(df['bmi'].max()), 50)
Wg, Bg = np.meshgrid(w_lin, b_lin)
Z = rf_reg.predict(np.column_stack([Wg.ravel(), Bg.ravel()])).reshape(Wg.shape)
fig = plt.figure(figsize=(7,5))
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_surface(Bg, Wg, Z, cmap='viridis', linewidth=0, antialiased=True,
alpha=0.9)
ax.scatter(df['bmi'].astype(float).values, df['week'].astype(float).values,
df['y_conc'].astype(float).values,

```

```

        c='k', s=8, alpha=0.35)
    ax.set_xlabel('BMI')
    ax.set_ylabel('孕周(周)')
    ax.set_zlabel('Y 浓度(RF 拟合)')
    plt.tight_layout()
    plt.savefig(os.path.join(OU_D, '问题二_RF_3D 拟合曲面.png'), dpi=220)
    plt.close()
except Exception:
    pass

try:
    from pygam import LinearGAM, s, te
    y_reg = df['y_conc'].astype(float).values
    X_full = X.values
    mask_reg = np.isfinite(X_full).all(axis=1) & np.isfinite(y_reg)

    gam_terms_reg = s(0, n_splines=10) + s(1, n_splines=8) + te(0,1, n_splines=[10,6])
    for i in range(2, X_full.shape[1]):
        gam_terms_reg += s(i, n_splines=6)

    gam_reg = LinearGAM(gam_terms_reg)
    gam_reg.fit(X_full[mask_reg], y_reg[mask_reg])

    w_lin = np.linspace(float(df['week'].min()), float(df['week'].max()), 50)
    b_lin = np.linspace(float(df['bmi'].min()), float(df['bmi'].max()), 50)
    Wg, Bg = np.meshgrid(w_lin, b_lin)

    X_medians = X.median(axis=0).values
    idx_week = X_cols.index('week')
    idx_bmi = X_cols.index('bmi')
    grid_full = np.tile(X_medians, (Wg.size, 1))
    grid_full[:, idx_week] = Wg.ravel()
    grid_full[:, idx_bmi] = Bg.ravel()

    Z = gam_reg.predict(grid_full).reshape(Wg.shape)

    fig = plt.figure(figsize=(7,5))
    ax = fig.add_subplot(111, projection='3d')
    ax.plot_surface(Bg, Wg, Z, cmap='viridis', linewidth=0, antialiased=True,
alpha=0.9)
    try:
        xs = df['bmi'].astype(float).values

```

```

        ys = df['week'].astype(float).values
        zs = df['y_conc'].astype(float).values
        ax.scatter(xs, ys, zs, c='k', s=8, alpha=0.35)
    except Exception:
        pass
    ax.set_xlabel('BMI')
    ax.set_ylabel('孕周(周)')
    ax.set_zlabel('Y 浓度(GAM 拟合)')
    plt.tight_layout()
    plt.savefig(os.path.join(OU_D, '问题二_GAMM_3D 拟合曲面.png'), dpi=220)
    plt.close()
except Exception:
    pass

try:
    plt.figure(figsize=(7.5,4.8))
    colors = plt.cm.Set2(np.linspace(0,1,len(intervals)))
    bmis_all = df['bmi'].astype(float).values
    y_all = df['y_conc'].astype(float).values
    for i, (lo, hi) in enumerate(intervals):
        if i < len(intervals) - 1:
            m = (bmis_all >= lo) & (bmis_all < hi)
        else:
            m = (bmis_all >= lo) & (bmis_all <= hi)
        plt.scatter(bmis_all[m], y_all[m], s=14, alpha=0.65, color=colors[i],
label=f'[{lo}, {hi}]' if i < len(intervals)-1 else f'[{lo}, {hi}]')
    for i, (_, hi) in enumerate(intervals[:-1]):
        plt.axvline(hi, color='grey', linestyle='--', linewidth=1.0, alpha=0.8)
    plt.xlabel('BMI')
    plt.ylabel('Y 浓度')
    plt.grid(True, alpha=0.3)
    plt.legend(title='BMI 区间')
    plt.tight_layout()
    plt.savefig(os.path.join(OU_D, '问题二_BMI 区间散点_Y 浓度.png'), dpi=220)
    plt.close()
except Exception:
    pass

try:
    X_cls = np.column_stack([df['week'].astype(float).values,
df['bmi'].astype(float).values])
    y_true = df['achieved'].astype(int).values

```

```

groups_all = df['patient_id'].astype(str).values if 'patient_id' in df.columns else
np.arange(len(y_true)).astype(str)

```

```

def _build_poly_features(X2: np.ndarray) -> np.ndarray:

```

```

    w = X2[:, 0]; b = X2[:, 1]

```

```

    return np.column_stack([w, b, w*w, b*b, w*b])

```

```

def _pseudo_r2(y: np.ndarray, p: np.ndarray) -> float:

```

```

    p = np.clip(p, 1e-12, 1-1e-12)

```

```

    ll_model = float(np.sum(y*np.log(p) + (1-y)*np.log(1-p)))

```

```

    p0 = float(np.mean(y))

```

```

    ll_null = float(np.sum(y*np.log(p0+1e-12) + (1-y)*np.log(1-p0+1e-12)))

```

```

    return float(1.0 - ll_model/ll_null) if ll_null != 0 else np.nan

```

```

model_probs = {}

```

```

timings = {}

```

```

t0 = time.perf_counter()

```

```

if gam_best_name == 'GAMM_Multi':

```

```

    p_gamm = np.clip(gam_best.predict_proba(X.values), 1e-9, 1-1e-9)

```

```

else:

```

```

    p_gamm = np.clip(gam_best.predict_proba(X_cls), 1e-9, 1-1e-9)

```

```

timings[gam_best_name] = time.perf_counter() - t0

```

```

model_probs[gam_best_name] = p_gamm

```

```

t0 = time.perf_counter()

```

```

X_poly = _build_poly_features(X_cls)

```

```

lr = LogisticRegression(max_iter=2000).fit(X_poly, y_true)

```

```

p_lr = np.clip(lr.predict_proba(X_poly)[:,1], 1e-9, 1-1e-9)

```

```

timings['Logit'] = time.perf_counter() - t0

```

```

model_probs['Logit'] = p_lr

```

```

import statsmodels.api as sm

```

```

from statsmodels.genmod.cov_struct import Exchangeable

```

```

t0 = time.perf_counter()

```

```

glm_res = sm.GLM(y_true, sm.add_constant(X_poly),
family=sm.families.Binomial()).fit()

```

```

p_glm = np.clip(glm_res.predict(sm.add_constant(X_poly)), 1e-9, 1-1e-9)

```

```

timings['GLM'] = time.perf_counter() - t0

```

```

model_probs['GLM'] = p_glm

```

```

t0 = time.perf_counter()

```

```

gb = GradientBoostingClassifier(random_state=42)
gb.fit(X_cls, y_true)
p_gb = np.clip(gb.predict_proba(X_cls)[:,:1], 1e-9, 1-1e-9)
timings['GB'] = time.perf_counter() - t0
model_probs['GB'] = p_gb

t0 = time.perf_counter()
rf = RandomForestClassifier(n_estimators=300, random_state=42)
rf.fit(X_cls, y_true)
p_rf = np.clip(rf.predict_proba(X_cls)[:,:1], 1e-9, 1-1e-9)
timings['RF'] = time.perf_counter() - t0
model_probs['RF'] = p_rf

try:
    t0 = time.perf_counter()
    gee = sm.GEE(y_true, sm.add_constant(X_poly), groups=groups_all,
family=sm.families.Binomial(), cov_struct=Exchangeable())
    gee_res = gee.fit()
    p_gee = np.clip(gee_res.predict(sm.add_constant(X_poly)), 1e-9, 1-1e-9)
    timings['GEE'] = time.perf_counter() - t0
    model_probs['GEE'] = p_gee
except Exception:
    pass

plt.figure(figsize=(7,5))
for name, p in model_probs.items():
    fpr, tpr, _ = metrics.roc_curve(y_true, p)
    auc_val = metrics.auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{name} (AUC={auc_val:.3f})')
plt.plot([0,1],[0,1], 'k--', alpha=0.5)
plt.xlabel('1-特异度')
plt.ylabel('灵敏度')
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(OU_D, '问题二_模型 ROC 曲线对比.png'), dpi=220)
plt.close()

rows_m = []
for name, p in model_probs.items():
    auc_val = float(metrics.roc_auc_score(y_true, p))
    acc = float(metrics.accuracy_score(y_true, (p>=0.7).astype(int)))

```

```

        ll = float(metrics.log_loss(y_true, p))
        mae = float(np.mean(np.abs(p - y_true)))
        rmse = float(np.sqrt(np.mean((p - y_true)**2)))
        r2 = _pseudo_r2(y_true, p)
        rows_m.append({
            '模型': name,
            'MAE': mae,
            'RMSE': rmse,
            'AUC': auc_val,
            'Accuracy@0.7': acc,
            'LogLoss': ll,
            'R2': r2,
            '运行时间_秒': float(timings.get(name, 0.0)),
        })
    pd.DataFrame(rows_m).to_csv(os.path.join(OU_D, '问题二_模型对比.csv'),
index=False, encoding='utf-8-sig')
except Exception:
    pass

try:
    X_cls = np.column_stack([df['week'].astype(float).values,
df['bmi'].astype(float).values])
    y_true = df['achieved'].astype(int).values
    groups = df['patient_id'].astype(str).values if 'patient_id' in df.columns else
np.arange(len(y_true))

    gkf = GroupKFold(n_splits=min(5, len(np.unique(groups))))
    n = len(y_true)
    oof = {
        gam_best_name: np.zeros(n, dtype=float),
        'GEE': np.zeros(n, dtype=float),
        'Logit': np.zeros(n, dtype=float),
        'GLM': np.zeros(n, dtype=float),
        'GB': np.zeros(n, dtype=float),
        'RF': np.zeros(n, dtype=float),
    }

    for tr, te in gkf.split(X_cls, y_true, groups):
        try:
            if gam_best_name == 'GAMM_Multi':
                Xtr_full, ytr = X.values[tr], y_true[tr]
                gam_fold = LogisticGAM(gam_terms_complex)

```

```

        gam_fold.fit(Xtr_full, ytr)
        oof[gam_best_name][te] =
np.clip(gam_fold.predict_proba(X.values[te]), 1e-9, 1-1e-9)
    else:
        Xtr, ytr = X_cls[tr], y_true[tr]
        gam_fold = LogisticGAM(s(0, n_splines=10) + s(1, n_splines=8) +
te(0,1, n_splines=[10,6]))
        gam_fold.fit(Xtr, ytr)
        oof[gam_best_name][te] =
np.clip(gam_fold.predict_proba(X_cls[te]), 1e-9, 1-1e-9)
    except Exception:
        oof[gam_best_name][te] = 0.5

    try:
        import statsmodels.api as sm
        from statsmodels.genmod.cov_struct import Exchangeable
        Xtr, ytr = X_cls[tr], y_true[tr]
        Xte = X_cls[te]
        Xtr_poly = _build_poly_features(Xtr)
        Xte_poly = _build_poly_features(Xte)
        gee = sm.GEE(ytr, sm.add_constant(Xtr_poly), groups=groups[tr],
family=sm.families.Binomial(), cov_struct=Exchangeable())
        gee_res = gee.fit()
        oof['GEE'][te] = np.clip(gee_res.predict(sm.add_constant(Xte_poly)),
1e-9, 1-1e-9)
    except Exception:
        oof['GEE'][te] = 0.5

    try:
        Xtr, ytr = X_cls[tr], y_true[tr]
        if SMOTE is not None:
            try:
                Xtr, ytr = SMOTE(sampling_strategy=0.5, k_neighbors=3,
random_state=42).fit_resample(Xtr, ytr)
            except Exception:
                pass
            Xtr_poly = _build_poly_features(Xtr)
            Xte_poly = _build_poly_features(X_cls[te])
            lr = LogisticRegression(max_iter=2000).fit(Xtr_poly, ytr)
            oof['Logit'][te] = np.clip(lr.predict_proba(Xte_poly)[:,-1], 1e-9, 1-1e-9)
    except Exception:
        oof['Logit'][te] = 0.5

```

```

try:
    import statsmodels.api as sm
    Xtr, ytr = X_cls[tr], y_true[tr]
    # GLM 用 SMOTE 意义有限，这里不加
    Xtr_poly = _build_poly_features(Xtr)
    Xte_poly = _build_poly_features(X_cls[te])
    glm_res = sm.GLM(ytr, sm.add_constant(Xtr_poly),
family=sm.families.Binomial()).fit()
    oof['GLM'][te] = np.clip(glm_res.predict(sm.add_constant(Xte_poly)),
1e-9, 1-1e-9)
except Exception:
    oof['GLM'][te] = 0.5

try:
    Xtr, ytr = X_cls[tr], y_true[tr]
    if SMOTE is not None:
        try:
            Xtr, ytr = SMOTE(sampling_strategy=0.5, k_neighbors=3,
random_state=42).fit_resample(Xtr, ytr)
        except Exception:
            pass
        gb = GradientBoostingClassifier(random_state=42).fit(Xtr, ytr)
        oof['GB'][te] = np.clip(gb.predict_proba(X_cls[te])[:,1], 1e-9, 1-1e-9)
    except Exception:
        oof['GB'][te] = 0.5

try:
    Xtr, ytr = X_cls[tr], y_true[tr]
    if SMOTE is not None:
        try:
            Xtr, ytr = SMOTE(sampling_strategy=0.5, k_neighbors=3,
random_state=42).fit_resample(Xtr, ytr)
        except Exception:
            pass
        rf = RandomForestClassifier(n_estimators=300, random_state=42).fit(Xtr,
ytr)
        oof['RF'][te] = np.clip(rf.predict_proba(X_cls[te])[:,1], 1e-9, 1-1e-9)
    except Exception:
        oof['RF'][te] = 0.5

plt.figure(figsize=(7,5))

```

```

for name, p in oof.items():
    fpr, tpr, _ = metrics.roc_curve(y_true, p)
    auc_val = metrics.auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{name} (AUC={auc_val:.3f})')
plt.plot([0,1],[0,1], 'k--', alpha=0.5)
plt.xlabel('1-特异度')
plt.ylabel('灵敏度')
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(OU_D, '问题二_模型 ROC 曲线对比_OOF.png'), dpi=220)
plt.close()
except Exception:
    pass

try:
    plt.figure(figsize=(8,5))
    colors = plt.cm.Set2(np.linspace(0,1,len(intervals)))
    for i, (lo, hi) in enumerate(intervals):
        bmi_mid = bmi_mid_list[i]

        if gam_best_name == 'GAMM_Multi':
            idx_week = X_cols.index('week')
            idx_bmi = X_cols.index('bmi')
            if i < len(intervals) - 1:
                mask_grp = (df['bmi'].astype(float).values >= lo) &
(df['bmi'].astype(float).values < hi)
            else:
                mask_grp = (df['bmi'].astype(float).values >= lo) &
(df['bmi'].astype(float).values <= hi)
            if np.any(mask_grp):
                X_medians = X.loc[mask_grp, :].median(axis=0).values
            else:
                X_medians = X.median(axis=0).values
            grid_full = np.tile(X_medians, (len(week_grid), 1))
            grid_full[:, idx_week] = week_grid
            grid_full[:, idx_bmi] = float(bmi_mid)
            pr_gam = np.clip(gam_best.predict_proba(grid_full), 1e-9, 1-1e-9)
        else:
            grid_pts = np.column_stack([week_grid, np.full_like(week_grid,
float(bmi_mid))])
            pr_gam = np.clip(gam_best.predict_proba(grid_pts), 1e-9, 1-1e-9)

```

```

plt.plot(week_grid, pr_gam, linewidth=2, color=colors[i], label=f'[{lo},{hi}]')
t_star_g = find_earliest_crossing(week_grid, pr_gam, p0=p0)
if t_star_g is not None:
    plt.scatter([t_star_g],[p0], color=colors[i], edgecolors='k', zorder=3)

plt.axhline(p0, color='#b22222', linestyle='--', linewidth=1.2, label=f'p0={p0:.2f}')
plt.xlabel('孕周(周)')
plt.ylabel('Pr(Y ≥ 4%)')
plt.xlim(w_min, w_max)
plt.ylim(0,1)
plt.grid(True, alpha=0.3)
plt.legend(title=f'KMeans BMI 组 ({gam_best_name})')
plt.tight_layout()
plt.savefig(os.path.join(OU_D, f'问题二_KMeans_{gam_best_name}_概率曲线.png'), dpi=200)
plt.close()
except Exception:
    pass

try:
    p_hat = np.clip(rf_clf.predict_proba(np.column_stack([df['week'].astype(float).values, df['bmi'].astype(float).values]))[:,1], 1e-9, 1-1e-9)
    y_true = df['achieved'].astype(int).values
    y_pred = (p_hat >= 0.90).astype(int)
    from sklearn.metrics import confusion_matrix
    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(4.2,3.6))
    im = ax.imshow(cm, cmap='Blues')
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, int(cm[i,j]), ha='center', va='center', color='black')
    ax.set_xlabel('预测')
    ax.set_ylabel('真实')
    ax.set_xticks([0,1]); ax.set_xticklabels(['阴性','阳性'])
    ax.set_yticks([0,1]); ax.set_yticklabels(['阴性','阳性'])
    plt.tight_layout()
    plt.savefig(os.path.join(OU_D, '问题二_RF_混淆矩阵.png'), dpi=220)
    plt.close()
except Exception:
    pass

```

```

try:
    if gam_best_name == 'GAMM_Multi':
        p_gamm = np.clip(gam_best.predict_proba(X.values), 1e-9, 1-1e-9)
    else:
        X_simple = np.column_stack([df['week'].astype(float).values,
df['bmi'].astype(float).values])
        p_gamm = np.clip(gam_best.predict_proba(X_simple), 1e-9, 1-1e-9)

    y_true = df['achieved'].astype(int).values
    y_pred_gamm = (p_gamm >= 0.90).astype(int)

    from sklearn.metrics import confusion_matrix
    cm_gamm = confusion_matrix(y_true, y_pred_gamm)

    fig, ax = plt.subplots(figsize=(4.2,3.6))
    im = ax.imshow(cm_gamm, interpolation='nearest', cmap='Blues')
    plt.colorbar(im, fraction=0.046, pad=0.04)

    thresh = cm_gamm.max() / 2.0 if cm_gamm.size > 0 else 0.5
    for i in range(cm_gamm.shape[0]):
        for j in range(cm_gamm.shape[1]):
            ax.text(j, i, format(cm_gamm[i, j], 'd'),
                    ha='center', va='center',
                    color='white' if cm_gamm[i, j] > thresh else 'black')

    ax.set_title(f'{gam_best_name} 混淆矩阵')
    ax.set_xlabel('预测值')
    ax.set_ylabel('实际值')
    ax.set_xticks([0,1]); ax.set_xticklabels(['未达标','达标'])
    ax.set_yticks([0,1]); ax.set_yticklabels(['未达标','达标'])
    plt.tight_layout()
    plt.savefig(os.path.join(OU_D, f'问题二_{gam_best_name}_混淆矩阵.png'),
dpi=220)
    plt.close()

    print(f'已生成 {gam_best_name} 混淆矩阵图")
except Exception as e:
    print(f'生成 GAMM 混淆矩阵失败: {e}")
    pass

print("\n=== BMI 区间生成结果摘要 ===")
print(f'聚类变量数: {len(cluster_vars)}")

```

```
print(f'最终 BMI 区间数: {len(intervals)}')
for i, ((lo, hi), count) in enumerate(zip(intervals, interval_counts)):
    print(f'  区间 {i+1}: [{lo:.3f}, {hi:.3f}), 样本数={count}')
```

```
print('\n 完成: 肘部图、RF/GAMM 概率曲线、推荐时点、3D 散点与 RF 曲面、模型 ROC 与指标表、RF 混淆矩阵、GAMM 混淆矩阵已输出。')
```

```
if __name__ == '__main__':
    main()
```

问题四代码

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib as mpl
mpl.use('Agg') # 使用非交互式后端
import seaborn as sns
from scipy import stats
from scipy.interpolate import BSpline
import warnings
warnings.filterwarnings('ignore')

mpl.rcParams["font.family"] = ["SimSun"]
plt.rcParams['axes.unicode_minus'] = False

RUN_BAYES = False
from sklearn.model_selection import GroupKFold
from sklearn.metrics import roc_auc_score, brier_score_loss, log_loss, roc_curve,
accuracy_score, confusion_matrix, precision_recall_fscore_support
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.cluster import KMeans
```

```

HAS_IMBLEARN = False
try:
    from imblearn.over_sampling import SMOTE
    from imblearn.pipeline import Pipeline as ImbPipeline
    HAS_IMBLEARN = True
except Exception:
    HAS_IMBLEARN = False

HAS_XGB = False
try:
    from xgboost import XGBClassifier
    HAS_XGB = True
except Exception:
    HAS_XGB = False

HAS_PYGAM = False
try:
    from pygam import LogisticGAM, s, l
    HAS_PYGAM = True
except Exception:
    HAS_PYGAM = False

HAS_SM_GLMM = False
try:
    from statsmodels.genmod.bayes_mixed_glm import BinomialBayesMixedGLM
    import statsmodels.api as sm
    HAS_SM_GLMM = True
except Exception:
    HAS_SM_GLMM = False

try:
    import pymc as pm
    import arviz as az
    import pytensor.tensor as pt
    HAS_PYMC = True
except ImportError:
    print("警告: PyMC 未安装, 将使用简化的频率派方法")
    HAS_PYMC = False

os.makedirs("输出 4", exist_ok=True)

def ld_prep_1(excel_path=r"C:\Users\27832\Desktop\附件_修正后.xlsx"):

```

```

print("正在加载数据...")

df = pd.read_excel(excel_path, sheet_name=1)
print(f'原始数据形状: {df.shape}')

col_mapping = {
    '序号': 'sample_id',
    '孕妇代码': 'patient_id',
    '年龄': 'age',
    '身高': 'height',
    '体重': 'weight',
    '末次月经': 'last_menstrual',
    'IVF 妊娠': 'ivf',
    '检测日期': 'test_time',
    '检测抽血次数': 'blood_count',
    '检测孕周': 'week_str',
    '孕妇 BMI': 'bmi',
    '原始读段数': 'total_reads',
    '在参考基因组上比对的比例': 'mapped_prop',
    '重复读段的比例': 'dup_prop',
    '唯一比对的读段数': 'unique_reads',
    'GC 含量': 'gc_content',
    '13 号染色体的 Z 值': 'z13',
    '18 号染色体的 Z 值': 'z18',
    '21 号染色体的 Z 值': 'z21',
    'X 染色体的 Z 值': 'zx',
    'X 染色体浓度': 'x_conc',
    '13 号染色体的 GC 含量': 'gc13',
    '18 号染色体的 GC 含量': 'gc18',
    '21 号染色体的 GC 含量': 'gc21',
    '被过滤掉读段数的比例': 'filtered_prop',
    '染色体的非整倍体': 'trisomy_label',
    '怀孕次数': 'pregnancy_count',
    '生产次数': 'birth_count',
    '胎儿是否健康': 'healthy'
}

df = df.rename(columns=col_mapping)

def _parse_wk_2(week_str):
    if pd.isna(week_str):
        return np.nan

```

```

    try:
        week_str = str(week_str).strip()
        if 'w' in week_str:
            parts = week_str.replace('w', '+').split('+')
            week = float(parts[0])
            if len(parts) > 1 and parts[1]:
                day = float(parts[1])
                week += day / 7.0
            return week
        else:
            return float(week_str)
    except:
        return np.nan

df['week'] = df['week_str'].apply(_parse_wk_2)

import re
def _parse_cnt_3(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    m = re.search(r"\d+", s)
    if not m:
        return np.nan
    val = int(m.group(0))
    if val >= 3:
        return 3
    return float(val)

for cnt_col in ['pregnancy_count', 'birth_count', 'blood_count']:
    if cnt_col in df.columns:
        df[cnt_col] = df[cnt_col].apply(_parse_cnt_3)
        vc = df[cnt_col].value_counts(dropna=False).sort_index()
        print(f'{cnt_col} -> dtype={df[cnt_col].dtype}; 取值分布:\n{vc}')

print("进行数据清洗...")

key_vars = ['week', 'bmi', 'z13', 'z18', 'z21', 'total_reads', 'mapped_prop']
n_before = len(df)
df = df.dropna(subset=key_vars)
n_after = len(df)
print(f'去除关键变量缺失后: {n_before} -> {n_after} 条记录')

```

```

df = df[(df['week'] >= 10) & (df['week'] <= 25)]

df = df[(df['bmi'] >= 15) & (df['bmi'] <= 50)]

df = df[df['total_reads'] >= 1e6]

df = df[df['mapped_prop'] >= 0.5]

print(f'质控后数据形状: {df.shape}')

df['test_date'] = pd.to_datetime(df['test_time'], errors='coerce')
df['batch'] = pd.Categorical(df['test_date'].dt.date).codes

continuous_vars = ['week', 'bmi', 'age', 'total_reads', 'mapped_prop', 'gc_content']
for var in continuous_vars:
    if var in df.columns:
        df[f'{var}_c'] = (df[var] - df[var].mean()) / df[var].std()

df['has_trisomy'] = (~df['trisomy_label'].isna()).astype(int)

df['ivf_binary'] = (df['ivf'] == 1).astype(int)

print("数据预处理完成！ ")
print(f'最终数据: {len(df)} 条记录, {len(df['patient_id'].unique())} 位孕妇')
print(f'trisomy 阳性: {df['has_trisomy'].sum()} 例 ({df['has_trisomy'].mean():.3%})')

return df

def exp_anl_4(df):
    print("\n=== 探索性数据分析 ===")

    print("\n 基本统计信息:")
    numeric_cols = ['week', 'bmi', 'age', 'total_reads', 'mapped_prop', 'z13', 'z18', 'z21']
    print(df[numeric_cols].describe())

    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    for i, (chr_name, ax) in enumerate(zip(['z13', 'z18', 'z21'], axes.flat[:3])):
        df[chr_name].hist(bins=50, alpha=0.7, ax=ax)
        ax.axvline(3, color='red', linestyle='--', label='阈值=3')
        ax.axvline(-3, color='red', linestyle='--')

```

```

ax.set_xlabel(f'{chr_name.upper()} Z 值')
ax.set_ylabel('频数')
ax.legend()
ax.grid(True, alpha=0.3)

trisomy_counts = df['has_trisomy'].value_counts()
axes[1,1].pie(trisomy_counts.values, labels=['正常', 'Trisomy'], autopct='%1.1f%%')
axes[1,1].set_title('Trisomy 分布')

plt.tight_layout()
plt.savefig('输出 4/问题四_探索性分析.png', dpi=300, bbox_inches='tight')
plt.close()

covariates = [
    'age', 'height', 'weight',
    'week', 'bmi', 'total_reads',
    'gc_content', 'x_conc', 'gc13'
]
labels_cn = {
    'age': '年龄',
    'height': '身高',
    'weight': '体重',
    'week': '检测孕周',
    'bmi': '孕妇 BMI',
    'total_reads': '原始读段数',
    'gc_content': 'GC 含量',
    'x_conc': 'X 染色体浓度',
    'gc13': '13 号染色体的 GC 含量',
}

with mpl.rc_context({'font.family': 'SimHei', 'axes.labelsize': 16, 'xtick.labelsize': 14,
'ytick.labelsize': 14}):
    fig, axes = plt.subplots(3, 3, figsize=(15, 15))
    for i, cov in enumerate(covariates):
        ax = axes.flat[i]
        ax.scatter(df[cov], df['z13'], alpha=0.5, s=18)
        ax.set_xlabel(labels_cn.get(cov, cov), fontsize=16)
        ax.set_ylabel('Z13 Z 值', fontsize=16)
        ax.axhline(3, color='red', linestyle='--', alpha=0.7)
        ax.axhline(-3, color='red', linestyle='--', alpha=0.7)
        if cov in ['total_reads']:
            try:
                ax.set_xscale('log')

```

```

        except Exception:
            pass
        ax.tick_params(axis='both', labelsiz=14)
        ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('输出 4/问题四_Z 值与协变量关系.png', dpi=300, bbox_inches='tight')
    plt.close()

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

patient_stats = df.groupby('patient_id').agg({
    'total_reads': 'mean',
    'z13': 'std',
    'z18': 'std',
    'z21': 'std'
}).reset_index()

z_std_mean = patient_stats[['z13', 'z18', 'z21']].mean(axis=1)
axes[0].scatter(patient_stats['total_reads'], z_std_mean, alpha=0.6)
axes[0].set_xlabel('平均读段数')
axes[0].set_ylabel('Z 值标准差（患者内）')
axes[0].set_xscale('log')
axes[0].grid(True, alpha=0.3)

axes[1].scatter(df['mapped_prop'], df['z21'], alpha=0.3, s=10, label='Z21')
axes[1].set_xlabel('比对率')
axes[1].set_ylabel('Z21 值')
axes[1].axhline(3, color='red', linestyle='--', alpha=0.7)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('输出 4/问题四_测序质量分析.png', dpi=300, bbox_inches='tight')
plt.close()

return df

def elbow_km_5(features: np.ndarray, k_min: int = 2, k_max: int = 8):
    inertias = []
    ks = list(range(k_min, k_max + 1))
    for k in ks:
        km = KMeans(n_clusters=k, n_init=20, random_state=42)

```

```

        km.fit(features)
        inertias.append(float(km.inertia_))
    x1, y1 = ks[0], inertias[0]
    x2, y2 = ks[-1], inertias[-1]
    best_k = ks[0]
    best_d = -1.0
    for k, y in zip(ks, inertias):
        num = abs((y2 - y1) * k - (x2 - x1) * y + x2*y1 - y2*x1)
        den = np.sqrt((y2 - y1)**2 + (x2 - x1)**2)
        d = num / (den + 1e-12)
        if d > best_d:
            best_d = d
            best_k = k
    return best_k, ks, inertias

def build_bmi_it_tree_6(
    df: pd.DataFrame,
    labels: np.ndarray,
    min_n: int = 30,
    max_leaf_nodes: int = 6,
    round_digits: int = 3,
):
    bmi_all = pd.to_numeric(df['bmi'], errors='coerce').astype(float).values
    y = np.asarray(labels, dtype=int)
    mask = np.isfinite(bmi_all) & np.isfinite(y)
    X = bmi_all[mask].reshape(-1, 1)
    y = y[mask]
    if X.size == 0:
        raise RuntimeError('BMI 为空，无法训练决策树')
    tree = DecisionTreeClassifier(
        criterion='entropy',
        random_state=42,
        max_leaf_nodes=max_leaf_nodes,
        min_samples_leaf=int(min_n),
    )
    tree.fit(X, y)
    thresholds = tree.tree_.threshold
    feats = tree.tree_.feature
    cuts = [float(t) for t, f in zip(thresholds, feats) if f == 0 and np.isfinite(t)]
    cuts = sorted({round(c, round_digits) for c in cuts})
    global_lo = float(np.nanmin(bmi_all))

```

```

global_hi = float(np.nanmax(bmi_all))
bounds = [global_lo] + cuts + [global_hi]
clean_bounds = [bounds[0]]
for v in bounds[1:]:
    if v <= clean_bounds[-1]:
        v = clean_bounds[-1] + 10**(-round_digits)
    clean_bounds.append(v)
intervals, counts, reps = [], [], []
for i in range(len(clean_bounds) - 1):
    lo = round(float(clean_bounds[i]), round_digits)
    hi = round(float(clean_bounds[i+1]), round_digits)
    if i < len(clean_bounds) - 2:
        m = (bmi_all >= lo) & (bmi_all < hi)
    else:
        m = (bmi_all >= lo) & (bmi_all <= hi)
    n_i = int(np.sum(m))
    if n_i == 0:
        continue
    intervals.append((lo, hi))
    counts.append(n_i)
    reps.append(float(np.median(bmi_all[m])))
if any(c < int(min_n) for c in counts) and len(intervals) > 1:
    while True:
        if len(intervals) <= 1:
            break
        idx_small = int(np.argmin(counts))
        if counts[idx_small] >= int(min_n):
            break
        if idx_small == 0:
            j = 1
        elif idx_small == len(intervals) - 1:
            j = len(intervals) - 2
        else:
            d_left = abs(reps[idx_small] - reps[idx_small-1])
            d_right = abs(reps[idx_small] - reps[idx_small+1])
            j = idx_small-1 if d_left <= d_right else idx_small+1
        lo = min(intervals[idx_small][0], intervals[j][0])
        hi = max(intervals[idx_small][1], intervals[j][1])
        n_new = counts[idx_small] + counts[j]
        reps_new = float(np.median(bmi_all[(bmi_all >= lo) & (bmi_all <= hi)]))
        keep = [k for k in range(len(intervals)) if k not in (idx_small, j)]
        intervals = [intervals[k] for k in keep] + [(round(lo, round_digits), round(hi,

```

```

round_digits))]
        counts = [counts[k] for k in keep] + [n_new]
        reps = [reps[k] for k in keep] + [reps_new]
        order = np.argsort([iv[0] for iv in intervals])
        intervals = [intervals[k] for k in order]
        counts = [counts[k] for k in order]
        reps = [reps[k] for k in order]
    return intervals, counts, reps

def mdlp_disc_7(
    bmi_series: pd.Series,
    labels: np.ndarray,
    min_n: int = 30,
    max_intervals: int = 8
):
    import math
    bmi_data = pd.to_numeric(bmi_series, errors='coerce').astype(float).values
    y = np.asarray(labels, dtype=int)
    mask = np.isfinite(bmi_data) & np.isfinite(y)
    X = bmi_data[mask]
    y = y[mask]
    if len(X) == 0:
        raise RuntimeError('BMI 数据为空，无法进行 MDLP 离散化')
    sorted_indices = np.argsort(X)
    X_sorted = X[sorted_indices]
    y_sorted = y[sorted_indices]
    def entropy_8(lbl):
        if len(lbl) == 0:
            return 0.0
        _, counts = np.unique(lbl, return_counts=True)
        probs = counts / len(lbl)
        return float(-np.sum(probs * np.log2(probs + 1e-12)))
    def mdl_crit_9(left_labels, right_labels, total_labels):
        n = len(total_labels)
        n_left = len(left_labels)
        n_right = len(right_labels)
        if n_left == 0 or n_right == 0:
            return float('inf')
        total_entropy = entropy_8(total_labels)
        weighted_entropy = (n_left/n) * entropy_8(left_labels) + (n_right/n) *
entropy_8(right_labels)

```

```

info_gain = total_entropy - weighted_entropy
k = len(np.unique(total_labels))
description_length = (math.log2(max(n,1)) + k * math.log2(max(n,1))) / max(n,1)
return description_length - info_gain
def find_best_split_10(start_idx, end_idx, depth=0):
    if end_idx - start_idx < 2 * min_n or depth >= math.log2(max_intervals):
        return None
    best_split, best_mdl = None, float('inf')
    for i in range(start_idx + min_n, end_idx - min_n):
        if X_sorted[i] == X_sorted[i-1]:
            continue
        mdl_score = mdl_crit_9(y_sorted[start_idx:i], y_sorted[i:end_idx],
y_sorted[start_idx:end_idx])
        if mdl_score < best_mdl:
            best_mdl = mdl_score
            best_split = i
    if best_split is not None and best_mdl < 0:
        split_value = (X_sorted[best_split-1] + X_sorted[best_split]) / 2
        return split_value, best_split
    return None
def build_splits_11(start_idx, end_idx, depth=0):
    res = find_best_split_10(start_idx, end_idx, depth)
    if res is None:
        return []
    split_value, split_idx = res
    return build_splits_11(start_idx, split_idx, depth+1) + [split_value] +
build_splits_11(split_idx, end_idx, depth+1)
split_points = sorted(list(set(build_splits_11(0, len(X_sorted)))))
if not split_points:
    n_bins = min(max_intervals, max(2, len(X)//max(1,int(min_n))))
    quantiles = np.linspace(0, 100, n_bins + 1)[1:-1]
    split_points = [np.percentile(X, q) for q in quantiles]
x_min, x_max = float(np.min(X)), float(np.max(X))
boundaries = sorted(list(set([x_min] + split_points + [x_max])))
intervals, counts, reps = [], [], []
for i in range(len(boundaries) - 1):
    lo, hi = boundaries[i], boundaries[i+1]
    if i < len(boundaries) - 2:
        mask_i = (bmi_data >= lo) & (bmi_data < hi)
    else:
        mask_i = (bmi_data >= lo) & (bmi_data <= hi)
    cnt = int(np.sum(mask_i))

```

```

        if cnt == 0:
            continue
        intervals.append((round(lo,3), round(hi,3)))
        counts.append(cnt)
        reps.append(float(np.median(bmi_data[mask_i])))
    while len(intervals) > 1 and min(counts) < min_n:
        min_idx = int(np.argmin(counts))
        if min_idx == 0:
            merge_with = 1
        elif min_idx == len(intervals) - 1:
            merge_with = len(intervals) - 2
        else:
            merge_with = min_idx - 1 if counts[min_idx - 1] <= counts[min_idx + 1] else
min_idx + 1
        new_lo = min(intervals[min_idx][0], intervals[merge_with][0])
        new_hi = max(intervals[min_idx][1], intervals[merge_with][1])
        new_cnt = counts[min_idx] + counts[merge_with]
        combined = (bmi_data >= new_lo) & (bmi_data <= new_hi)
        new_rep = float(np.median(bmi_data[combined]))
        keep = [i for i in range(len(intervals)) if i not in (min_idx, merge_with)]
        intervals = [intervals[i] for i in keep] + [(round(new_lo,3), round(new_hi,3))]
        counts = [counts[i] for i in keep] + [new_cnt]
        reps = [reps[i] for i in keep] + [new_rep]
        order = np.argsort([iv[0] for iv in intervals])
        intervals = [intervals[i] for i in order]
        counts = [counts[i] for i in order]
        reps = [reps[i] for i in order]
    return intervals, counts, reps

def crt_bmi_grp_12(df, method='tree_mdlp', n_groups=4):
    if method == 'quantile':
        df['bmi_group'] = pd.qcut(df['bmi'], n_groups, labels=[f'Group_{i+1}' for i in
range(n_groups)])
        bmi_ranges = pd.qcut(df['bmi'], n_groups).cat.categories
    elif method == 'clinical':
        bins = [0, 28, 32, 36, 40, 100]
        labels = ['<28', '28-32', '32-36', '36-40', '≥40']
        df['bmi_group'] = pd.cut(df['bmi'], bins=bins, labels=labels, right=False)
        bmi_ranges = pd.cut(df['bmi'], bins=bins, right=False).cat.categories
    elif method == 'tree_mdlp':
        candidate = [

```

```

        'bmi', 'week', 'age',
        'z13', 'z18', 'z21', 'zx',
        'total_reads', 'mapped_prop', 'dup_prop', 'unique_reads',
        'gc_content', 'gc13', 'gc18', 'gc21', 'filtered_prop'
    ]
    feats_cols = [c for c in candidate if c in df.columns]
    Xc = df[feats_cols].copy()
    Xc = Xc.apply(pd.to_numeric, errors='coerce').astype(float)
    mask = np.isfinite(Xc.values).all(axis=1)
    Xc_valid = Xc[mask].values
    if Xc_valid.shape[0] < 50:
        df['bmi_group'] = pd.qcut(df['bmi'], n_groups, labels=[f'Group_{i+1}' for i in
range(n_groups)])
        bmi_ranges = pd.qcut(df['bmi'], n_groups).cat.categories
    else:
        scaler = StandardScaler().fit(Xc_valid)
        Z = scaler.transform(Xc_valid)
        best_k, _, _ = elbow_km_5(Z, k_min=2, k_max=8)
        km = KMeans(n_clusters=best_k, n_init=20, random_state=42).fit(Z)
        labels = km.labels_
        df_k = df.loc[mask].copy()
        try:
            intervals, counts, _ = build_bmi_it_tree_6(df_k, labels, min_n=30,
max_leaf_nodes=min(6, max(3, best_k+1)))
        except Exception:
            intervals, counts, _ = mdlp_disc_7(df_k['bmi'], labels, min_n=30,
max_intervals=min(8, best_k+2))
        names = []
        for i, (lo, hi) in enumerate(intervals):
            if i < len(intervals) - 1:
                names.append(f'[{lo:.1f}, {hi:.1f})')
            else:
                names.append(f'[{lo:.1f}, {hi:.1f}]')
    def map_one_13(b):
        if pd.isna(b):
            return names[0]
        for i, (lo, hi) in enumerate(intervals):
            if i < len(intervals) - 1:
                if (b >= lo) and (b < hi):
                    return names[i]
            else:
                if (b >= lo) and (b <= hi):

```

```

        return names[i]
    return names[0] if b < intervals[0][0] else names[-1]
    df['bmi_group'] = df['bmi'].apply(map_one_13)
    df['bmi_group'] = pd.Categorical(df['bmi_group'], categories=names,
ordered=True)
    bmi_ranges = pd.CategoricalDtype(categories=names,
ordered=True).categories

    print(f"\nBMI 分组 ({method}):")
    print(df['bmi_group'].value_counts().sort_index())

    return df, bmi_ranges

def fit_bayes_14(df, chromosome='z21', n_samples=1000, n_tune=1000):
    if not HAS_PYMC:
        print("PyMC 未安装，跳过贝叶斯建模")
        return None, None

    print(f"\n=== 拟合 {chromosome.upper()} 的贝叶斯层级模型 ===")

    y_obs = df[chromosome].values
    week_c = df['week_c'].values
    bmi_c = df['bmi_c'].values
    age_c = df['age_c'].values
    ivf = df['ivf_binary'].values
    log_reads = np.log(df['total_reads'].values)
    mapped_prop = df['mapped_prop'].values

    patient_idx = pd.Categorical(df['patient_id']).codes
    batch_idx = pd.Categorical(df['batch']).codes
    n_patients = len(np.unique(patient_idx))
    n_batches = len(np.unique(batch_idx))

    print(f"数据维度: n={len(y_obs)}, 患者数={n_patients}, 批次数={n_batches}")

    def crt_bspline_15(x, n_knots=5, degree=3):
        x_min, x_max = x.min(), x.max()
        knots = np.linspace(x_min, x_max, n_knots)

        extended_knots = np.concatenate([
            np.repeat(x_min, degree),
            knots,

```

```

        np.repeat(x_max, degree)
    ])

    n_basis = len(extended_knots) - degree - 1
    basis_matrix = np.zeros((len(x), n_basis))

    for i in range(n_basis):
        try:
            spl = BSpline(extended_knots, np.eye(n_basis)[i], degree)
            basis_matrix[:, i] = spl(x)
        except:
            basis_matrix[:, i] = x ** i if i < 4 else 0

    return basis_matrix

week_basis = crt_bspline_15(week_c, n_knots=5)
n_spline_coef = week_basis.shape[1]

with pm.Model() as model:
    print("构建贝叶斯模型...")

    intercept = pm.Normal('intercept', 0, 2)
    beta_bmi = pm.Normal('beta_bmi', 0, 1)
    beta_age = pm.Normal('beta_age', 0, 1)
    beta_ivf = pm.Normal('beta_ivf', 0, 1)

    spline_sigma = pm.HalfNormal('spline_sigma', 1)
    spline_coef = pm.Normal('spline_coef', 0, spline_sigma, shape=n_spline_coef)

    sigma_patient = pm.HalfNormal('sigma_patient', 0.5)
    sigma_batch_obs = pm.HalfNormal('sigma_batch_obs', 0.3)

    patient_effect = pm.Normal('patient_effect', 0, sigma_patient, shape=n_patients)
    batch_effect = pm.Normal('batch_effect', 0, sigma_batch_obs, shape=n_batches)

    sigma_base = pm.HalfNormal('sigma_base', 0.5)
    sigma_reads_coef = pm.HalfNormal('sigma_reads_coef', 0.3)
    sigma_mapped_coef = pm.HalfNormal('sigma_mapped_coef', 0.2)

    spline_term = pm.math.dot(week_basis, spline_coef)

    mu_true = (intercept +

```

```

        spline_term +
        beta_bmi * bmi_c +
        beta_age * age_c +
        beta_ivf * ivf +
        patient_effect[patient_idx])

log_sigma_sq = (pm.math.log(sigma_base**2) +
                sigma_reads_coef * (10 - log_reads) +
                sigma_mapped_coef * (1 - mapped_prop))

sigma_obs = pm.math.exp(0.5 * log_sigma_sq)

mu_obs = mu_true + batch_effect[batch_idx]

y_likelihood = pm.Normal('y_obs', mu=mu_obs, sigma=sigma_obs,
observed=y_obs)

print("开始 MCMC 采样...")
trace = pm.sample(
    draws=n_samples,
    tune=n_tune,
    chains=2,
    cores=2,
    return_inferencedata=True,
    progressbar=True
)

print("\n=== MCMC 诊断 ===")
print(az.summary(trace, var_names=['intercept', 'beta_bmi', 'beta_age',
'sigma_base']))

rhat_vals = az.rhat(trace)
max_rhat = float(rhat_vals.max())
print(f"最大 R-hat: {max_rhat:.4f}")

if max_rhat > 1.1:
    print("警告: 部分参数 R-hat > 1.1, 可能未收敛")
else:
    print("所有参数 R-hat < 1.1, 收敛良好")

return trace, model

```

```

def post_pred_16(trace, model, df, chromosome='z21'):
    if not HAS_PYMC or trace is None:
        print("跳过后验预测分析")
        return

    print(f"\n=== {chromosome.upper()} 后验预测分析 ===")

    with model:
        ppc = pm.sample_posterior_predictive(trace, progressbar=True)

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    y_obs = df[chromosome].values
    y_pred_mean = ppc.posterior_predictive['y_obs'].mean(dim=['chain', 'draw']).values

    axes[0].scatter(y_obs, y_pred_mean, alpha=0.5)
    lims = [min(y_obs.min(), y_pred_mean.min()), max(y_obs.max(), y_pred_mean.max())]
    axes[0].plot(lims, lims, 'r--', alpha=0.8)
    axes[0].set_xlabel('观测值')
    axes[0].set_ylabel('预测值')
    axes[0].set_title('观测 vs 预测')
    axes[0].grid(True, alpha=0.3)

    residuals = y_obs - y_pred_mean
    axes[1].hist(residuals, bins=50, alpha=0.7, density=True)
    axes[1].axvline(0, color='red', linestyle='--')
    axes[1].set_xlabel('残差')
    axes[1].set_ylabel('密度')
    axes[1].set_title('残差分布')
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(f'输出 4/ 问题四_{chromosome}_后验预测检验.png', dpi=300,
bbox_inches='tight')
    plt.close()

    return ppc

def calc_tris_prob_17(trace, model, df, chromosome='z21', z_threshold=3.0):
    if not HAS_PYMC or trace is None:
        print("跳过 trisomy 概率计算")
        return None

```

```
print(f"\n=== 计算 {chromosome.upper()} Trisomy 概率 (阈值={z_threshold}) ===")
```

```
with model:
```

```
    ppc = pm.sample_posterior_predictive(trace, progressbar=True)
```

```
y_pred_samples = ppc.posterior_predictive['y_obs'].values
```

```
prob_trisomy = (y_pred_samples > z_threshold).mean(axis=(0, 1))
```

```
df[f'prob_trisomy_{chromosome}'] = prob_trisomy
```

```
print(f'Trisomy 概率统计:')
```

```
print(f'    平均概率: {prob_trisomy.mean():.4f}')
```

```
print(f'    概率>0.5 的样本数: {(prob_trisomy > 0.5).sum()}')
```

```
print(f'    概率>0.9 的样本数: {(prob_trisomy > 0.9).sum()}')
```

```
return prob_trisomy
```

```
def anl_bmi_bayes_18(trace, model, df, chromosome='z21', z_threshold=3.0, p0=0.95):
```

```
    if not HAS_PYMC or trace is None:
```

```
        print("跳过 BMI 组别贝叶斯分析")
```

```
        return None
```

```
print(f"\n=== BMI 组别 {chromosome.upper()} 最佳时点分析 ===")
```

```
posterior = trace.posterior
```

```
week_grid = np.linspace(10, 25, 151)
```

```
week_grid_c = (week_grid - df['week'].mean()) / df['week'].std()
```

```
results = []
```

```
for group_name in df['bmi_group'].cat.categories:
```

```
    print(f'\n 分析 BMI 组: {group_name}')
```

```
    group_data = df[df['bmi_group'] == group_name]
```

```
    bmi_rep = group_data['bmi'].median()
```

```
    bmi_rep_c = (bmi_rep - df['bmi'].mean()) / df['bmi'].std()
```

```
    age_rep_c = (group_data['age'].median() - df['age'].mean()) / df['age'].std()
```

```
    ivf_rep = group_data['ivf_binary'].mode().iloc[0] if
```

```
len(group_data['ivf_binary'].mode()) > 0 else 0
```

```
print(f"  代表 BMI: {bmi_rep:.1f}")
print(f"  样本数: {len(group_data)}")
```

```
try:
```

```
    intercept_mean = posterior['intercept'].mean().values
    beta_bmi_mean = posterior['beta_bmi'].mean().values
    beta_age_mean = posterior['beta_age'].mean().values
    beta_ivf_mean = posterior['beta_ivf'].mean().values
    spline_coef_mean = posterior['spline_coef'].mean(dim=['chain', 'draw']).values
```

```
    spline_term = np.polyval(spline_coef_mean[:-1][:3], week_grid_c)
```

```
    mu_pred = (intercept_mean +
               spline_term +
               beta_bmi_mean * bmi_rep_c +
               beta_age_mean * age_rep_c +
               beta_ivf_mean * ivf_rep)
```

```
    sigma_mean = posterior['sigma_base'].mean().values
    prob_curve = 1 - stats.norm.cdf(z_threshold, mu_pred, sigma_mean)
```

```
    valid_weeks = week_grid[prob_curve >= p0]
```

```
    if len(valid_weeks) > 0:
```

```
        optimal_week = valid_weeks[0]
        status = f" {optimal_week:.1f} 周"
```

```
    else:
```

```
        max_prob = prob_curve.max()
        max_week = week_grid[np.argmax(prob_curve)]
        status = f"未达阈值(峰值 {max_prob:.3f} @ {max_week:.1f} 周)"
```

```
    results.append({
```

```
        'BMI 组': group_name,
```

```
        'BMI 范 围': f"[{group_data['bmi'].min():.1f}, {group_data['bmi'].max():.1f})",
```

```
        '样本数': len(group_data),
```

```
        '最佳时点': status,
```

```
        '代表 BMI': bmi_rep,
```

```
        '概率曲线': prob_curve
```

```
    })
```

```

        print(f' 最佳检测时点: {status}')

    except Exception as e:
        print(f' 预测失败: {e}')
        results.append({
            'BMI 组': group_name,
            'BMI 范 围': f"[{group_data['bmi'].min():.1f},
{group_data['bmi'].max():.1f})",
            '样本数': len(group_data),
            '最佳时点': "计算失败",
            '代表 BMI': bmi_rep,
            '概率曲线': None
        })

    results_df = pd.DataFrame(results)
    results_df.to_csv(f'输出 4/问题四_{chromosome}_BMI 组别分析.csv', index=False,
encoding='utf-8-sig')

    plt.figure(figsize=(12, 8))

    colors = plt.cm.Set1(np.linspace(0, 1, len(results)))

    for i, result in enumerate(results):
        if result['概率曲线'] is not None:
            plt.plot(week_grid, result['概率曲线'],
                    label=f'{result["BMI 组"]} (BMI={result["代表 BMI"]:.1f})',
                    color=colors[i], linewidth=2)

    plt.axhline(p0, color='red', linestyle='--', alpha=0.7, label=f'目标概率={p0}')
    plt.xlabel('孕周')
    plt.ylabel(f'{chromosome.upper()} > {z_threshold} 的概率')
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.grid(True, alpha=0.3)
    plt.xlim(10, 25)
    plt.ylim(0, 1)

    plt.tight_layout()
    plt.savefig(f'输出 4/问题四_{chromosome}_BMI 概率曲线.png', dpi=300,
bbox_inches='tight')
    plt.close()

    return results_df, results

```

```

def sens_anl_19(df, chromosomes=['z13', 'z18', 'z21']):
    print("\n=== 敏感性分析 ===")

    thresholds = [2.5, 2.8, 3.0, 3.2, 3.5]

    results = []

    for chrom in chromosomes:
        for thresh in thresholds:
            n_positive = (df[chrom] > thresh).sum()
            prop_positive = n_positive / len(df)

            results.append({
                '染色体': chrom.upper(),
                '阈值': thresh,
                '阳性数': n_positive,
                '阳性率': prop_positive
            })

    sens_df = pd.DataFrame(results)
    sens_df.to_csv('输出 4/ 问题四 _ 阈值 敏感性 分析 .csv', index=False,
encoding='utf-8-sig')

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for chrom in chromosomes:
    chrom_data = sens_df[sens_df['染色体'] == chrom.upper()]
    axes[0].plot(chrom_data['阈值'], chrom_data['阳性数'],
                marker='o', label=chrom.upper(), linewidth=2)

axes[0].set_xlabel('Z 值阈值')
axes[0].set_ylabel('阳性样本数')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

for chrom in chromosomes:
    chrom_data = sens_df[sens_df['染色体'] == chrom.upper()]
    axes[1].plot(chrom_data['阈值'], chrom_data['阳性率'],
                marker='o', label=chrom.upper(), linewidth=2)

axes[1].set_xlabel('Z 值阈值')

```

```

axes[1].set_ylabel('阳性率')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('输出 4/问题四_阈值敏感性.png', dpi=300, bbox_inches='tight')
plt.close()

return sens_df

def _build_ml_feat_20(df):
    y = (~df['trisomy_label'].isna()).astype(int).values

    candidate_features = [
        'z13', 'z18', 'z21', 'zx',
        'bmi', 'age', 'week',
        'total_reads', 'mapped_prop', 'dup_prop', 'unique_reads',
        'gc_content', 'gc13', 'gc18', 'gc21',
        'filtered_prop',
        'pregnancy_count', 'birth_count'
    ]
    features = [f for f in candidate_features if f in df.columns]
    X = df[features].copy()

    groups = pd.Categorical(df['patient_id']).codes

    gam_features = [f for f in ['z13', 'z18', 'z21', 'bmi', 'week', 'mapped_prop', 'gc_content'] if
f in features]
    X_gam = df[gam_features].copy() if len(gam_features) > 0 else None

    return X, y, groups, features, X_gam, gam_features

def _sigmoid_21(x):
    return 1.0 / (1.0 + np.exp(-x))

def _oof_glm_ri_22(df, features, y, groups_codes, n_splits=5):
    gkf = GroupKFold(n_splits=n_splits)
    oof = np.full(len(y), np.nan, dtype=float)

    for tr_idx, va_idx in gkf.split(df[features], y, groups=groups_codes):
        try:

```

```

df_tr = df.iloc[tr_idx].copy()
df_va = df.iloc[va_idx].copy()

patient_means = df_tr.groupby('patient_id')[features].mean()
patient_means.columns = [f'{col}_patient_mean' for col in
patient_means.columns]

global_means = df_tr[features].mean()
global_means_dict = {f'{col}_patient_mean': global_means[col] for col in
features}

df_tr_aug = df_tr.merge(patient_means, left_on='patient_id', right_index=True,
how='left')

df_va_aug = df_va.copy()
for col in features:
    patient_mean_col = f'{col}_patient_mean'
    df_va_aug[patient_mean_col] = df_va['patient_id'].map(
        patient_means[patient_mean_col] if patient_mean_col in
patient_means.columns else pd.Series()
    ).fillna(global_means_dict[patient_mean_col])

enhanced_features = features + [f'{col}_patient_mean' for col in features]

pipe = make_pipeline(
    SimpleImputer(strategy='median'),
    StandardScaler(),
    LogisticRegression(
        max_iter=2000,
        class_weight='balanced',
        solver='liblinear',
        C=0.5,
        random_state=42
    )
)

pipe.fit(df_tr_aug[enhanced_features], y[tr_idx])
oof_proba = pipe.predict_proba(df_va_aug[enhanced_features])[:, 1]

prob_std = np.std(oof_proba)
if prob_std < 0.1:
    prob_mean = np.mean(oof_proba)

```

```

        oof_proba = prob_mean + (oof_proba - prob_mean) * 2.0
        oof_proba = np.clip(oof_proba, 0.01, 0.99)

    oof[va_idx] = oof_proba

except Exception as e:
    try:
        pipe = make_pipeline(
            SimpleImputer(strategy='median'),
            StandardScaler(),
            LogisticRegression(max_iter=1000, class_weight='balanced',
                               solver='lbfgs', C=0.1, random_state=42)
        )
        pipe.fit(df.iloc[tr_idx][features], y[tr_idx])
        base_proba = pipe.predict_proba(df.iloc[va_idx][features])[:, 1]

        np.random.seed(42)
        noise = np.random.normal(0, 0.02, len(base_proba))
        oof[va_idx] = np.clip(base_proba + noise, 0.01, 0.99)

    except Exception:
        oof[va_idx] = 0.5

return oof

def run_ml_grp_cv_23(df, n_splits=5, random_state=42, use_smote=True):
    print("\n=== 机器学习按孕妇分组交叉验证（GroupKFold） ===")
    X, y, groups, features, X_gam, gam_features = _build_ml_feat_20(df)
    n = len(y)
    print(f"样本数={n}, 阳性={y.sum()} ({y.mean():.2%}), 特征数={len(features)}")

    use_smote = bool(use_smote and HAS_IMBLEARN)
    if use_smote:
        print("使用 SMOTE 过采样以缓解类别不平衡（按训练折内执行）")
        models = {
            'Logit': ImbPipeline(steps=[
                ('imputer', SimpleImputer(strategy='median')),
                ('smote', SMOTE(random_state=random_state, sampling_strategy=0.5,
                                k_neighbors=3)),
                ('scaler', StandardScaler(with_mean=True, with_std=True)),
                ('clf', LogisticRegression(max_iter=1000, class_weight='balanced',
                                           solver='lbfgs'))
            ]

```

```

    ]),
    'GB': ImbPipeline(steps=[
        ('imputer', SimpleImputer(strategy='median')),
        ('smote', SMOTE(random_state=random_state, sampling_strategy=0.5,
k_neighbors=3))),
        ('clf', GradientBoostingClassifier(random_state=random_state))
    ]),
    'RF': ImbPipeline(steps=[
        ('imputer', SimpleImputer(strategy='median')),
        ('smote', SMOTE(random_state=random_state, sampling_strategy=0.5,
k_neighbors=3))),
        ('clf', RandomForestClassifier(
            n_estimators=400,
            min_samples_leaf=3,
            random_state=random_state,
            n_jobs=-1,
            class_weight='balanced_subsample'
        ))
    ]),
    'DT': ImbPipeline(steps=[
        ('imputer', SimpleImputer(strategy='median')),
        ('smote', SMOTE(random_state=random_state, sampling_strategy=0.5,
k_neighbors=3))),
        ('clf', DecisionTreeClassifier(max_depth=5, min_samples_leaf=10,
                                     class_weight='balanced',
random_state=random_state))
    ])
}
else:
    if not HAS_IMBLEARN and use_smote:
        print("提示: 未检测到 imbalanced-learn, SMOTE 不可用, 按原方式训练")
    models = {
        'Logit': make_pipeline(
            SimpleImputer(strategy='median'),
            StandardScaler(with_mean=True, with_std=True),
            LogisticRegression(max_iter=1000, class_weight='balanced',
solver='lbfgs')
        ),
        'GB': make_pipeline(
            SimpleImputer(strategy='median'),
            StandardScaler(with_mean=True, with_std=True),
            GradientBoostingClassifier(random_state=random_state)

```

```

    ),
    'RF': make_pipeline(
        SimpleImputer(strategy='median'),
        RandomForestClassifier(
            n_estimators=400,
            min_samples_leaf=3,
            random_state=random_state,
            n_jobs=-1,
            class_weight='balanced_subsample'
        )
    ),
    'DT': make_pipeline(
        SimpleImputer(strategy='median'),
        DecisionTreeClassifier(max_depth=5, min_samples_leaf=10,
                               class_weight='balanced',
random_state=random_state)
    )
}

if HAS_XGB:
    if use_smote:
        models['XGB'] = ImbPipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
            ('smote', SMOTE(random_state=random_state, sampling_strategy=0.5,
k_neighbors=3)),
            ('clf', XGBClassifier(
                n_estimators=400,
                max_depth=3,
                learning_rate=0.05,
                subsample=0.9,
                colsample_bytree=0.8,
                reg_lambda=1.0,
                objective='binary:logistic',
                eval_metric='logloss',
                n_jobs=-1,
                random_state=random_state
            ))
        ])
    else:
        models['XGB'] = make_pipeline(
            SimpleImputer(strategy='median'),
            XGBClassifier(

```

```

        n_estimators=400,
        max_depth=3,
        learning_rate=0.05,
        subsample=0.9,
        colsample_bytree=0.8,
        reg_lambda=1.0,
        objective='binary:logistic',
        eval_metric='logloss',
        n_jobs=-1,
        random_state=random_state
    )
)
else:
    print("提示: 未检测到 xgboost, 跳过 XGB 模型")

use_gam = HAS_PYGAM and (X_gam is not None) and (len(gam_features) >= 3)
if use_gam:
    print(f"GAM 将使用特征: {gam_features}")
else:
    print("提示: 未检测到 pygam 或 GAM 特征不足, 跳过 GAM 模型")

gkf = GroupKFold(n_splits=n_splits)
oof_results = {}
metrics_rows = []

for name in list(models.keys()) + (["GAM"] if use_gam else []):
    oof_results[name] = np.full(n, np.nan, dtype=float)

fold_idx = 0
for tr_idx, va_idx in gkf.split(X, y, groups=groups):
    fold_idx += 1
    print(f"Fold {fold_idx}/{n_splits} 训练: {len(tr_idx)}, 验证: {len(va_idx)}")

    X_tr, X_va = X.iloc[tr_idx], X.iloc[va_idx]
    y_tr, y_va = y[tr_idx], y[va_idx]

    for name, pipe in models.items():
        try:
            pipe.fit(X_tr, y_tr)
            if hasattr(pipe[-1], 'predict_proba'):
                proba = pipe.predict_proba(X_va)[: , 1]
            elif hasattr(pipe[-1], 'decision_function'):

```

```

        proba = _sigmoid_21(pipe.decision_function(X_va))
    else:
        proba = pipe.predict(X_va).astype(float)
    oof_results[name][va_idx] = proba
except Exception as e:
    print(f"  模型 {name} 在 fold {fold_idx} 失败: {e}")

if use_gam:
    try:
        Xg_tr = X_gam.iloc[tr_idx].values
        Xg_va = X_gam.iloc[va_idx].values
        terms = s(0) + s(1) + s(2)
        for j in range(3, Xg_tr.shape[1]):
            terms += l(j)
        gam = LogisticGAM(terms=terms)
        gam.fit(Xg_tr, y_tr)
        proba_gam = gam.predict_proba(Xg_va)
        oof_results['GAM'][va_idx] = proba_gam
    except Exception as e:
        print(f"  模型 GAM 在 fold {fold_idx} 失败: {e}")

if HAS_SM_GLMM:
    try:
        glmm_features = [f for f in features if f in X.columns]
        df_tr = df.iloc[tr_idx]
        df_va = df.iloc[va_idx]
    except Exception as e:
        pass

plt.figure(figsize=(8, 6))
if HAS_SM_GLMM:
    try:
        glmm_features = [f for f in features if f in X.columns]
        oof_glmm = _oof_glmm_ri_22(df, glmm_features, y, groups,
n_splits=n_splits)
        oof_results['GLMM_RE'] = oof_glmm
    except Exception as e:
        print(f"  GLMM_RE 失败: {e}")

for name, proba in oof_results.items():
    mask = ~np.isnan(proba)
    if mask.sum() == 0:

```

```

        continue
    y_true = y[mask]
    y_prob = proba[mask]

    try:
        auc = roc_auc_score(y_true, y_prob)
    except Exception:
        auc = np.nan

    try:
        brier = brier_score_loss(y_true, y_prob)
    except Exception:
        brier = np.nan

    try:
        ll = log_loss(y_true, np.vstack([1-y_prob, y_prob]).T)
    except Exception:
        ll = np.nan

    acc = accuracy_score(y_true, (y_prob >= 0.5).astype(int))

    metrics_rows.append({
        '模型': name,
        'AUC': auc,
        'Brier': brier,
        'LogLoss': ll,
        'Accuracy@0.5': acc,
        '有效样本': int(mask.sum())
    })

    try:
        fpr, tpr, _ = roc_curve(y_true, y_prob)
        plt.plot(fpr, tpr, linewidth=2, label=f'{name} AUC={auc:.3f}')
    except Exception:
        pass

plt.plot([0,1], [0,1], 'k--', alpha=0.5)
plt.xlabel('假阳性率 FPR')
plt.ylabel('真阳性率 TPR')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('输出 4/问题四_ML_ROC 曲线对比.png', dpi=300, bbox_inches='tight')
plt.close()

```

```

metrics_df = pd.DataFrame(metrics_rows)
metrics_df.sort_values('AUC', ascending=False, inplace=True)
metrics_df.to_csv('输出 4/问题四_ML_交叉验证_按 PID.csv', index=False,
encoding='utf-8-sig')

if 'DT' in models:
    try:
        if HAS_IMBLEARN:
            dt_pipe = ImbPipeline(steps=[
                ('imputer', SimpleImputer(strategy='median')),
                ('smote', SMOTE(random_state=random_state,
sampling_strategy=0.5, k_neighbors=3)),
                ('clf', DecisionTreeClassifier(max_depth=5, min_samples_leaf=10,
class_weight='balanced',
random_state=random_state))
            ])
        else:
            dt_pipe = make_pipeline(
                SimpleImputer(strategy='median'),
                DecisionTreeClassifier(max_depth=5, min_samples_leaf=10,
class_weight='balanced',
random_state=random_state)
            )
        dt_pipe.fit(X, y)
        tree = dt_pipe.named_steps.get('clf') if hasattr(dt_pipe, 'named_steps') else
dt_pipe[-1]
        rules_txt = export_text(tree, feature_names=list(X.columns), max_depth=4)
        with open('输出 4/问题四_决策树_简化规则.txt', 'w', encoding='utf-8-sig') as
f:
            f.write(rules_txt)
    except Exception:
        pass

print("机器学习分组交叉验证完成，已输出：")
print("- 输出 4/问题四_ML_交叉验证_按 PID.csv")
print("- 输出 4/问题四_ML_ROC 曲线对比.png")
return metrics_df

def add_per_chr_lbl_24(df):
    txt = df['trisomy_label'].astype(str).fillna("")
    norm = txt.str.replace('; ', ',').str.replace(', ', ',').str.replace('\ ', ',')
    for chrom in ['13', '18', '21']:

```

```

        df[f'y{chrom}'] = norm.str.contains(chrom, regex=False).astype(int)
    return df

def _choose_thr_grp_25(y_true, y_prob, groups, group_names, target_metric='youden'):
    thresholds = {}
    for g in group_names:
        mask = (groups == g)
        y = y_true[mask]
        p = y_prob[mask]
        if len(y) == 0 or y.sum() == 0 or y.sum() == len(y):
            thresholds[g] = 0.5
            continue
        cand = np.unique(np.clip(p, 0, 1))
        if cand.size > 200:
            cand = np.quantile(cand, np.linspace(0, 1, 200))
        best_t, best_j = 0.5, -1
        for t in cand:
            pred = (p >= t).astype(int)
            tp = int(((pred == 1) & (y == 1)).sum())
            fp = int(((pred == 1) & (y == 0)).sum())
            fn = int(((pred == 0) & (y == 1)).sum())
            tn = int(((pred == 0) & (y == 0)).sum())
            tpr = tp / (tp + fn) if (tp + fn) > 0 else 0.0
            fpr = fp / (fp + tn) if (fp + tn) > 0 else 0.0
            j = tpr - fpr
            if j > best_j:
                best_j, best_t = j, t
        thresholds[g] = float(best_t)
    return thresholds

def run_chr_spec_rule_26(df, n_splits=5, random_state=42, use_gc_layer=True,
gc_low=0.40, gc_high=0.60, use_smote=True):
    print("\n=== 基于每染色体的判别方法（BMI 自适应阈值） ===")
    df = add_per_chr_lbl_24(df.copy())

    if 'bmi_group' not in df.columns or not str(df['bmi_group'].dtype).startswith('category'):
        df, _ = crt_bmi_grp_12(df, method='quantile', n_groups=4)
    group_names = list(df['bmi_group'].cat.categories)
    bmi_group_arr = df['bmi_group'].astype(str).values

    def gc_flag_27(col):
        return ((df[col] >= gc_low) & (df[col] <= gc_high)).astype(int) if col in df.columns

```

```

else pd.Series(1, index=df.index)
    gc_ok_total = gc_flag_27('gc_content')

X, _, groups, features, X_gam, gam_features = _build_ml_feat_20(df)
gkf = GroupKFold(n_splits=n_splits)

def oof_for_target_28(y_target):
    use_local_smote = bool(use_smote and HAS_IMBLEARN)
    if use_local_smote:
        models = {
            'Logit': ImbPipeline(steps=[('imputer', SimpleImputer(strategy='median')),
                                         ('smote',
SMOTE(random_state=random_state, sampling_strategy=0.5, k_neighbors=3)),
                                         ('scaler', StandardScaler()),
                                         ('clf',
LogisticRegression(max_iter=1000, class_weight='balanced', solver='lbfgs'))]),
            'GB': ImbPipeline(steps=[('imputer', SimpleImputer(strategy='median')),
                                     ('smote',
SMOTE(random_state=random_state, sampling_strategy=0.5, k_neighbors=3)),
                                     ('clf',
GradientBoostingClassifier(random_state=random_state))]),
            'RF': ImbPipeline(steps=[('imputer', SimpleImputer(strategy='median')),
                                     ('smote',
SMOTE(random_state=random_state, sampling_strategy=0.5, k_neighbors=3)),
                                     ('clf',
RandomForestClassifier(n_estimators=400, min_samples_leaf=3,
random_state=random_state, n_jobs=-1,
class_weight='balanced_subsample'))]),
            'DT': ImbPipeline(steps=[('imputer', SimpleImputer(strategy='median')),
                                     ('smote',
SMOTE(random_state=random_state, sampling_strategy=0.5, k_neighbors=3)),
                                     ('clf',
DecisionTreeClassifier(max_depth=5, min_samples_leaf=10,
class_weight='balanced', random_state=random_state))])
        }
    else:
        models = {
            'Logit': make_pipeline(SimpleImputer(strategy='median'),
                                  StandardScaler(),

```

```

        LogisticRegression(max_iter=1000,
class_weight='balanced', solver='lbfgs')),
        'GB': make_pipeline(SimpleImputer(strategy='median'),
                             StandardScaler(),

GradientBoostingClassifier(random_state=random_state)),
        'RF': make_pipeline(SimpleImputer(strategy='median'),
                             RandomForestClassifier(n_estimators=400,
min_samples_leaf=3,

random_state=random_state, n_jobs=-1,

class_weight='balanced_subsample')),
        'DT': make_pipeline(SimpleImputer(strategy='median'),
                             DecisionTreeClassifier(max_depth=5,
min_samples_leaf=10,

class_weight='balanced', random_state=random_state))
    }
    if HAS_XGB:
        if use_local_smote:
            models['XGB'] = ImbPipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
                                                ('smote',
SMOTE(random_state=random_state, sampling_strategy=0.5, k_neighbors=3)),
                                                ('clf',
XGBClassifier(n_estimators=400, max_depth=3, learning_rate=0.05,

subsample=0.9, colsample_bytree=0.8, reg_lambda=1.0,

objective='binary:logistic', eval_metric='logloss',

n_jobs=-1, random_state=random_state))])
        else:
            models['XGB'] = make_pipeline(SimpleImputer(strategy='median'),
                                           XGBClassifier(n_estimators=400,

max_depth=3, learning_rate=0.05,
                                                         subsample=0.9,

colsample_bytree=0.8, reg_lambda=1.0,

objective='binary:logistic', eval_metric='logloss',
                                                         n_jobs=-1,

```

```

random_state=random_state))
    base_name = 'Logit'
    oof = {name: np.full(len(y_target), np.nan) for name in models.keys()}
    for tr, va in gkf.split(X, y_target, groups=groups):
        for name, mdl in models.items():
            try:
                mdl.fit(X.iloc[tr], y_target[tr])
                proba = mdl.predict_proba(X.iloc[va])[:, 1] if hasattr(mdl[-1],
'predict_proba') else \
                    _sigmoid_21(mdl.decision_function(X.iloc[va]))
                oof[name][va] = proba
            except Exception as e:
                print(f" {name} 失败: {e}")
    return oof, base_name

outputs = []
thresh_rows = []
final_pred_any = np.zeros(len(df), dtype=int)

for chrom, ycol in [('13', 'y13'), ('18', 'y18'), ('21', 'y21')]:
    y = df[ycol].values
    oof_probs_dict, base_name = oof_for_target_28(y)
    base_prob = oof_probs_dict[base_name]
    if use_gc_layer:
        gc_col = f'gc{chrom}' if f'gc{chrom}' in df.columns else 'gc_content'
        gc_ok_chrom = gc_flag_27(gc_col)
        gc_state = np.where((gc_ok_total.values == 1) & (gc_ok_chrom.values == 1),
'GC_OK', 'GC_BAD')
        thresholds = {}
        for g in group_names:
            for s in ['GC_OK', 'GC_BAD']:
                mask = (bmi_group_arr == g) & (gc_state == s)
                if mask.sum() == 0:
                    thresholds[(g, s)] = 0.5
                    continue
                thresholds[(g, s)] = _choose_thr_grp_25(y[mask], base_prob[mask],
np.array([g]*mask.sum()), [g])[g]
                n_g = int(mask.sum())
                pos_g = int((y[mask] == 1).sum())
                thresh_rows.append({'染色体': chrom, 'BMI 组': g, 'GC 状态': s, '阈
值': thresholds[(g, s)], '样本数': n_g, '阳性数': pos_g})
                pred_chrom = np.array([1 if base_prob[i] >= thresholds[(bmi_group_arr[i],

```

```

gc_state[i])) else 0 for i in range(len(df))]
    else:
        thresholds = _choose_thr_grp_25(y, base_prob, bmi_group_arr, group_names)
        for g in group_names:
            n_g = int((bmi_group_arr == g).sum())
            pos_g = int(((bmi_group_arr == g) & (y == 1)).sum())
            thresh_rows.append({'染色体': chrom, 'BMI 组': g, 'GC 状态': 'NA', '阈值': thresholds[g], '样本数': n_g, '阳性数': pos_g})
        pred_chrom = np.array([1 if base_prob[i] >= thresholds[bmi_group_arr[i]] else 0 for i in range(len(df))])
        final_pred_any = np.maximum(final_pred_any, pred_chrom)
        out_chrom = pd.DataFrame({
            'patient_id': df['patient_id'],
            'BMI 组': bmi_group_arr,
            f'OOF_{chrom}': base_prob,
            f'pred_{chrom}': pred_chrom,
            f'y_{chrom}': y
        })
        outputs.append(out_chrom)

    thresh_df = pd.DataFrame(thresh_rows)
    pd.concat(outputs, axis=1).to_csv('输出 4/问题四_每染色体_OOF 概率与判定.csv',
index=False, encoding='utf-8-sig')
    thresh_df.to_csv('输出 4/问题四_每染色体_分组阈值.csv', index=False,
encoding='utf-8-sig')

    y_any = ((df['y13'] + df['y18'] + df['y21']) > 0).astype(int).values
    final_df = pd.DataFrame({
        'patient_id': df['patient_id'],
        'BMI 组': bmi_group_arr,
        'pred_any': final_pred_any,
        'y_any': y_any
    })
    final_df.to_csv('输出 4/问题四_最终判定_OOF.csv', index=False, encoding='utf-8-sig')

    acc = (final_pred_any == y_any).mean()
    try:
        auc = roc_auc_score(y_any, final_pred_any)
    except Exception:
        auc = np.nan
    print(f'最终规则(任一染色体异常) OOF Accuracy={acc:.3f} AUC~{auc if not np.isnan(auc) else 'NA'}")

```

```

print("已输出： ")
print("- 输出 4/问题四_每染色体_OOF 概率与判定.csv")
print("- 输出 4/问题四_每染色体_分组阈值.csv")
print("- 输出 4/问题四_最终判定_OOF.csv")
return thresh_df, final_df

def write_final_sch_29(df, thresh_csv_path='输出 4/问题四_每染色体_分组阈值.csv',
scheme_path='输出 4/问题四_最终判定方案.txt', gc_low=0.40, gc_high=0.60):
    lines = []
    lines.append("问题四 女胎异常最终判定方案\n")
    lines.append("一、 BMI 分组（基于分位数）： ")
    if 'bmi_group' in df.columns and str(df['bmi_group'].dtype).startswith('category'):
        for cat in df['bmi_group'].cat.categories:
            mask = (df['bmi_group'] == cat)
            if mask.any():
                lo, hi = float(df.loc[mask, 'bmi'].min()), float(df.loc[mask, 'bmi'].max())
                lines.append(f"    - {cat}: [{lo:.1f}, {hi:.1f})")
    lines.append("")
    lines.append("二、 可见闭式规则（便于人工快速判读）： ")
    lines.append("    - 不设 GC 硬门, GC 作为连续变量供模型使用; 人工快速判读可用:")
    lines.append("任一 Z13、Z18、Z21  $\geq 3 \Rightarrow$  女胎异常, 否则正常。")
    lines.append("")
    lines.append("四、 数据驱动模型阈值（用于系统自动判读, 可替代闭式规则）:")
    try:
        thr = pd.read_csv(thresh_csv_path)
        has_gc = 'GC 状态' in thr.columns
        if has_gc:
            lines.append("    - 采用 BMI $\times$ GC 分层自适应阈值（OOF 校准）： \n")
            for chrom in ['13','18','21']:
                lines.append(f"        染色体{chrom}: ")
                sub = thr[thr['染色体'].astype(str) == chrom]
                for _, r in sub.iterrows():
                    lines.append(f"        - {r['BMI 组']} | {r.get('GC 状态','NA')}: 阈值")
                    lines.append(f"        = {float(r['阈值']):.4f} (样本数={int(r['样本数'])}, 阳性数={int(r['阳性数'])})")
            else:
                lines.append("    - 采用 BMI 分组自适应阈值（OOF 校准）： \n")
                for chrom in ['13','18','21']:
                    lines.append(f"        染色体{chrom}: ")
                    sub = thr[thr['染色体'].astype(str) == chrom]
                    for _, r in sub.iterrows():

```

```

        lines.append(f'      - {r['BMI 组']}: 阈值={float(r['阈值']):.4f} (样
本数={int(r['样本数'])}, 阳性数={int(r['阳性数']))')
    except Exception as e:
        lines.append(f' [提示] 未找到阈值文件或解析失败: {e}')
    with open(scheme_path, 'w', encoding='utf-8-sig') as f:
        f.write('\n'.join(lines) + '\n')
    print(f'已生成最终判定方案: {scheme_path}')

def fit_z_gc_thr_30(df, p0=0.5):
    df = add_per_chr_lbl_24(df.copy())
    if 'bmi_group' not in df.columns or not str(df['bmi_group'].dtype).startswith('category'):
        df, _ = crt_bmi_grp_12(df, method='quantile', n_groups=4)
    results_coef = []
    results_demo = []
    logit_p0 = np.log(p0/(1-p0))
    for chrom in ['13','18','21']:
        zcol = f'z{chrom}'
        gccol = f'gc{chrom}' if f'gc{chrom}' in df.columns else 'gc_content'
        for g in df['bmi_group'].cat.categories:
            sub = df[df['bmi_group'] == g].dropna(subset=[zcol, 'gc_content', gccol])
            if len(sub) < 30:
                continue
            y = sub[f'y{chrom}'].values if f'y{chrom}' in sub.columns else
(sub['trisomy_label'].astype(str).str.contains(chrom)).astype(int).values
            X = pd.DataFrame({
                zcol: sub[zcol].values,
                'gc_total': sub['gc_content'].values,
                'gc_chr': sub[gccol].values
            })
            X['z_gc_total'] = X[zcol]*X['gc_total']
            X['z_gc_chr'] = X[zcol]*X['gc_chr']
            try:
                clf = LogisticRegression(C=1e6, solver='lbfgs', max_iter=500)
                clf.fit(X.values, y)
                a = float(clf.intercept_[0])
                b, c, d, e, f = [float(v) for v in clf.coef_[0]]
                results_coef.append({
                    '染色体': chrom, 'BMI 组': g,
                    'a': a, 'b_Z': b, 'c_GCt': c, 'd_GCj': d, 'e_ZxGCt': e, 'f_ZxGCj': f,
                    '样本数': int(len(sub)), '阳性数': int(y.sum())
                })
            except:
                gc_t_med = float(sub['gc_content'].median())

```

```

        gc_j_med = float(sub[gccol].median())
        denom = (b + e*gc_t_med + f*gc_j_med)
        z_star = (logit_p0 - a - c*gc_t_med - d*gc_j_med)/denom if denom != 0
    else np.nan

    results_demo.append({
        '染色体': chrom, 'BMI 组': g,
        'GC_total@50%': gc_t_med, 'GC_chr@50%': gc_j_med,
        f'Z* (p={p0})': z_star,
        '样本数': int(len(sub)), '阳性数': int(y.sum())
    })
except Exception:
    continue

if results_coef:
    pd.DataFrame(results_coef).to_csv('输出 4/问题四_Z-GC 阈值_按 BMI 按染色体.csv', index=False, encoding='utf-8-sig')
    if results_demo:
        pd.DataFrame(results_demo).to_csv('输出 4/问题四_Z-GC 阈值_示例阈值.csv', index=False, encoding='utf-8-sig')
    print('已生成 Z-GC 联合阈值: 输出 4/问题四_Z-GC 阈值_按 BMI 按染色体.csv 与 输出 4/问题四_Z-GC 阈值_示例阈值.csv')

def exp_z_gc_bd_31(df, p0=0.5, n_points=61):
    df = add_per_chr_lbl_24(df.copy())
    if 'bmi_group' not in df.columns or not str(df['bmi_group'].dtype).startswith('category'):
        df, _ = crt_bmi_grp_12(df, method='quantile', n_groups=4)
    logit_p0 = np.log(p0/(1-p0))

    curves_total = []
    curves_chr = []

    import matplotlib.pyplot as plt
    fig, axes = plt.subplots(4, 3, figsize=(12, 14), sharex=True, sharey=False)

    for c_idx, chrom in enumerate(['13','18','21']):
        zcol = f'z{chrom}'
        gccol = f'gc{chrom}' if f'gc{chrom}' in df.columns else 'gc_content'
        for r_idx, g in enumerate(df['bmi_group'].cat.categories):
            sub = df[df['bmi_group'] == g].dropna(subset=[zcol, 'gc_content', gccol])
            if len(sub) < 30:
                continue
            y = sub[f'y{chrom}'].values if f'y{chrom}' in sub.columns else

```

```

(sub['trisomy_label'].astype(str).str.contains(chrom)).astype(int).values
    X = pd.DataFrame({
        zcol: sub[zcol].values,
        'gc_total': sub['gc_content'].values,
        'gc_chr': sub[gccol].values
    })
    X['z_gc_total'] = X[zcol]*X['gc_total']
    X['z_gc_chr'] = X[zcol]*X['gc_chr']
    try:
        clf = LogisticRegression(C=1e6, solver='lbfgs', max_iter=500)
        clf.fit(X.values, y)
        a = float(clf.intercept_[0])
        b, c, d, e, f = [float(v) for v in clf.coef_[0]]
        gc_t_med = float(sub['gc_content'].median())
        gc_j_med = float(sub[gccol].median())
        gc_grid = np.linspace(0.35, 0.65, n_points)
        denom1 = (b + e*gc_grid + f*gc_j_med)
        z_star1 = (logit_p0 - a - c*gc_grid - d*gc_j_med) / denom1
        for gi, gc_t in enumerate(gc_grid):
            curves_total.append({'染色体': chrom, 'BMI 组': g, 'GC_total':
float(gc_t), 'GC_chr(固定)': gc_j_med, f'Z*(p={p0})': float(z_star1[gi])})
        gcg_grid = np.linspace(0.35, 0.65, n_points)
        denom2 = (b + e*gc_t_med + f*gcg_grid)
        z_star2 = (logit_p0 - a - c*gc_t_med - d*gcg_grid) / denom2
        for gi, gcj in enumerate(gcg_grid):
            curves_chr.append({'染色体': chrom, 'BMI 组': g, 'GC_chr':
float(gcj), 'GC_total(固定)': gc_t_med, f'Z*(p={p0})': float(z_star2[gi])})
        ax = axes[r_idx, c_idx]
        ax.plot(gc_grid, z_star1, label='Z*(GC_total) | GC_chr=中位', color='C0')
        ax.plot(gcg_grid, z_star2, label='Z*(GC_chr) | GC_total= 中位 ',
color='C1', linestyle='--')
        ax.axhline(3.0, color='red', linestyle=':', linewidth=1)
        ax.set_title(f'Chr{chrom} {g}')
        ax.set_xlabel('GC')
        ax.set_ylabel('Z* at p={}'.format(p0))
        ax.grid(True, alpha=0.3)
    except Exception:
        continue
plt.tight_layout()
plt.savefig('输出 4/问题四_Z-GC 阈值_曲线图.png', dpi=300, bbox_inches='tight')
plt.close()

```

```

    if curves_total:
        pd.DataFrame(curves_total).to_csv('输出 4/问题四_Z-GC 阈值_曲线_按
GCtotal.csv', index=False, encoding='utf-8-sig')
    if curves_chr:
        pd.DataFrame(curves_chr).to_csv('输出 4/问题四_Z-GC 阈值_曲线_按 GCchr.csv',
index=False, encoding='utf-8-sig')
        print('已输出 Z - GC 可见判定边界曲线与图表')

def _safe_get_col_32(cols, prefix):
    for c in cols:
        if str(c).startswith(prefix):
            return c
    return None

def cmp_meth_conf_33(df, p0=0.5,
                                oof_detail_csv='输出 4/问题四_每染色体_OOF 概率与
判定.csv',
                                zgc_coef_csv='输出 4/问题四_Z-GC 阈值_按 BMI 按染
染色体.csv',
                                out_csv_per_chr='输出 4/问题四_判定方法对比_混淆
矩阵_按染色体.csv',
                                out_csv_any='输出 4/问题四_判定方法对比_混淆矩阵
_最终 ANY.csv',
                                out_png_per_chr='输出 4/问题四_判定方法对比_混淆
矩阵_按染色体.png',
                                out_png_any='输出 4/问题四_判定方法对比_混淆矩
阵_最终 ANY.png'):
    df = add_per_chr_lbl_24(df.copy())
    df = df.reset_index(drop=True)

    import os
    if not os.path.exists(oof_detail_csv):
        print(f'[compare] OOF 明细文件不存在: {oof_detail_csv}')
        print("[compare] 请先运行 run_chr_spec_rule_26() 生成该文件")
        return None, None

    try:
        oof = pd.read_csv(oof_detail_csv)
    except Exception as e:
        print(f'[compare] 读取 OOF 明细失败: {e}')

```

```

        return None, None
    try:
        coef = pd.read_csv(zgc_coef_csv)
    except Exception as e:
        print(f'[compare] 读取 Z-GC 系数失败: {e}')
        return None, None

    logit_p0 = np.log(p0/(1-p0))

    coef_map = {}
    for _, r in coef.iterrows():
        key = (str(r['染色体']), str(r['BMI 组']))
        coef_map[key] = (float(r['a']), float(r['b_Z']), float(r['c_GCt']), float(r['d_GCj']),
float(r['e_ZxGCt']), float(r['f_ZxGCj']))

    rows = []
    cm_map = {}
    preds_any_oof = []
    preds_any_zgc = []
    y_any = ((df.get('y13',0) + df.get('y18',0) + df.get('y21',0)) > 0).astype(int).values

    for chrom in ['13','18','21']:
        zcol = f'z{chrom}'
        gccol = f'gc{chrom}' if f'gc{chrom}' in df.columns else 'gc_content'
        oof_pred_col = _safe_get_col_32(list(oof.columns), f'pred_{chrom}')
        if oof_pred_col is None:
            print(f'[compare] 未在 OOF 文件中找到 pred_{chrom}, 跳过该染色体")
            continue
        pred_oof = oof[oof_pred_col].astype(int).values
        pred_zgc = np.full(len(df), np.nan)
        for i, r in df.iterrows():
            g = str(r['bmi_group']) if 'bmi_group' in r and not pd.isna(r['bmi_group']) else
None
            key = (chrom, g)
            if key not in coef_map:
                continue
            if pd.isna(r.get(zcol, np.nan)) or pd.isna(r.get('gc_content', np.nan)) or
pd.isna(r.get(gccol, np.nan)):
                continue
            a,b,c,d,e,fv = coef_map[key]
            denom = (b + e*float(r['gc_content']) + fv*float(r[gccol]))
            if abs(denom) < 1e-8:

```

```

        pred_zgc[i] = 1 if float(r[zcol]) >= 3.0 else 0
    else:
        z_star = (logit_p0 - a - c*float(r['gc_content']) - d*float(r[gccol]))/denom
        pred_zgc[i] = 1 if float(r[zcol]) >= z_star else 0
y_true = df[f'y{chrom}'].values
mask_valid = ~np.isnan(pred_zgc)
if mask_valid.sum() == 0:
    print(f"[compare] 染色体{chrom}在 Z-GC 法下无有效样本，跳过")
    continue
y1 = y_true[mask_valid]
p_o = pred_oof[mask_valid]
p_z = pred_zgc[mask_valid].astype(int)
tn, fp, fn, tp = confusion_matrix(y1, p_o, labels=[0,1]).ravel()
cm_map[('OOF', chrom)] = np.array([[tn, fp],[fn, tp]])
prec, rec, f1, _ = precision_recall_fscore_support(y1, p_o, average='binary',
zero_division=0)
rows.append({'方法':'OOF', '染色体':chrom, 'TN':int(tn), 'FP':int(fp), 'FN':int(fn),
'TP':int(tp),
               'Accuracy': (tn+tp)/len(y1), 'Precision':prec, 'Recall':rec, 'F1':f1, '有
效样本':int(len(y1))})
tn, fp, fn, tp = confusion_matrix(y1, p_z, labels=[0,1]).ravel()
cm_map[('Z-GC', chrom)] = np.array([[tn, fp],[fn, tp]])
prec, rec, f1, _ = precision_recall_fscore_support(y1, p_z, average='binary',
zero_division=0)
rows.append({'方法':'Z-GC', '染色体':chrom, 'TN':int(tn), 'FP':int(fp), 'FN':int(fn),
'TP':int(tp),
               'Accuracy': (tn+tp)/len(y1), 'Precision':prec, 'Recall':rec, 'F1':f1, '有
效样本':int(len(y1))})

preds_any_oof.append(pred_oof)
pz_full = np.where(np.isnan(pred_zgc), 0, pred_zgc).astype(int)
preds_any_zgc.append(pz_full)

if rows:
    pd.DataFrame(rows).to_csv(out_csv_per_chr, index=False, encoding='utf-8-sig')
    print(f"已输出对比混淆矩阵（按染色体）： {out_csv_per_chr}")
    try:
        chrs = [c for c in ['13','18','21'] if ('OOF', c) in cm_map and ('Z-GC', c) in
cm_map]
        if len(chrs) > 0:
            fig, axes = plt.subplots(2, len(chrs), figsize=(4*len(chrs), 8))
            for j, chrom in enumerate(chrs):

```

```

        for i, method in enumerate(['OOF', 'Z-GC']):
            ax = axes[i, j] if len(chrs) > 1 else axes[i]
            cm = cm_map[(method, chrom)]
            sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
cbar=False, ax=ax,
                                xticklabels=['Pred 0', 'Pred 1'], yticklabels=['True
0', 'True 1'])

            ax.set_title(f'{method} Chr{chrom}')
            plt.tight_layout()
            plt.savefig(out_png_per_chr, dpi=300, bbox_inches='tight')
            plt.close()
    except Exception as e:
        print(f'[compare] 绘制按染色体混淆矩阵图失败: {e}')

    if len(preds_any_oof) == 3 and len(preds_any_zgc) == 3:
        any_oof = (preds_any_oof[0] | preds_any_oof[1] | preds_any_oof[2]).astype(int)
        any_zgc = (preds_any_zgc[0] | preds_any_zgc[1] | preds_any_zgc[2]).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_any, any_oof, labels=[0, 1]).ravel()
        prec, rec, f1, _ = precision_recall_fscore_support(y_any, any_oof, average='binary',
zero_division=0)
        row_oof = {'方法': 'OOF', 'TN': int(tn), 'FP': int(fp), 'FN': int(fn), 'TP': int(tp),
                    'Accuracy': (tn+tp)/len(y_any), 'Precision': prec, 'Recall': rec, 'F1': f1, '
样本数': int(len(y_any))}
        cm_any_oof = np.array([[tn, fp], [fn, tp]])
        tn, fp, fn, tp = confusion_matrix(y_any, any_zgc, labels=[0, 1]).ravel()
        prec, rec, f1, _ = precision_recall_fscore_support(y_any, any_zgc, average='binary',
zero_division=0)
        row_zgc = {'方法': 'Z-GC', 'TN': int(tn), 'FP': int(fp), 'FN': int(fn), 'TP': int(tp),
                    'Accuracy': (tn+tp)/len(y_any), 'Precision': prec, 'Recall': rec, 'F1': f1, '
样本数': int(len(y_any))}
        pd.DataFrame([row_oof, row_zgc]).to_csv(out_csv_any, index=False,
encoding='utf-8-sig')
        print(f'已输出对比混淆矩阵（最终 ANY）： {out_csv_any}')
        try:
            cm_any_zgc = np.array([[row_zgc['TN'], row_zgc['FP']], [row_zgc['FN'],
row_zgc['TP']]])
            fig, axes = plt.subplots(1, 2, figsize=(8, 4))
            sns.heatmap(cm_any_oof, annot=True, fmt='d', cmap='Blues', cbar=False,
ax=axes[0],
                                xticklabels=['Pred 0', 'Pred 1'], yticklabels=['True 0', 'True 1'])
            axes[0].set_title('OOF ANY')
            sns.heatmap(cm_any_zgc, annot=True, fmt='d', cmap='Oranges', cbar=False,

```

```

ax=axes[1],
                                xticklabels=['Pred 0','Pred 1'], yticklabels=['True 0','True 1'])
    axes[1].set_title('Z-GC ANY')
    plt.tight_layout()
    plt.savefig(out_png_any, dpi=300, bbox_inches='tight')
    plt.close()
except Exception as e:
    print(f"[compare] 绘制 ANY 混淆矩阵图失败: {e}")
return None

def _choose_thr_spec_34(y_true: np.ndarray, y_prob: np.ndarray, target_spec: float) -> float:
    if len(y_true) == 0:
        return 0.5
    cand = np.unique(np.clip(y_prob, 0, 1))
    cand = np.concatenate(([0.0], cand, [1.0]))
    cand.sort()
    best = 1.0
    reached = False
    for t in cand:
        pred = (y_prob >= t).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_true, pred, labels=[0,1]).ravel()
        spec = tn / (tn + fp) if (tn + fp) > 0 else 0.0
        if spec >= target_spec:
            best = t
            reached = True
            break
    return best if reached else 1.0

def eval_oof_thr_strat_35(oof_detail_csv='输出 4/问题四_每染色体_OOF 概率与判定.csv',
                        out_prefix='输出 4/问题四_OOF 阈值策略
对比_', target_spec=0.90):
    import os
    if not os.path.exists(oof_detail_csv):
        print(f"[spec] OOF 明细文件不存在: {oof_detail_csv}")
        print("[spec] 请先运行 run_chr_spec_rule_26() 生成该文件")
        return

    try:
        dfp = pd.read_csv(oof_detail_csv)
    except Exception as e:

```

```

        print(f'[spec] 读取 OOF 明细失败: {e}')
        return
    if 'BMI 组' not in dfp.columns:
        print('[spec] OOF 明细缺少 BMI 组')
        return
    chrs = [c for c in ['13','18','21'] if f'OOF_{c}' in dfp.columns and f'y_{c}' in dfp.columns]
    if not chrs:
        print('[spec] 未找到 OOF 概率列')
        return
    groups = sorted(dfp['BMI 组'].astype(str).unique())

    rows_chr = []
    cm_map_youden = {}
    cm_map_spec = {}

    preds_any_youden = []
    preds_any_spec = []
    y_any = ((dfp.get('y_13',0) + dfp.get('y_18',0) + dfp.get('y_21',0)) > 0).astype(int).values

    for chrom in chrs:
        prob = dfp[f'OOF_{chrom}'].values
        y = dfp[f'y_{chrom}'].values
        g = dfp['BMI 组'].astype(str).values
        pred_youden = dfp[f'pred_{chrom}'].astype(int).values if f'pred_{chrom}' in
dfp.columns else (prob>=0.5).astype(int)
        tn, fp, fn, tp = confusion_matrix(y, pred_youden, labels=[0,1]).ravel()
        cm_map_youden[chrom] = np.array([[tn, fp],[fn, tp]])
        rows_chr.append({'策略': 'Youden', '染色体': chrom,
'TN':int(tn),'FP':int(fp),'FN':int(fn),'TP':int(tp),
'Accuracy': (tn+tp)/len(y)})

    th_map = {}
    for gg in groups:
        mask = (g==gg)
        if mask.sum() == 0:
            th_map[gg] = 1.0
            continue
        th_map[gg] = _choose_thr_spec_34(y[mask], prob[mask], target_spec)
    pred_spec = np.array([1 if prob[i] >= th_map[g[i]] else 0 for i in range(len(prob))])
    tn, fp, fn, tp = confusion_matrix(y, pred_spec, labels=[0,1]).ravel()
    cm_map_spec[chrom] = np.array([[tn, fp],[fn, tp]])
    rows_chr.append({'策略': f'Spec>={int(target_spec*100)}%', '染色体': chrom,

```

```

'TN':int(tn),'FP':int(fp),'FN':int(fn),'TP':int(tp),
    'Accuracy': (tn+tp)/len(y)})

    preds_any_youden.append(pred_youden)
    preds_any_spec.append(pred_spec)

    if rows_chr:
        pd.DataFrame(rows_chr).to_csv(out_prefix+' 按 染 色 体 .csv', index=False,
encoding='utf-8-sig')

    try:
        with mpl.rc_context({'font.family': 'SimHei'}):
            fig, axes = plt.subplots(2, len(chrs), figsize=(4*len(chrs), 8))
            for j, chrom in enumerate(chrs):
                ax = axes[0, j] if len(chrs)>1 else axes[0]
                sns.heatmap(
                    cm_map_youden[chrom], annot=True, fmt='d', cmap='Blues',
cbar=False, ax=ax,
                    xticklabels=['预测阴性','预测阳性'], yticklabels=['真实阴性','真实
阳性'])
                )
                ax.set_title(f'Youden 指数策略 染色体{chrom}')
                ax = axes[1, j] if len(chrs)>1 else axes[1]
                sns.heatmap(
                    cm_map_spec[chrom], annot=True, fmt='d', cmap='Oranges',
cbar=False, ax=ax,
                    xticklabels=['预测阴性','预测阳性'], yticklabels=['真实阴性','真实
阳性'])
                )
                ax.set_title(f'特异度 $\geq$ {int(target_spec*100)}% 策略 染色体{chrom}')
            plt.tight_layout()
            plt.savefig(out_prefix+'按染色体.png', dpi=300, bbox_inches='tight')
            plt.close()
    except Exception as e:
        print(f'[spec] 绘制按染色体图失败: {e}')

    if len(preds_any_youden)==len(chrs) and len(preds_any_spec)==len(chrs):
        any_youden = (preds_any_youden[0] | preds_any_youden[1] |
preds_any_youden[2]).astype(int)
        any_spec = (preds_any_spec[0] | preds_any_spec[1] |
preds_any_spec[2]).astype(int)
        rows_any = []

```

```

        tn, fp, fn, tp = confusion_matrix(y_any, any_youden, labels=[0,1]).ravel()
        rows_any.append({'
                                策
                                略
        ': 'Youden', 'TN':int(tn), 'FP':int(fp), 'FN':int(fn), 'TP':int(tp), 'Accuracy':(tn+tp)/len(y_any)})
        cm_any_y = np.array([[tn, fp], [fn, tp]])
        tn, fp, fn, tp = confusion_matrix(y_any, any_spec, labels=[0,1]).ravel()
        rows_any.append({'
                                策
                                略
        ': f'Spec>={int(target_spec*100)}%', 'TN':int(tn), 'FP':int(fp), 'FN':int(fn), 'TP':int(tp), 'Accuracy':
        (tn+tp)/len(y_any)})
        cm_any_s = np.array([[tn, fp], [fn, tp]])
        pd.DataFrame(rows_any).to_csv(out_prefix+' 最 终   ANY.csv',   index=False,
encoding='utf-8-sig')
        try:
            with mpl.rc_context({'font.family': 'SimHei'}):
                fig, axes = plt.subplots(1, 2, figsize=(8,4))
                sns.heatmap(
                    cm_any_y,   annot=True,   fmt='d',   cmap='Blues',   cbar=False,
ax=axes[0],
                    xticklabels=['预测阴性','预测阳性'], yticklabels=['真实阴性','真实
阳性'])
                )
                axes[0].set_title('Youden 指数策略 最终任一阳性')
                sns.heatmap(
                    cm_any_s,   annot=True,   fmt='d',   cmap='Oranges',   cbar=False,
ax=axes[1],
                    xticklabels=['预测阴性','预测阳性'], yticklabels=['真实阴性','真实
阳性'])
                )
                axes[1].set_title(f'特异度 $\geq$  {int(target_spec*100)}% 策略 最终任一阳
性')

                plt.tight_layout()
                plt.savefig(out_prefix+'最终 ANY.png', dpi=300, bbox_inches='tight')
                plt.close()
        except Exception as e:
            print(f'[spec] 绘制 ANY 图失败: {e}')

def main_36():
    print("=" * 60)
    print("问题四： 女胎异常判定的贝叶斯层级模型")
    print("=" * 60)

```

```

df = ld_prep_1()

df = exp_anl_4(df)

df, bmi_ranges = crt_bmi_grp_12(df, method='tree_md1p', n_groups=4)

sens_df = sens_anl_19(df)

try:
    ml_metrics = run_ml_grp_cv_23(df, n_splits=5, random_state=42)
except Exception as e:
    print(f'机器学习分组交叉验证出错: {e}')

try:
    fit_z_gc_thr_30(df, p0=0.5)
except Exception as e:
    print(f'生成 Z-GC 联合阈值出错: {e}')

try:
    exp_z_gc_bd_31(df, p0=0.5)
except Exception as e:
    print(f'导出 Z-GC 边界曲线出错: {e}')

try:
    thresh_df, final_df = run_chr_spec_rule_26(df, n_splits=5, random_state=42,
use_gc_layer=False)
except Exception as e:
    print(f'每染色体阈值规则计算出错: {e}')

try:
    cmp_meth_conf_33(df, p0=0.5)
except Exception as e:
    print(f'输出方法对比混淆矩阵出错: {e}')

try:
    eval_oof_thr_strat_35(oof_detail_csv='输出 4/问题四_每染色体_OOF 概率与判
定.csv',
                                                                    out_prefix='输出 4/问题四_OOF 阈值策
略对比_', target_spec=0.90)
except Exception as e:
    print(f'输出阈值策略对比出错: {e}')

try:
    write_final_sch_29(df)
except Exception as e:

```

```

print(f'生成最终判定方案出错: {e}')

if RUN_BAYES and HAS_PYMC:
    print("\n 开始贝叶斯建模...")

    trace, model = fit_bayes_14(df, chromosome='z21', n_samples=500, n_tune=500)

    if trace is not None:
        ppc = post_pred_16(trace, model, df, 'z21')

        probb_trisomy = calc_tris_prob_17(trace, model, df, 'z21', z_threshold=3.0)

        bmi_results_df, bmi_results = anl_bmi_bayes_18(trace, model, df, 'z21')

        print("\n=== 贝叶斯分析完成 ===")
        print("主要输出文件:")
        print("- 输出 4/问题四_z21_BMI 组别分析.csv")
        print("- 输出 4/问题四_z21_BMI 概率曲线.png")
        print("- 输出 4/问题四_z21_后验预测检验.png")

    else:
        print("贝叶斯建模失败，仅生成频率派分析结果")

else:
    print("\n 已跳过贝叶斯部分（或 PyMC 不可用），仅进行频率派/机器学习分析")

print("\n 使用简化方法分析 BMI 组别...")

results = []
for group_name in df['bmi_group'].cat.categories:
    group_data = df[df['bmi_group'] == group_name]

    week_bins = pd.cut(group_data['week'], bins=5)
    week_bins.name = 'week_bin'
    week_stats = group_data.groupby(week_bins).agg({
        'z21': lambda x: (x > 3).mean(),
        'week': 'mean'
    }).reset_index()
    week_stats = week_stats.rename(columns={'z21': 'pos_rate', 'week':
'week_mean'})

```

```

        high_pos_weeks = week_stats[week_stats['pos_rate'] >= 0.95]['week_mean']
        if len(high_pos_weeks) > 0:
            optimal_week = high_pos_weeks.iloc[0]
            status = f"{optimal_week:.1f} 周"
        else:
            max_pos_rate = week_stats['pos_rate'].max()
            max_week = week_stats.loc[week_stats['pos_rate'].idxmax(),
'week_mean']
            status = f"未达阈值(最高{max_pos_rate:.3f}@{max_week:.1f} 周)"

        results.append({
            'BMI 组': group_name,
            'BMI 范围': f"[{group_data['bmi'].min():.1f},
{group_data['bmi'].max():.1f})",
            '样本数': len(group_data),
            '最佳时点': status,
            'Z21 阳性率': group_data['z21'].apply(lambda x: x > 3).mean()
        })

    freq_results_df = pd.DataFrame(results)
    freq_results_df.to_csv('输出 4/问题四_频率派 BMI 分析.csv', index=False,
encoding='utf-8-sig')

    print("频率派分析结果:")
    print(freq_results_df.to_string(index=False))

    print("\n=== 问题四分析完成 ===")
    print("输出文件:")
    print("- 输出 4/问题四_探索性分析.png")
    print("- 输出 4/问题四_Z 值与协变量关系.png")
    print("- 输出 4/问题四_测序质量分析.png")
    print("- 输出 4/问题四_阈值敏感性分析.csv")
    print("- 输出 4/问题四_阈值敏感性.png")
    print("- 输出 4/问题四_ML_交叉验证_按 PID.csv")
    print("- 输出 4/问题四_ML_ROC 曲线对比.png")

    if RUN_BAYES and HAS_PYMC:
        print("- 输出 4/问题四_z21_BMI 组别分析.csv")
        print("- 输出 4/问题四_z21_BMI 概率曲线.png")
    else:
        print("- 输出 4/问题四_频率派 BMI 分析.csv")

```

```
if __name__ == "__main__":  
    main_36()
```
